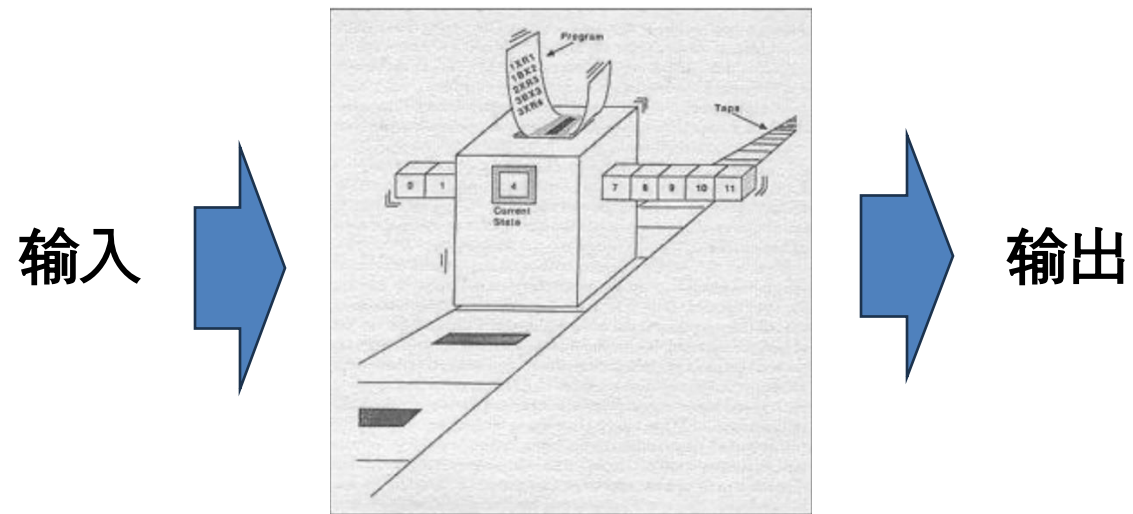


回顾

- 计算问题定义

- 给定数据输入，计算满足某种性质（可机械验证）输出的问题



- 良定义的计算过程
 - 一系列的计算步骤

算法正确 $\Leftrightarrow$ 任意输入实例，算法**终止** + 结果**正确**

回顾

- 算法的设计
 - 分治法、贪心方法、动态规划、回溯法、分支-限界法
- 算法的表示——伪代码
- 算法的确认——数学归纳法
- 算法的分析
 - 渐近上界 O
 - 渐近下界 Ω
 - 渐近紧确界 Θ



吉林大学



第四章 分治法



目录

- 4.1 一般方法
- 4.2 二分查找
- 4.3 归并排序
- 4.4 快速排序
- 4.5 斯特拉森矩阵乘法
- 4.6 二维极大点问题
- 4.7 分治法的优化
- 4.8 递归式分析方法
- 4.9 小结



目录

- 4.1 一般方法
- 4.2 二分查找
- 4.3 归并排序
- 4.4 快速排序
- 4.5 斯特拉森矩阵乘法
- 4.6 二维极大点问题
- 4.7 分治法的优化
- 4.8 递归式分析方法
- 4.9 小结



一个例子

- 上课点名，找出学号对应的姓名

有序排列

15211519	金**
16230518	董**
17220819	刘**
20231518	***
24221029	***
40230101	***
40230102	***
...	...
40230229	***

40230102	?
----------	---

一个例子

- 上课点名，找出学号对应的姓名

有序排列

15211519	金**
16230518	董**
17220819	刘**
20231518	***
24221029	***
40230101	***
40230102	***
...	...
40230229	***

蛮力枚举

40230102	?
----------	---



一个例子

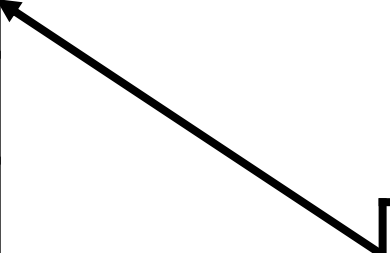
- 上课点名，找出学号对应的姓名

有序排列

15211519	金**
16230518	董**
17220819	刘**
20231518	***
24221029	***
40230101	***
40230102	***
...	...
40230229	***

蛮力枚举

40230102	?
----------	---



一个例子

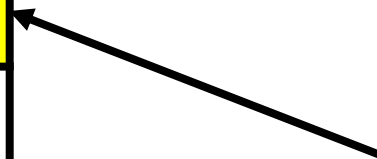
- 上课点名，找出学号对应的姓名

蛮力枚举

有序排列

15211519	金**
16230518	董**
17220819	刘**
20231518	***
24221029	***
40230101	***
40230102	***
...	...
40230229	***

40230102	?
----------	---



一个例子

- 上课点名，找出学号对应的姓名

蛮力枚举

有序排列

15211519	金**
16230518	董**
17220819	刘**
20231518	***
24221029	***
40230101	***
40230102	***
...	...
40230229	***

40230102	?
----------	---



一个例子

- 上课点名，找出学号对应的姓名

有序排列

15211519	金**
16230518	董**
17220819	刘**
20231518	***
24221029	***
40230101	***
40230102	***
...	...
40230229	***

蛮力枚举

40230102 | ?

思考: $T(n) = ?$

一个例子

- 上课点名，找出学号对应的姓名

有序排列

15211519	金**
16230518	董**
17220819	刘**
20231518	***
24221029	***
40230101	***
40230102	***
...	...
40230229	***

蛮力枚举

40230102 | ?

思考: $T(n) = n$

一个例子

- 上课点名，找出学号对应的姓名

有序排列	15211519	金**
	16230518	董**
	17220819	刘**
	20231518	***
	24221029	***
	40230101	***
	40230102	***
	...	...
	40230229	***

蛮力枚举

40230102	?
----------	---

$$T(n) = n$$

思考：有更快的方法吗？

一个例子

- 上课点名，找出学号对应的姓名

注意到学号序列是有序的！

有序排列

15211519	金**
16230518	董**
17220819	刘**
20231518	***
24221029	***
40230101	***
40230102	***
...	...
40230229	***

40230102	?
----------	---

一个例子

- 上课点名，找出学号对应的姓名

15211519	金**
16230518	董**
17220819	刘**
...	...
40230123	***

40230101	***
40230102	***
...	...
40230229	***

有序排列

注意到学号序列是有序的！

- 可以将该问题拆分成k个子问题，这里假设k=2

40230102	?
----------	---

一个例子

- 上课点名，找出学号对应的姓名

15211519	金**
16230518	董**
17220819	刘**
...	...
40230123	***

有序排列

40230101	***
40230102	***
...	...
40230229	***

注意到学号序列是有序的！

- 可以将该问题拆分成k个子问题，这里假设k=2

查询姓名

40230102	?
----------	---

一个例子

- 上课点名，找出学号对应的姓名

15211519	金**
16230518	董**
17220819	刘**
...	...
40230123	***

40230101	***
40230102	***
...	...
40230229	***

有序排列

注意到学号序列是有序的！

- 可以将该问题拆分成k个子问题，这里假设k=2

40230102	?
----------	---

查询姓名

一个例子

- 上课点名，找出学号对应的姓名

15211519	金**
16230518	董**
17220819	刘**
...	...
40230123	***

40230101	***
40230102	***
...	...
40230229	***

有序排列

注意到学号序列是有序的！

- 可以将该问题拆分成k个子问题，这里假设k=2

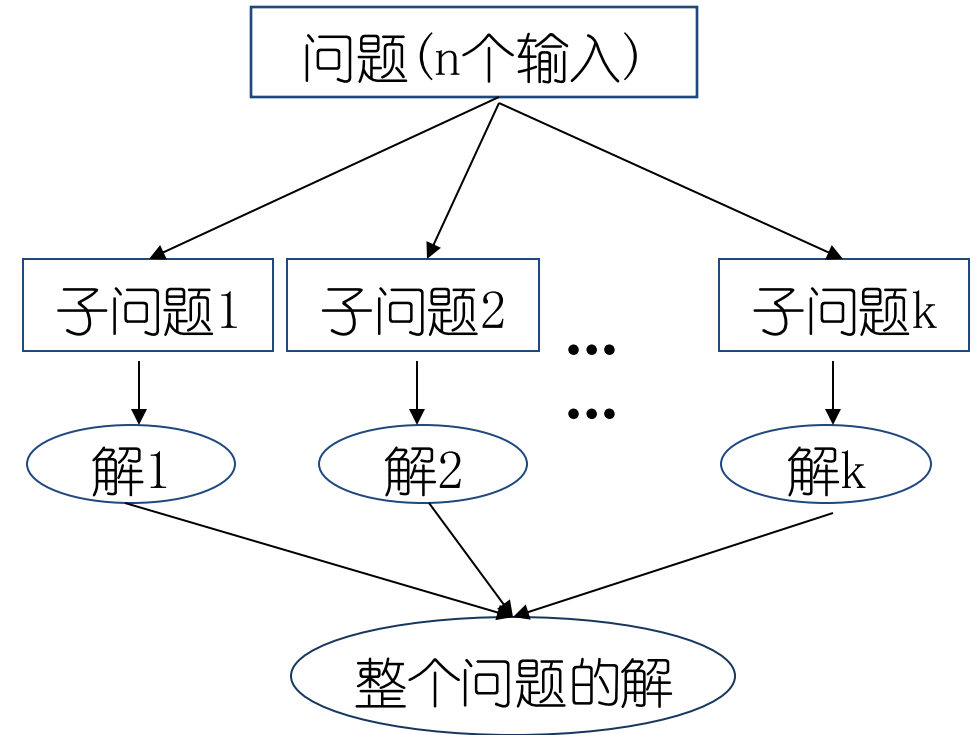
40230102	?
----------	---

查询姓名

子问题和原问题形式完全相同
分治法！

分治法适用的问题

- n 取值相当大，直接求解往往非常困难，甚至无法求出。
- 将 n 个输入分解成 k 个不同子集，得到 k 个不同的可独立求解的子问题。
- 在求出子问题的解之后，能找到适当的方法把它们合并成整个问题的解。



分治法的求解思想

- 思想：将整个问题分成若干个小问题后分而治之。
 - 分(Divide)：子问题与原问题具有相同的特征，但规模更小。
 - 治(Conquer)：反复使用分治策略，直到可以直接求解子问题为止。
 - 合(Combine)：将子问题的解组合成原问题的解。

分治法天然适
合递归实现



子问题的个数K?

- 显然有 $K > 1$, K越大越好么?

- 小故事

分与合的代价

- 短视频团队要剪辑一段10分钟的视频。组长小A提议分成5段，每人负责2分钟，大家很快完成。实习生小B建议分成20段，每人30秒，小A摇头：“段数太多，衔接麻烦，效率反而低了。”数据分析师小C提出分成8段，每人1分多钟，最后2分钟一起调整。团队尝试后，发现这样既高效又保证质量。小A总结：“分治法就像剪辑视频，分段太少任务重，分段太多太琐碎，找到合适的段数才能事半功倍！”

思考：子问题的规模相近或不相近是否会影响算法效率？

分治策略DANDC的抽象化控制

```
procedure DANDC(p, q)
//1≤p≤q≤n//
if SMALL(p,q)
then
    return(G(p,q))
else
    m←DIVIDE(p,q)
    return(COMBINE( DANDC(p,m),
        DANDC(m+1,q)))
end
end DANDC
```

原问题为A(1:n) , 调用函数DANDC(1,n)

- **SMALL**: 判断输入规模 $q-p+1$ 是否足够小, 可直接求解
- **G**: 求解该规模问题的函数
- **DIVIDE**: 分割函数, $p \leq m \leq q$, 原问题被分为A(p:m)和A(m+1:q)两个子问题
- **COMBINE**: 合并函数, 将两个子问题的解合并为原问题的解

分治策略DANDC的时间复杂度

- 假设所分成的两个子问题的输入规模大致相等，则DANDC的计算时间可表示为：

$$T(n) = \begin{cases} g(n) & n \text{ 足够小} \\ 2T(n/2) + f(n) & \text{否则} \end{cases}$$

- $T(n)$ 是输入规模为 n 的分治策略的计算时间
- $g(n)$ 是对足够小的输入规模能直接计算出答案的时间
- $f(n)$ 是COMBINE函数合成原问题解的计算时间



目录

- 4.1 一般方法
- 4.2 二分查找
- 4.3 归并排序
- 4.4 快速排序
- 4.5 斯特拉森矩阵乘法
- 4.6 二维极大点问题
- 4.7 分治法的优化
- 4.8 递归式分析方法
- 4.9 小结



问题描述

- 查找问题：已知一个非降序排列 $A_1, A_2, \dots, A_n$ ，判定给定元素 x 是否在其中。若是，则找出 x 位置，并将下标赋给变量 j ；否则， j 置成0。
- 输入：非降序数组 $A[1\dots n]$ ，元素 x
- 输出：若 x 在 A 中， x 在 A 中的下标，否则0



分治法建模

- 将查找问题表示为： $I=(n, a_1, \dots, a_n, x)$
 - 分：选取一个下标 k ，可得到三个子问题
 - $I_1=(k-1, a_1, \dots, a_{k-1}, x)$
 - $I_2=(1, a_k, x)$
 - $I_3=(n-k, a_{k+1}, \dots, a_n, x)$
 - 治：
 - 若 $x = a_k$ ， $j \leftarrow k$ ，算法结束
 - 若 $x < a_k$ ，在子问题 I_1 中求解
 - 若 $x > a_k$ ，在子问题 I_3 中求解
 - 合：
 - 当前子问题的解就是 I 的解

取中间位置

思考1： $k=?$

思考2：合并策略是否正确？即为什么可以忽略掉其他子问题？



算法实例

$n=9$

A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]
-15	-6	0	7	9	23	54	82	101

情况1: $x=9$



情况2: $x=-15$



情况3: $x=101$



情况4: $x=8$



思考: 对称的元素为什么比较次数不同?

算法描述

```
BINSRCH(A, n, x, j)
  low ← 1;  high ← n
  while low ≤ high do
    mid ←  $\lfloor (low + high) / 2 \rfloor$   //取中间值
    case
      :x < A(mid): high ← mid - 1 //前半
      :x > A(mid): low ← mid + 1 //后半
      :else: j ← mid; return; //查找成功结束
    end
  end
  j ← 0 //查找失败
end BINSRCH
```



算法正确性证明

证明思路:

- 1 检验到参数的每类取值
- 2 检验到算法的每个分支

- 如果 $n=0$ ，则不进入循环， $j=0$ ，算法终止。
- 否则就会进入循环与数组A中的元素进行比较。
- 成功情况：
 - 若 $x=A[mid]$ ，则 $j=mid$ ，查找成功，算法终止。
 - 否则，若 $x < A[mid]$ ，则 x 根本不可能在 $mid \sim high$ ，查找范围缩小到 $low \sim mid-1$ 。反之， $x > A[mid]$ 时亦然。
- 不成功情况：
 - 因为 low ， $high$ 和 mid 都是整形变量，按算法所述方式缩小查找范围总可以在有限步内使 $low > high$ ，说明 x 不在A中，退出循环， $j=0$ ，算法终止。

算法的计算时间分析

- A有 n 个元素， x 和A中元素比较
 - 成功查找： n 种情况
 - 不成功查找： $n+1$ 种情况
- 分析算法时间复杂度
 - 成功查找：最好、平均、最坏情况下
 - 不成功查找：最好、平均、最坏情况下

元素比较

思考：算法的基本运算是什麼？

用一棵二元比较树来描述元素比较的过程，即算法的执行过程

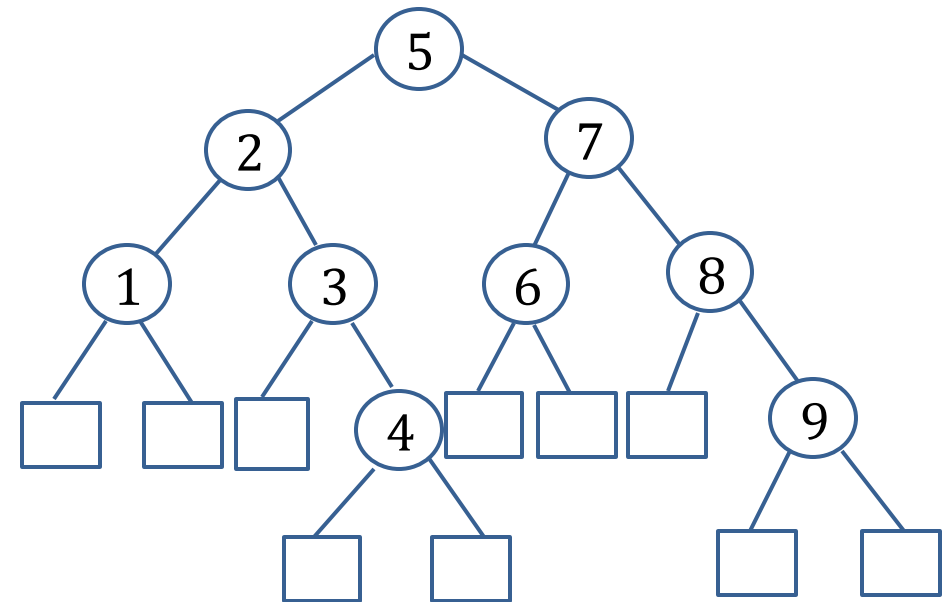


二元比较树

- 描述二分查找算法执行过程的二元比较树由内结点和外结点组成
 - 内结点记录一个mid值，表示一次元素的比较
 - 外结点表示不成功查找的一种情况
 - 一条路径表示一个元素的比较序列

$n=9$

A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]
-15	-6	0	7	9	23	54	82	101

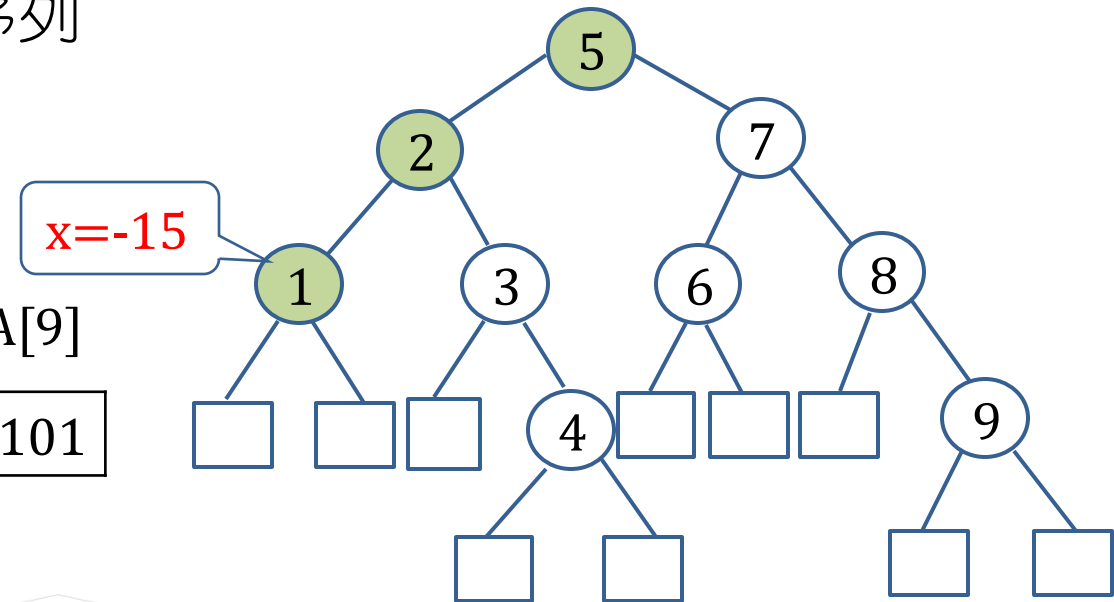


二元比较树

- 描述二分查找算法执行过程的二元比较树由内结点和外结点组成
 - 内结点记录一个mid值，表示一次元素的比较
 - 外结点表示不成功查找的一种情况
 - 一条路径表示一个元素的比较序列

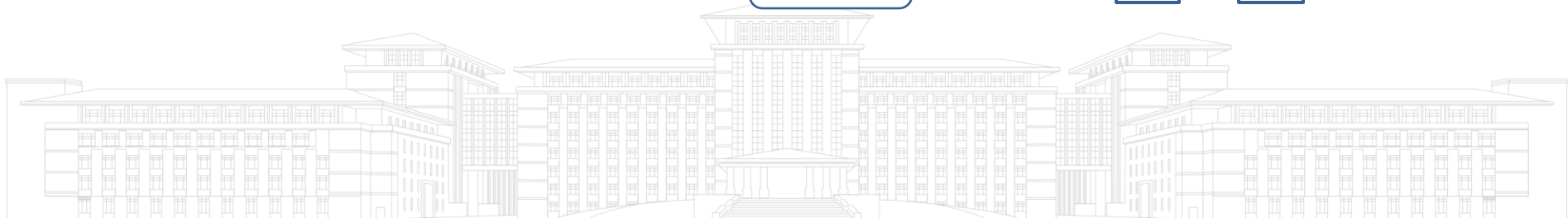
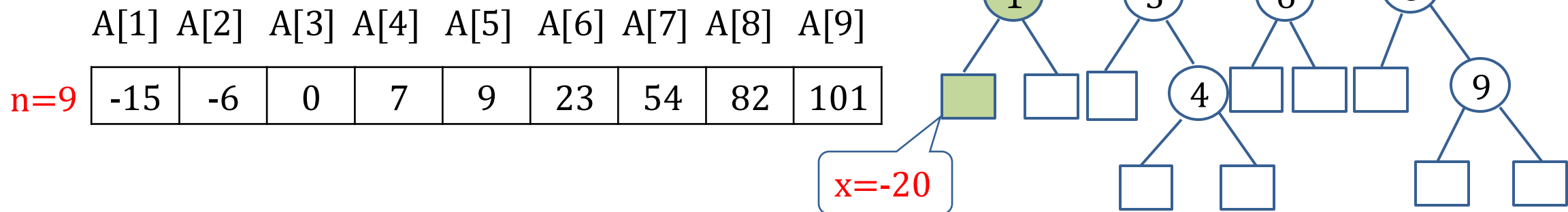
$n=9$

A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]
-15	-6	0	7	9	23	54	82	101



二元比较树

- 描述二分查找算法执行过程的二元比较树由内结点和外结点组成
 - 内结点记录一个mid值，表示一次元素的比较
 - 外结点表示不成功查找的一种情况
 - 一条路径表示一个元素的比较序列

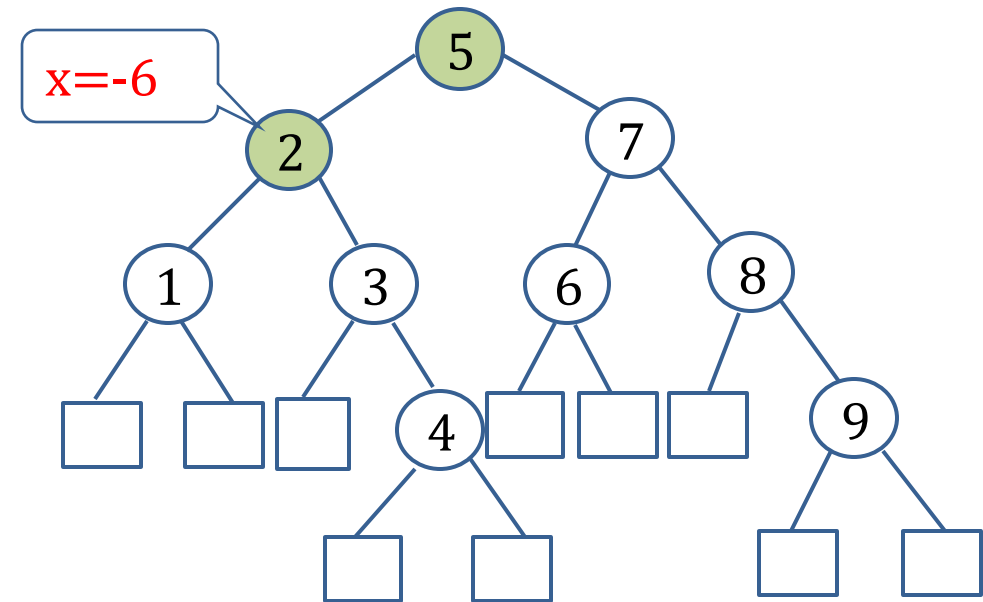


二元比较树

- 描述二分查找算法执行过程的二元比较树由内结点和外结点组成
 - 内结点记录一个mid值，表示一次元素的比较
 - 外结点表示不成功查找的一种情况
 - 一条路径表示一个元素的比较序列

$n=9$

A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]
-15	-6	0	7	9	23	54	82	101

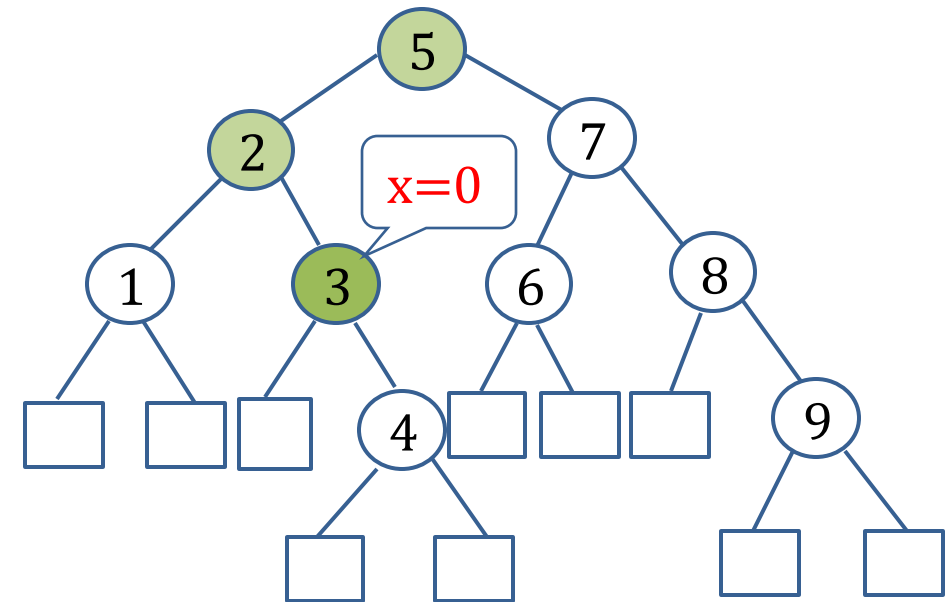


二元比较树

- 描述二分查找算法执行过程的二元比较树由内结点和外结点组成
 - 内结点记录一个mid值，表示一次元素的比较
 - 外结点表示不成功查找的一种情况
 - 一条路径表示一个元素的比较序列

$n=9$

A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]
-15	-6	0	7	9	23	54	82	101

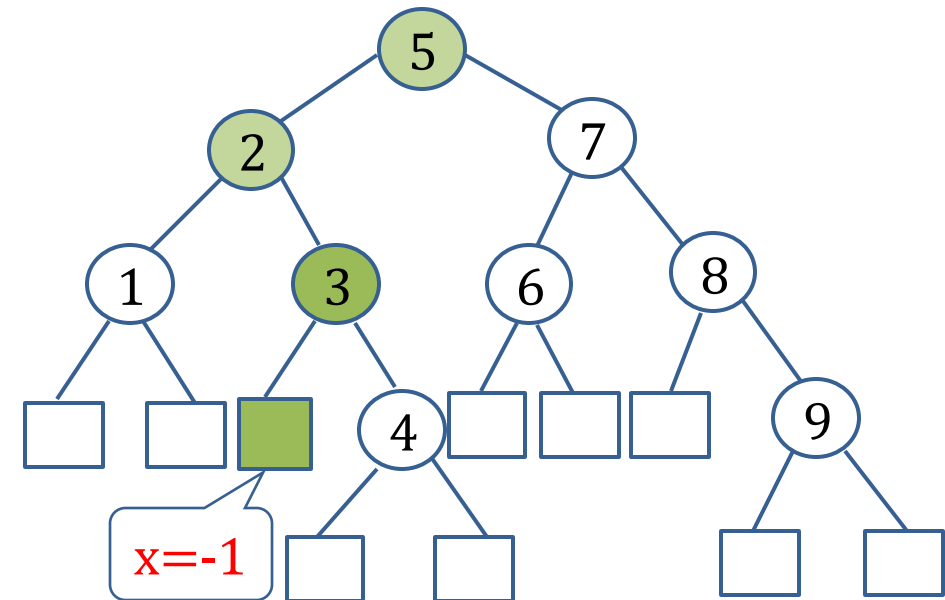


二元比较树

- 描述二分查找算法执行过程的二元比较树由内结点和外结点组成
 - 内结点记录一个mid值，表示一次元素的比较
 - 外结点表示不成功查找的一种情况
 - 一条路径表示一个元素的比较序列

$n=9$

A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]
-15	-6	0	7	9	23	54	82	101

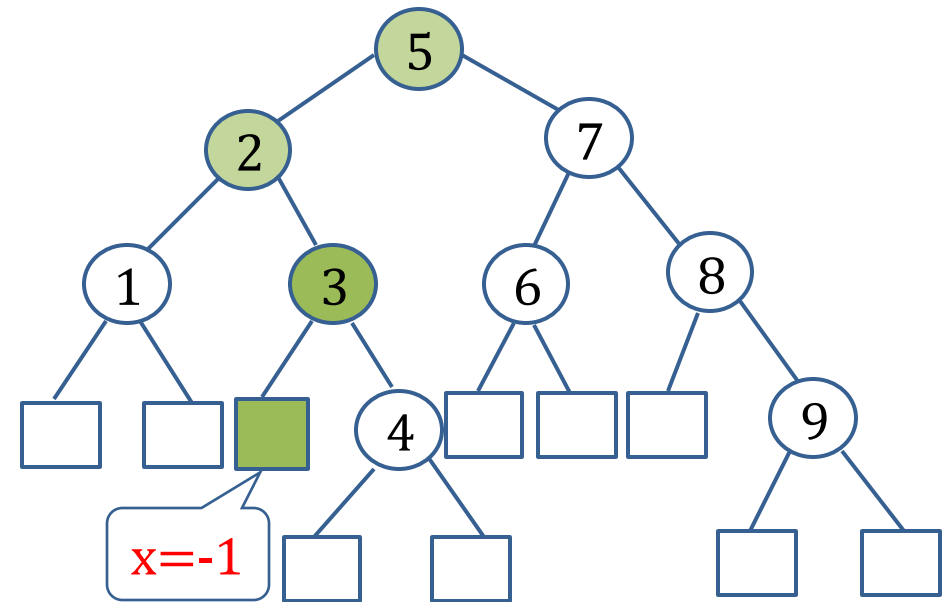


二元比较树

- 描述二分查找算法执行过程的二元比较树由内结点和外结点组成
 - 内结点记录一个mid值，表示一次元素的比较
 - 外结点表示不成功查找的一种情况
 - 一条路径表示一个元素的比较序列

$n=9$

A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]
-15	-6	0	7	9	23	54	82	101

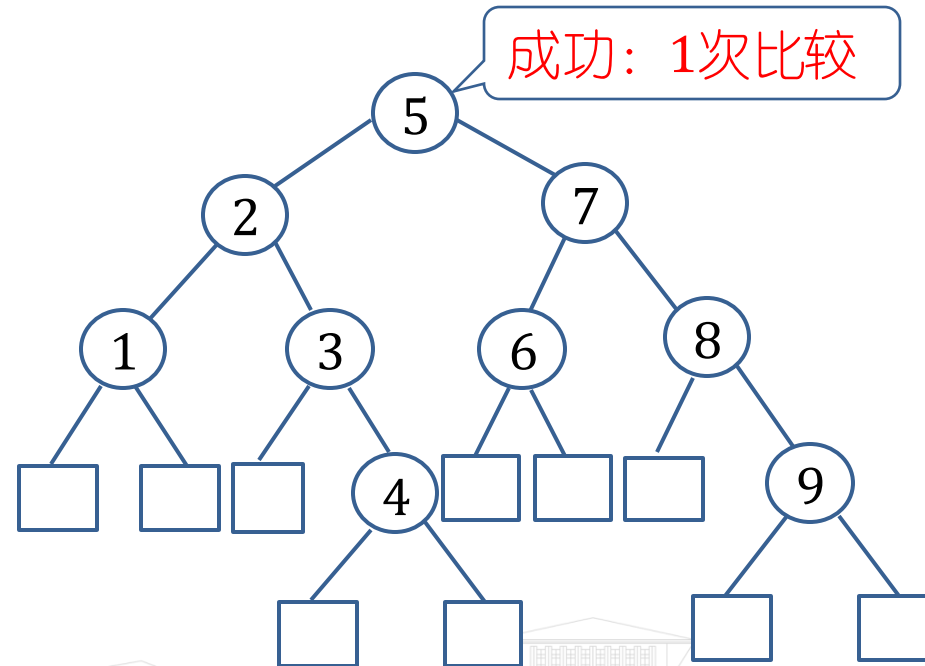


思考：二元比较树的结构受问题实例影响么？

算法的执行过程实质上就是x与一系列中间元素A(mid)的比较过程。

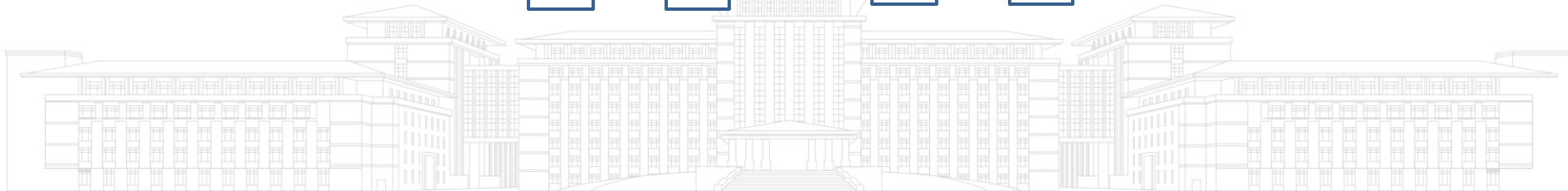
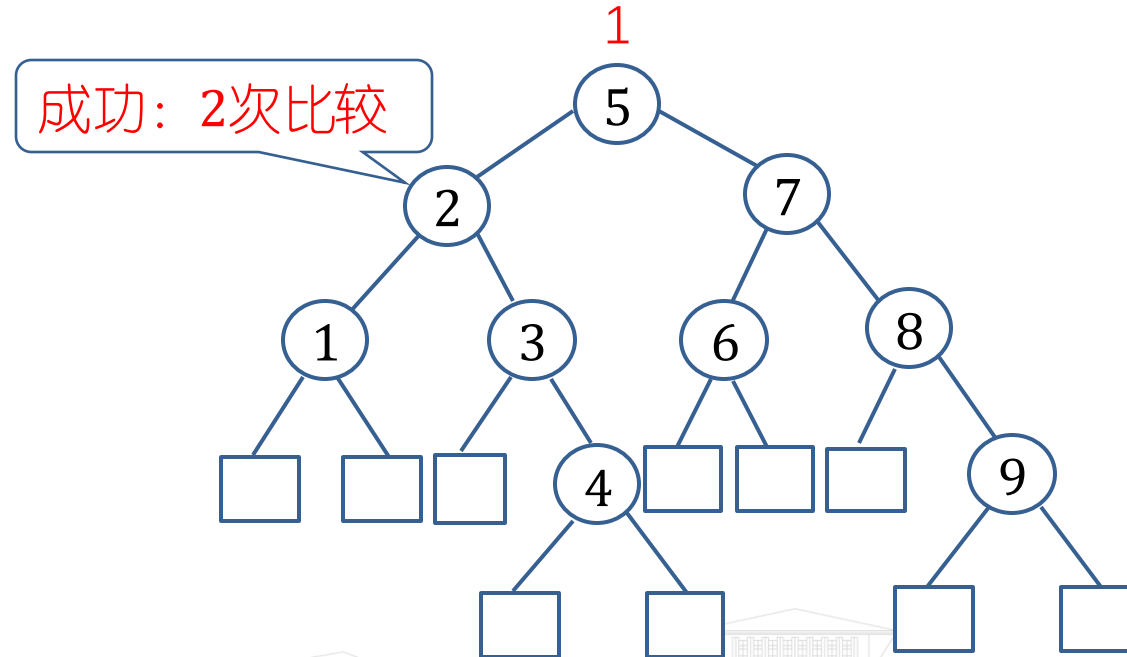
二元比较树分析

- 令根在1级，内节点最大级数为 k
- 成功：元素比较次数等于内节点级数，最少1次，最多 k
- 失败：元素比较次数等于外节点级数-1，最少 $k-1$ 次，最多 k 次



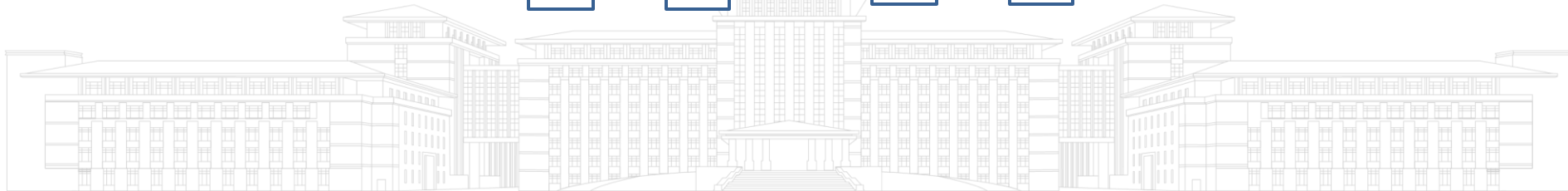
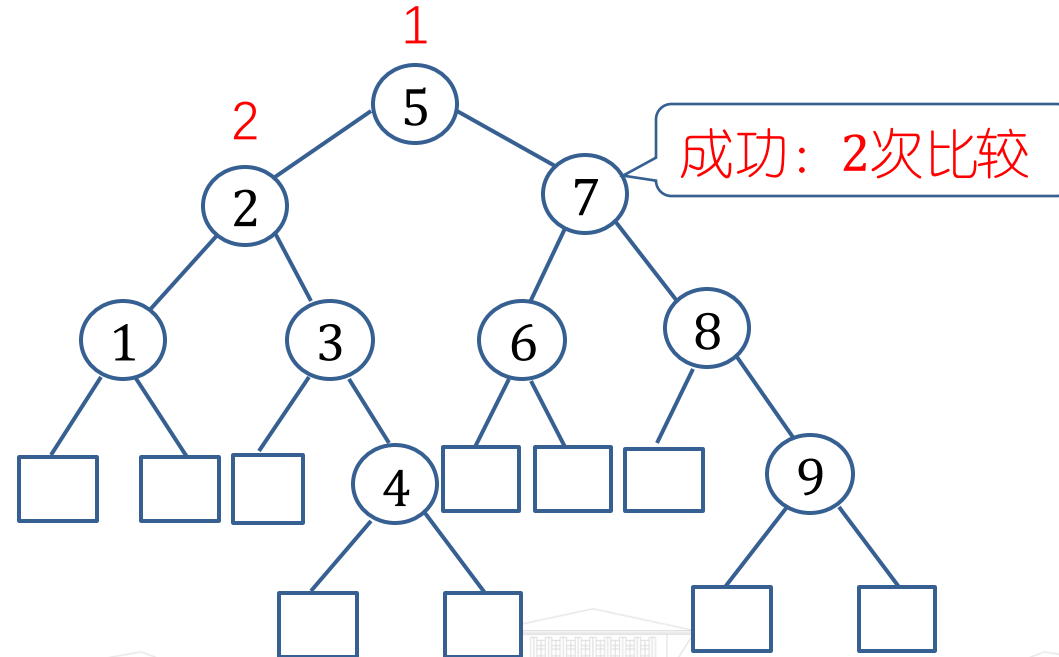
二元比较树分析

- 令根在1级，内节点最大级数为 k
- 成功：元素比较次数等于内节点级数，最少1次，最多 k
- 失败：元素比较次数等于外节点级数-1，最少 $k-1$ 次，最多 k 次



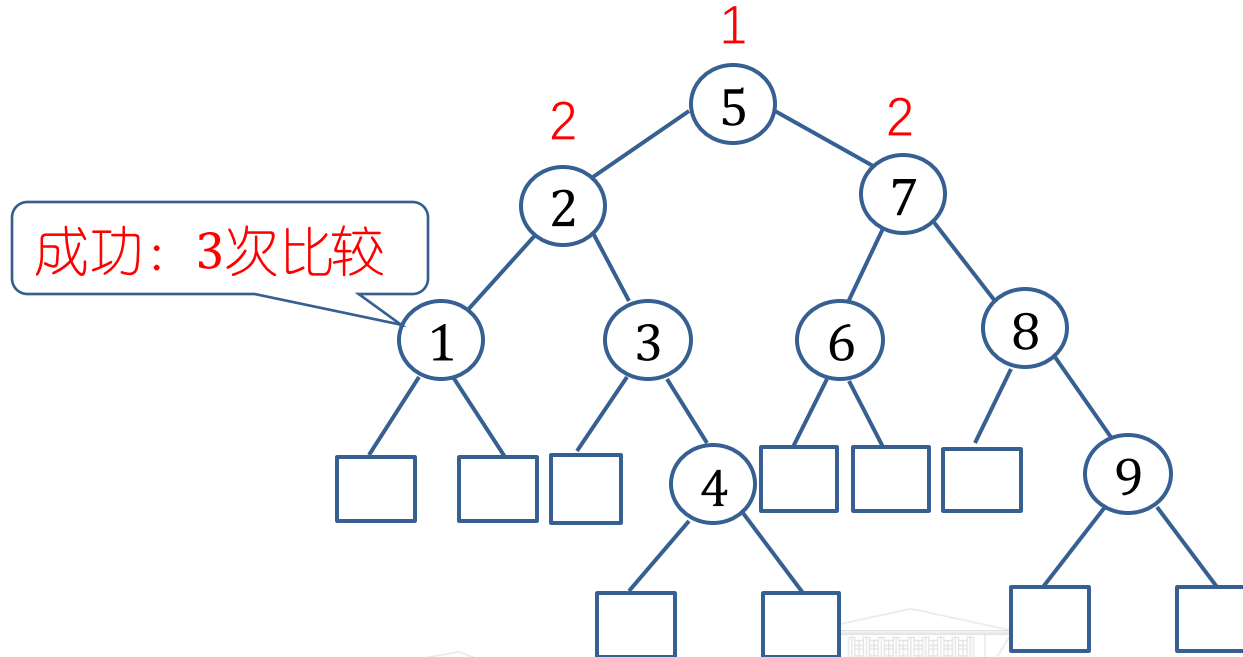
二元比较树分析

- 令根在1级，内节点最大级数为 k
- 成功：元素比较次数等于内节点级数，最少1次，最多 k
- 失败：元素比较次数等于外节点级数-1，最少 $k-1$ 次，最多 k 次



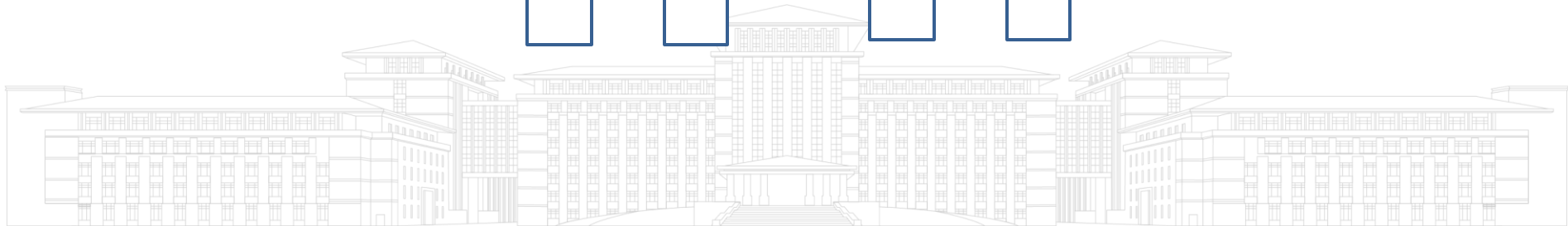
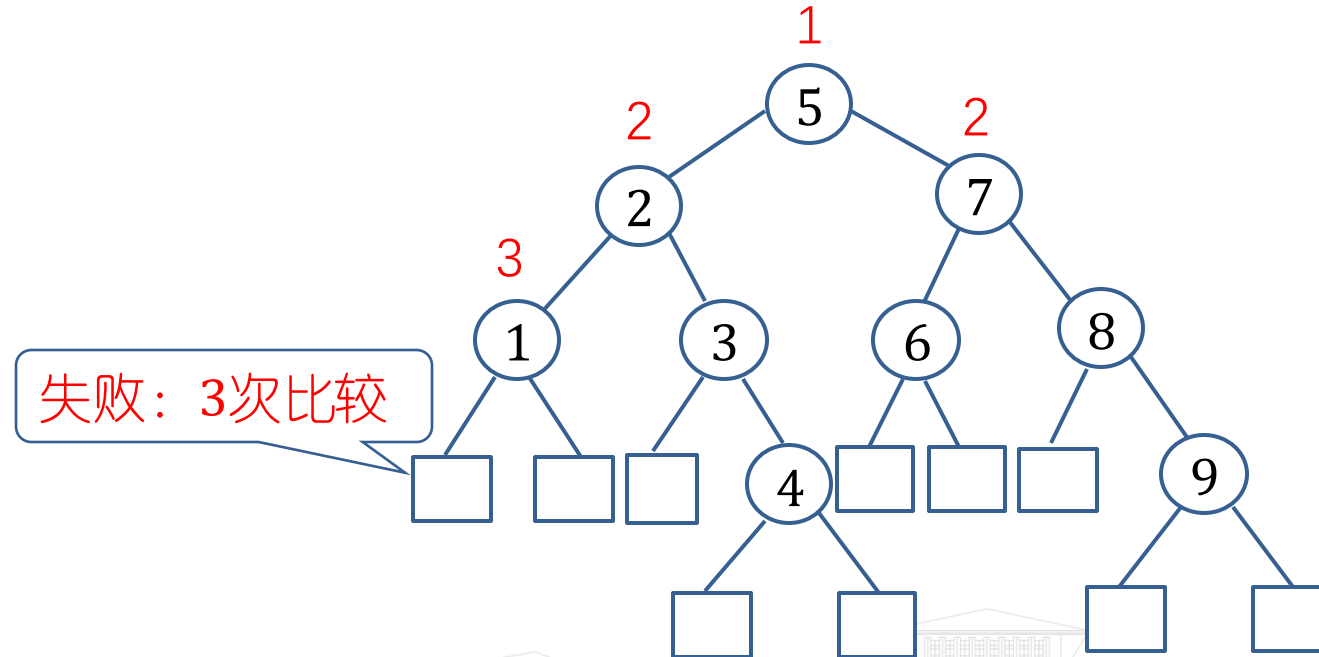
二元比较树分析

- 令根在1级，内节点最大级数为 k
- 成功：元素比较次数等于内节点级数，最少1次，最多 k
- 失败：元素比较次数等于外节点级数-1，最少 $k-1$ 次，最多 k 次



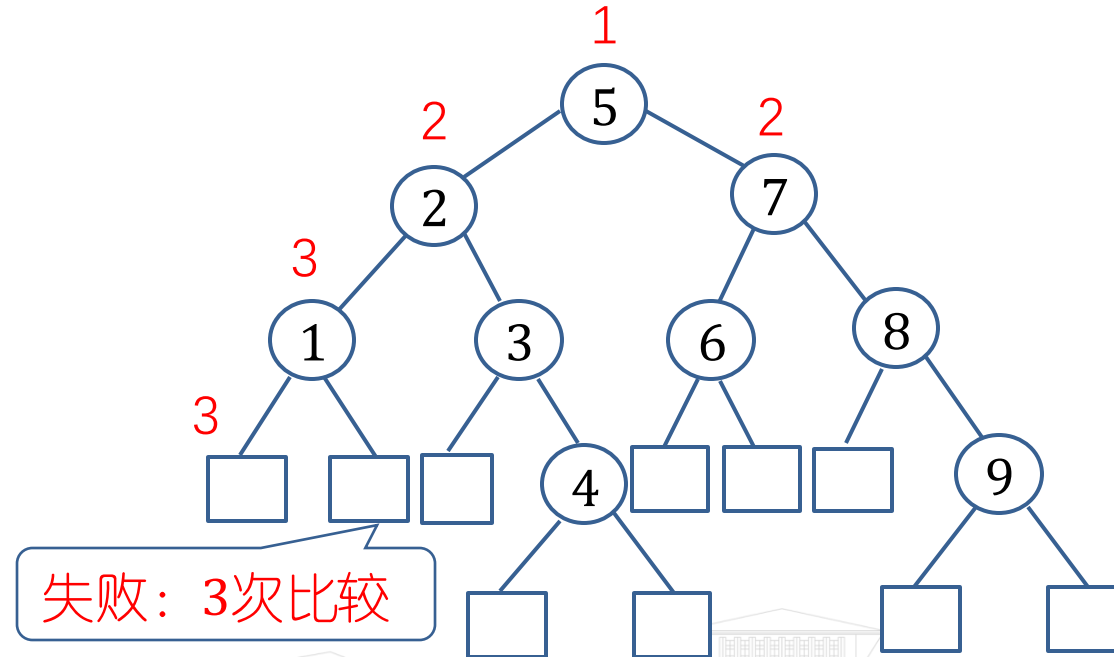
二元比较树分析

- 令根在1级，内节点最大级数为 k
- 成功：元素比较次数等于内节点级数，最少1次，最多 k
- 失败：元素比较次数等于外节点级数-1，最少 $k-1$ 次，最多 k 次



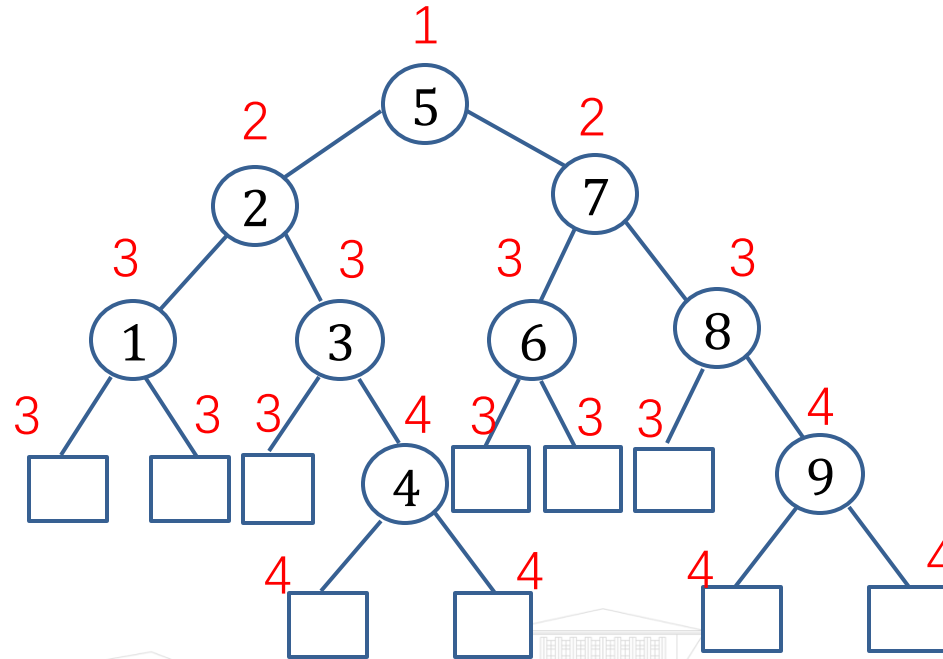
二元比较树分析

- 令根在1级，内节点最大级数为 k
- 成功：元素比较次数等于内节点级数，最少1次，最多 k
- 失败：元素比较次数等于外节点级数-1，最少 $k-1$ 次，最多 k 次



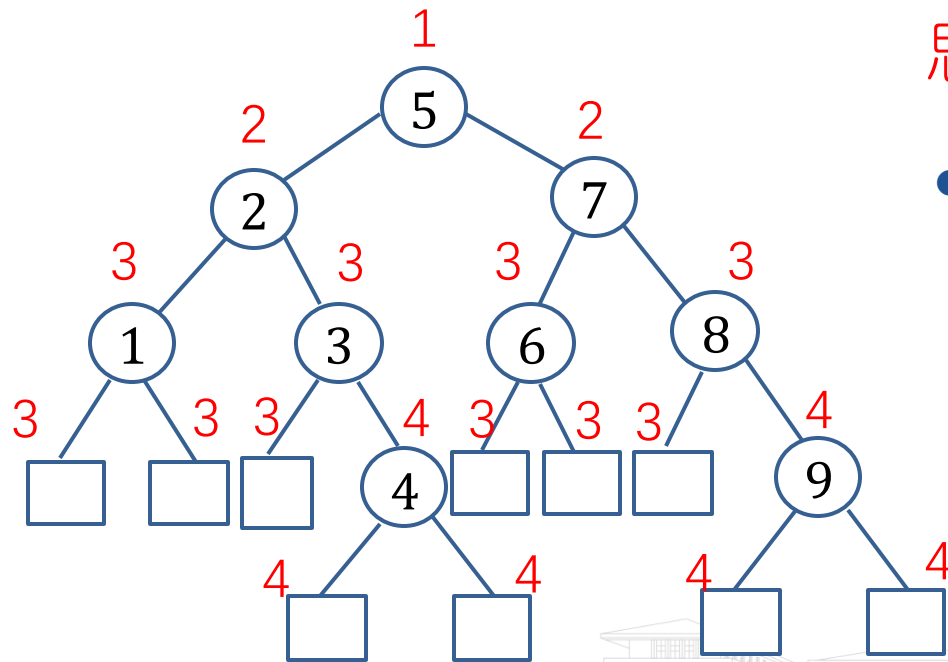
二元比较树分析

- 令根在1级，内节点最大级数为 k
- 成功：元素比较次数等于内节点级数，最少1次，最多 k
- 失败：元素比较次数等于外节点级数-1，最少 $k-1$ 次，最多 k 次



二元比较树分析

- 令根在1级，内节点最大级数为k
- 成功：元素比较次数等于内节点级数，最少1次，最多k
- 失败：元素比较次数等于外节点级数-1，最少k-1次，最多k次



思考：平均比较次数？

• n=9:

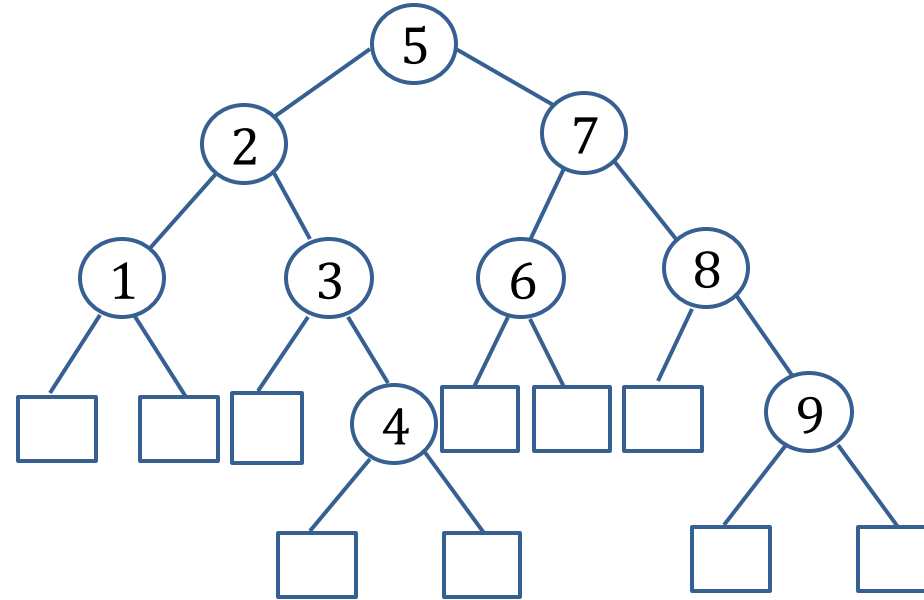
- 成功比较总数 = $1 + 2 \times 2 + 3 \times 4 + 4 \times 2 = 25$
- 失败比较总数 = $3 \times 6 + 4 \times 4 = 34$

	最好	最坏	平均
成功	1	4	25/9
失败	3	4	34/10

二元比较树分析

n是输入规模

- 内节点个数n与k之间的关系：
 - 内节点最大级数为k的二元比较树，其k-1层必为满。
 - 故 $2^{k-1} \leq n < 2^k$ 为什么？



$$k-1 \leq \log n \Rightarrow k \leq \log n + 1, \text{ 又 } k > \log n$$

$$\Rightarrow \log n < k \leq \log n + 1 \Rightarrow k = \lfloor \log n \rfloor + 1$$

$$k = \Theta(\log n)$$



算法BINSRCH的时间复杂度分析

定理4.1: 若 n 在区域 $[2^{k-1}, 2^k)$ 中, 则对于一次成功的查找, 二分查找算法至多作 k 次比较, 而对于一次不成功的查找, 或者作 $k-1$ 次比较或者作 k 次比较。

● 证明

$$k = \Theta(\log n)$$

- 考察二分查找算法执行过程的二元比较树: 成功查找在内结点终止, 不成功查找在外结点终止。当 $2^{k-1} \leq n < 2^k$ 时, 内结点数级数 $\leq k$ (根在1级), 外结点数级数在 k 和 $k+1$ 级。又知成功查找所需要的元素比较次数=级数, 不成功查找所需要的元素比较次数=级数-1。定理得证。

思考: 基于该定理可知算法在哪些情况下的时间复杂度?

成功: 最好、最坏
失败: 最好、最坏、平均

算法BINSRCH的时间复杂度和空间复杂度

• 时间

计算时间	最好情况	最坏情况	平均情况
成功查找	$\Theta(1)$	$\Theta(\log n)$	?
失败查找	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$

• 空间

- 用 n 个位置存放数组 A ，还有 $low, high, mid, x, j$ 五个变量需要存储，共需空间 $n+5 = \Theta(n)$



不成功查找平均情况下的时间复杂度？

定理4.1: 若 n 在区域 $[2^{k-1}, 2^k)$ 中, 则对于一次成功的查找, 二分查找算法至多作 k 次比较, 而对于一次不成功的查找, 或者作 $k-1$ 次比较或者作 k 次比较。

- 每次不成功都查找需要 $k-1$ 次或 k 次比较
- 不成功查找平均情况下的时间复杂度为 $\Theta(k) = \Theta(\log n)$, 记为 $U(n)$



成功查找平均情况下的时间复杂度？

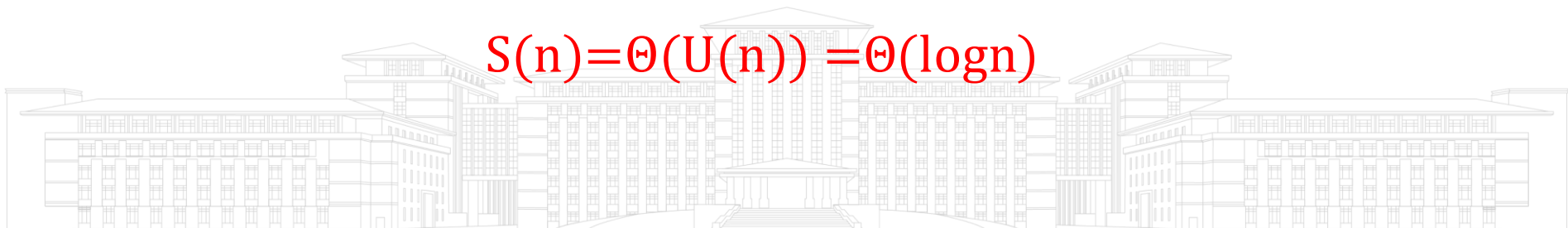
思想：利用内外节点到根的距离和之间的一种简单关系，由不成功查找的平均比较数求出成功查找的平均比较数。

- 内部路径长度I：根到所有内节点的距离之和；
- 外部路径长度E：根到所有外节点的距离之和；
- 容易证明 $E=I+2n$ ；
- 成功查找的平均比较次数 $S(n)=(I/n)+1$ ；
- 不成功查找的平均比较次数 $U(n)=E/(n+1)$ ；

$$E = (n+1)U(n) = I + 2n = n(S(n)-1) + 2n$$

$$\Rightarrow S(n) = \frac{n+1}{n}U(n) - 1 = \left(1 + \frac{1}{n}\right)U(n) - 1$$

$$S(n) = \Theta(U(n)) = \Theta(\log n)$$



算法BINSRCH的时间复杂度和空间复杂度

• 时间

计算时间	最好情况	最坏情况	平均情况
成功查找	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
失败查找	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$

• 空间

- 用 n 个位置存放数组 A ，还有 $low, high, mid, x, j$ 五个变量需要存储，共需空间 $n+5 = \Theta(n)$



算法结束后的思考

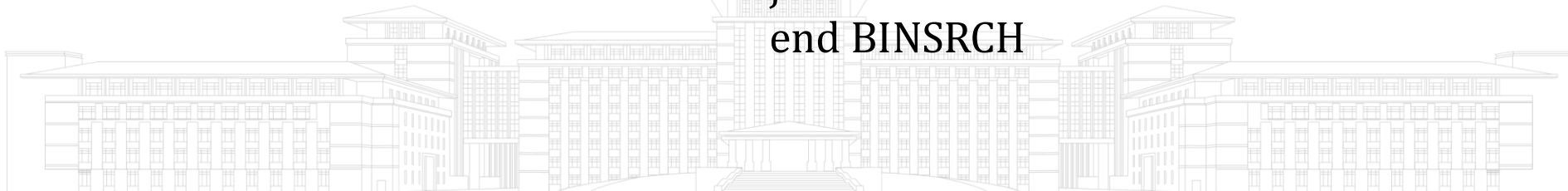
- 二分思想下的查找算法的时间复杂度是固定不变的么？时间复杂度会受到算法描述细节的影响吗？
- BINSRCH算法是解决查找问题的最优算法么？是否可能进一步优化？



算法BINSRCH的变型

- case语句用if语句来替换。一次关键字比较不是三种分支，而是两种分支。
- 最坏情况下，x总被比较两次。

```
procedure BINSRCH(A, n, x, j)
  low←1; high←n
  while low≤high do
    mid ← ⌊(low+high)/2⌋
    if x<A(mid)
      then high←mid-1
    else if x>A(mid)
      then low←mid+1
    else j←mid; return;
    end
  end
end
j←0
end BINSRCH
```



算法BINSRCH的变型

```
procedure BINSRCH1(A, n, x, j)
  low←1; high←n+1 //high总比可能的取值大1
  while low<high-1 do
    mid← $\lfloor (low+high)/2 \rfloor$ 
    if x<A(mid) //在循环中只比较一次
      then high←mid
      else low←mid //x≥A(mid)
    end
  end
  if x=A(low) then j←low //x出现
  else j←0 //x不出现
end BINSRCH1
```



Diagram illustrating a binary search step on an array A of size $n=9$. The array elements are: $A[1] = -15$, $A[2] = -6$, $A[3] = 0$, $A[4] = 7$, $A[5] = 9$, $A[6] = 23$, $A[7] = 54$, $A[8] = 82$, $A[9] = 101$.

The search range is defined by low and $high$ pointers. The current value being compared is $x=9$. The middle element $A[mid]$ is 9 . Since $x \geq 9$, the search range is updated to mid to $high$.



算法BINSRCH1实例

```

while low < high - 1 do
    mid ← ⌊(low + high) / 2⌋
    if x < A[mid] // 在循环中只比较一次
        then high ← mid
        else low ← mid // x ≥ A[mid]
    end
end

```

Diagram illustrating a binary search step on an array A of size $n=9$. The target value is $x=9$.

	$A[1]$	$A[2]$	$A[3]$	$A[4]$	$A[5]$	$A[6]$	$A[7]$	$A[8]$	$A[9]$
	-15	-6	0	7	9	23	54	82	101

The search range is defined by low and $high$. The middle element is $A[mid]$.

Since $x < A[mid]$ ($9 < 54$), the search range is updated to the left half: $high = mid - 1$.

算法BINSRCH1实例

```

while low < high - 1 do
    mid ← ⌊(low + high) / 2⌋
    if x < A[mid] // 在循环中只比较一次
        then high ← mid
        else low ← mid // x ≥ A[mid]
    end
end

```

Diagram illustrating a binary search step on an array A of size $n=9$. The array elements are: $A[1] = -15$, $A[2] = -6$, $A[3] = 0$, $A[4] = 7$, $A[5] = 9$, $A[6] = 23$, $A[7] = 54$, $A[8] = 82$, $A[9] = 101$. The search key is $x=9$. The current search range is from low to $high$, with mid pointing to the middle element $A[6] = 23$. Since $x < 23$, the search range is updated to the left half, from low to $mid-1$.

x=9 **n=9**

A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]
-15	-6	0	7	9	23	54	82	101

↑ ↑
low high

A[low] = x! 成功查找



算法BINSRCH的变型

```
procedure BINSRCH1(A, n, x, j)
  low←1; high←n+1 //high总比可能的取值大1
  while low<high-1 do
    mid←⌊(low+high)/2⌋
    if x<A(mid) //在循环中只比较一次
      then high←mid
      else low←mid //x≥A(mid)
    end
  end
  if x=A(low) then j←low //x出现
    else j←0 //x不出现
end BINSRCH1
```

最好、最坏和平均情况时间对于成功和不成功的查找都是 $\Theta(\log n)$ 。

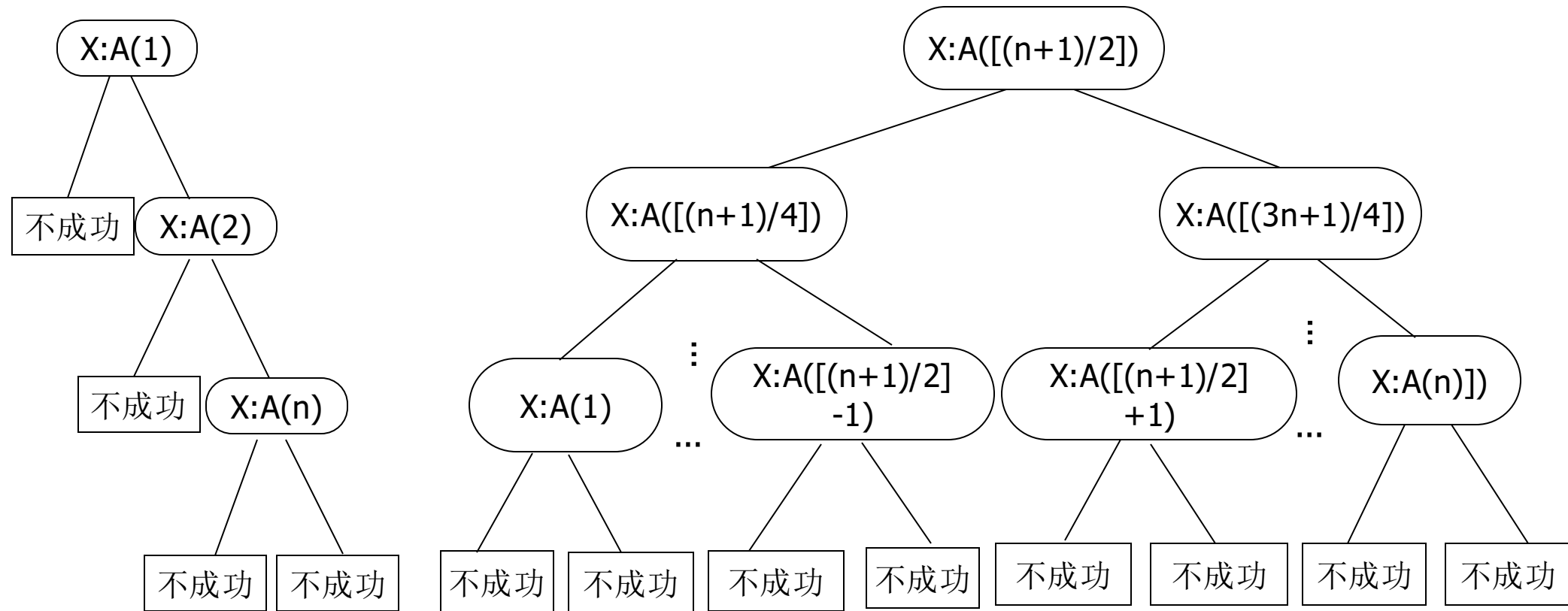
思考：二分思想下的查找算法的时间复杂度是固定不变的么？时间复杂度会受到算法实现细节的影响吗？

以比较为基础查找的时间下界

- 对于已排序的 n 个元素, 查找某元素 x 是否出现时, 是否存在以比较为基础的查找算法, 在**最坏**情况下该算法的计算时间比二分查找算法的计算时间更低?
 - 对于以比较为基础的查找算法的分析, 采取构建二元比较树的方式



〔问题描述〕 有 n 个如下关系的元素: $A(1) \leq A(2) \leq \dots \leq A(n)$, 查找某一给定元素 X 是否在 A 中出现。



线性查找

二分查找

以比较为基础查找的时间下界

定理4.2: 设 $A[1..n]$ 含有 $n(n \geq 1)$ 个不同的元素, 排序为 $A(1) \leq A(2) \leq \dots \leq A(n)$ 。又设以比较为基础去判断是否 $x \in A(1:n)$ 的任何算法在最坏情况下所需的最小比较次数是 $\text{FIND}(n)$, 那么 $\text{FIND}(n) \geq \lceil \log(n+1) \rceil$

- 证明: 通过考察模拟求解查找问题的各种可能算法的比较树可知, $\text{FIND}(n)$ 为树中由根到一个叶子的最长路径的距离。在这所有的树中都必定有 n 个内节点与 x 在 A 中可能的 n 种出现情况相对应。如果一棵二元树的所有内节点所在的级数小于或等于级数 k , 则最多有 $2^k - 1$ 个内节点。因此 $n \leq 2^k - 1$, 即 $\text{FIND}(n) = k \geq \lceil \log(n+1) \rceil$ 。证毕。

不存在其最坏情况下计算时间低于 $\Theta(\log n)$ 的算法。
二分查找是解决查找问题最坏情况的最优算法

思考题

- 证明BINSRCH1的最好、最坏和平均情况时间对于成功和不成功的查找都是 $\Theta(\log n)$ 。
- 三分（多分）查找会得到更优的结果吗？



目录

- 4.1 一般方法
- 4.2 二分查找
- 4.3 归并排序
- 4.4 快速排序
- 4.5 斯特拉森矩阵乘法
- 4.6 二维极大点问题
- 4.7 分治法的优化
- 4.8 递归式分析方法
- 4.9 小结



杠铃增重问题

- 每位参赛运动员向组委会提交**排好序**的三次试举重量

125	130	132
-----	-----	-----



123	127	129
-----	-----	-----



117	121	126
-----	-----	-----

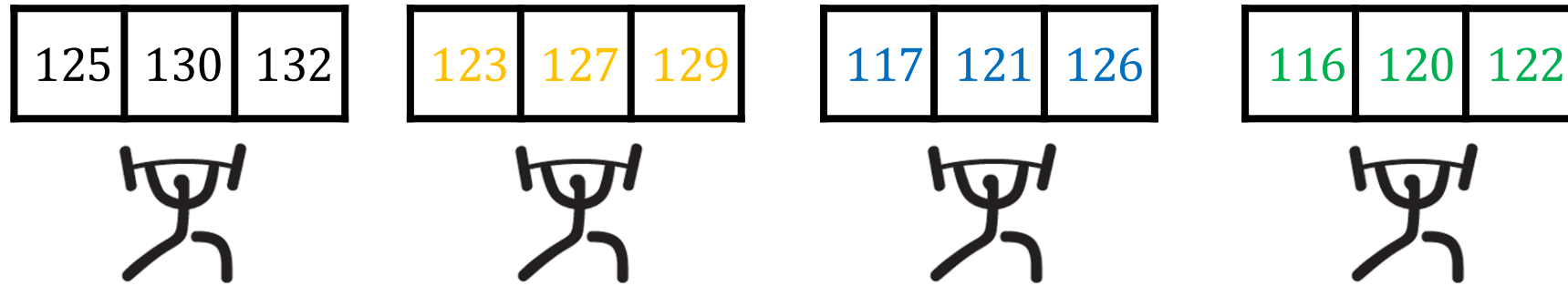


116	120	122
-----	-----	-----



杠铃增重问题

- 每位参赛运动员向组委会提交**排好序**的三次试举重量
- 为便于杠铃拆卸，组委会需对**所有**试举重量**递增排序**



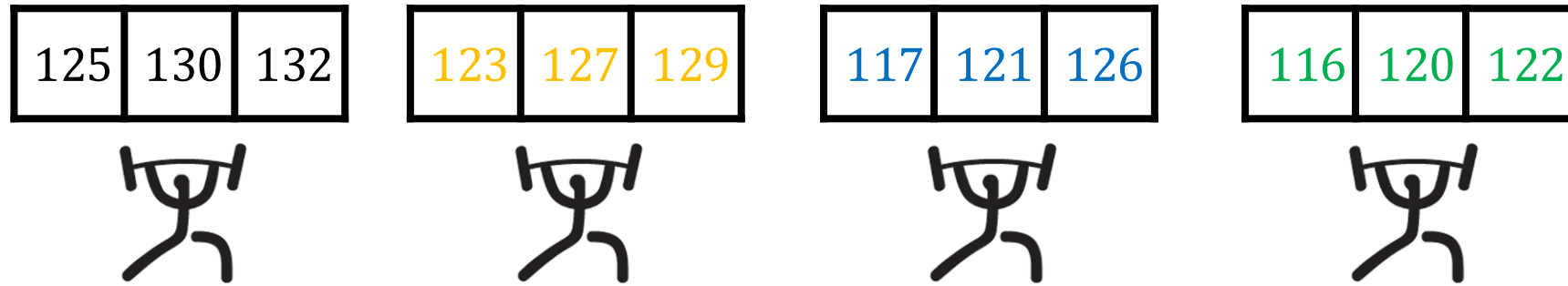
杠铃增重顺序：

116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----



杠铃增重问题

- 每位参赛运动员向组委会提交**排好序**的三次试举重量
- 为便于杠铃拆卸，组委会需对**所有**试举重量**递增排序**

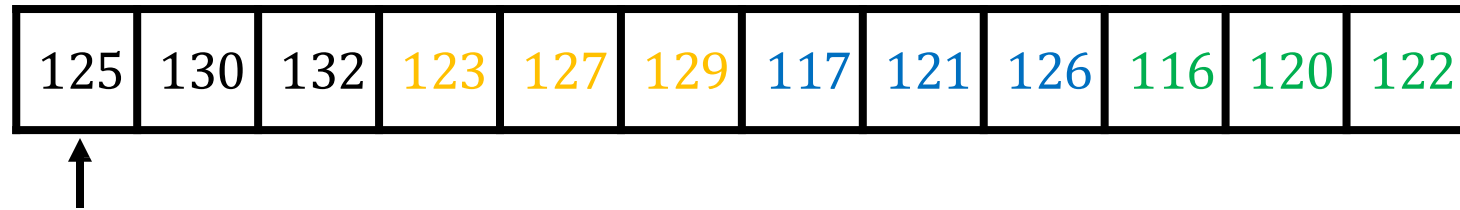


杠铃增重顺序：116 117 120 121 122 123 125 126 127 129 130 132

问题：组委会如何根据试举重量安排杠铃增重顺序？

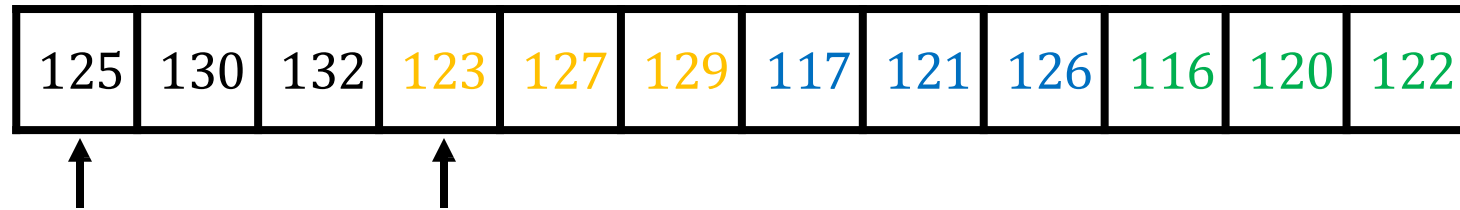
杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序



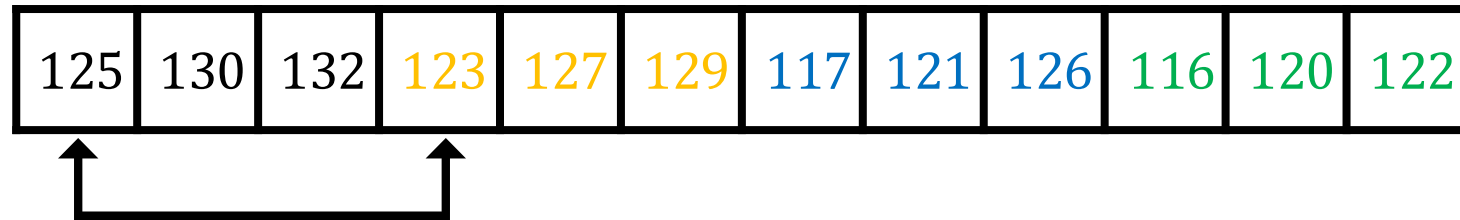
杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序



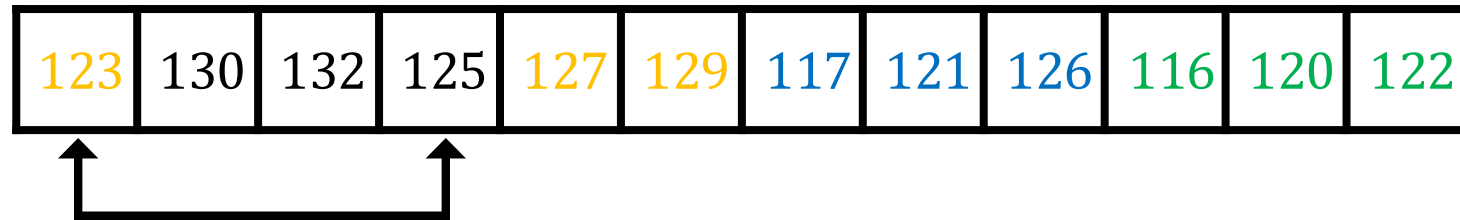
杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序



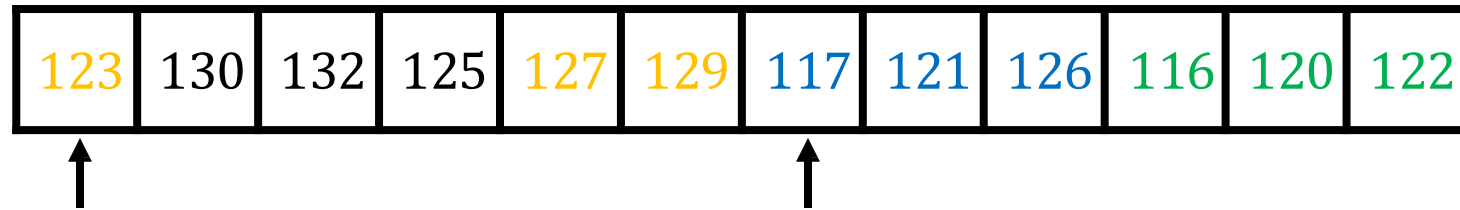
杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序



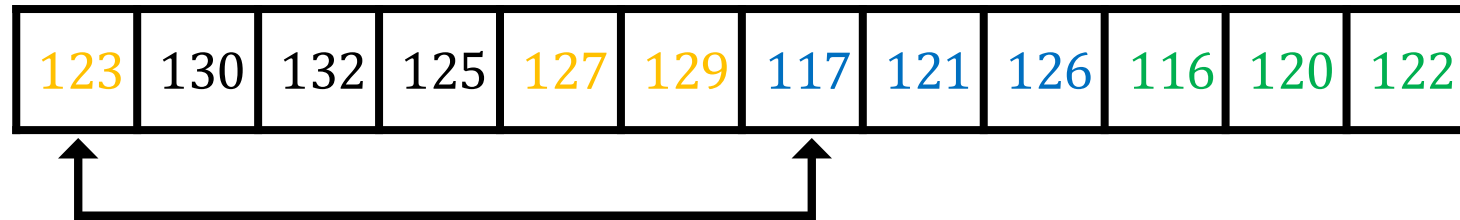
杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序



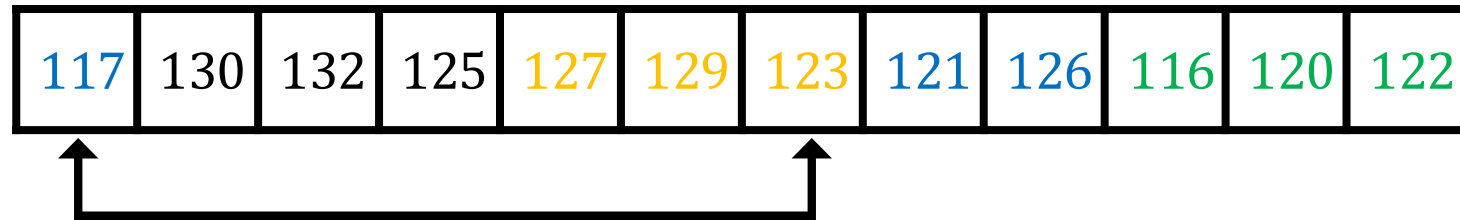
杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序



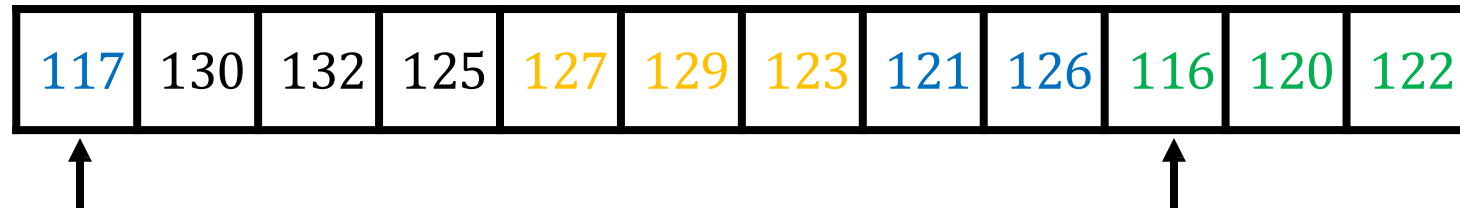
杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序



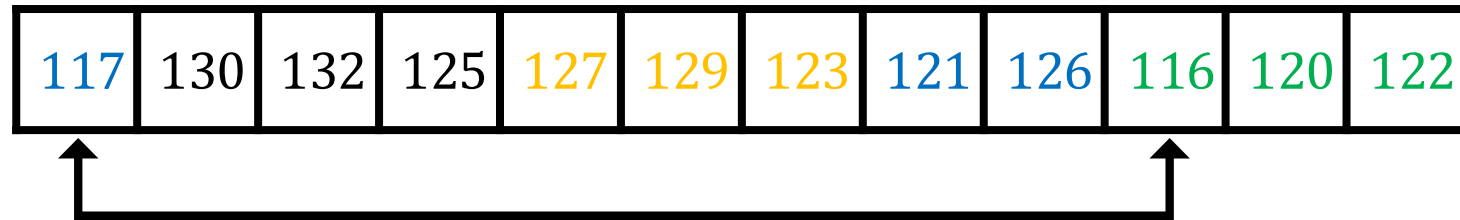
杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序



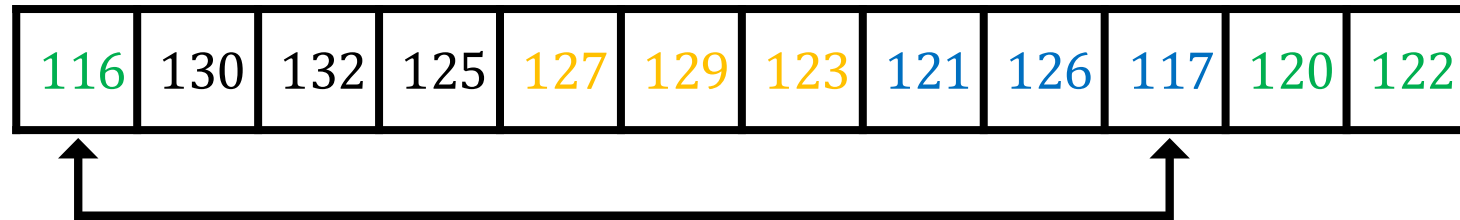
杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序



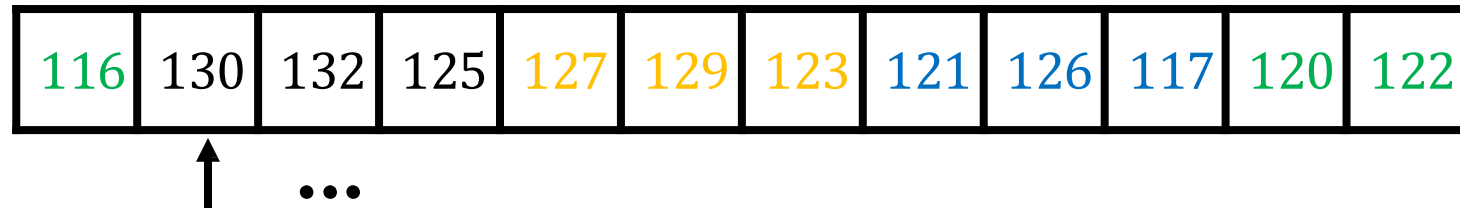
杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序



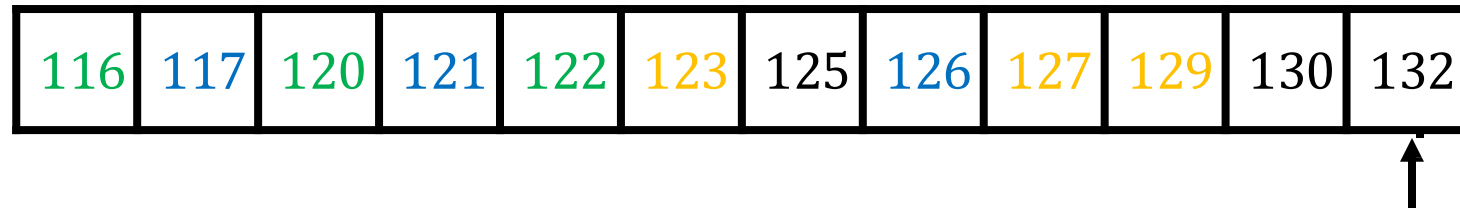
杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序



杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序



杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序
- 比较次数：66次

116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----



杠铃增重问题

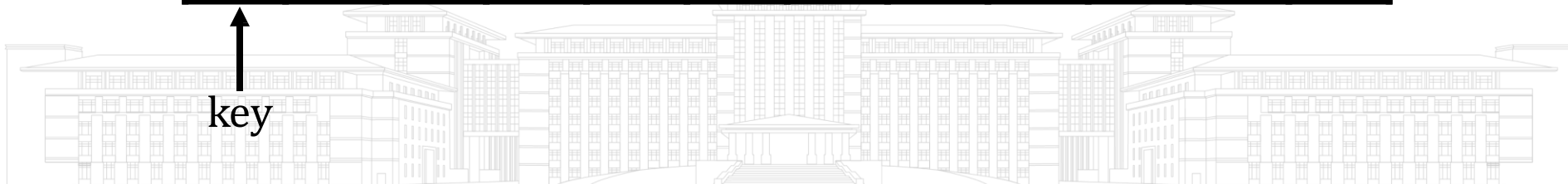
- 选择排序
- 从待排序元素中迭代选出最小值并排序
- 比较次数：66次

116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

- 插入排序
- 依次将每个元素插入到已排序序列之中

125	130	132	123	127	129	117	121	126	116	120	122
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

↑
key



杠铃增重问题

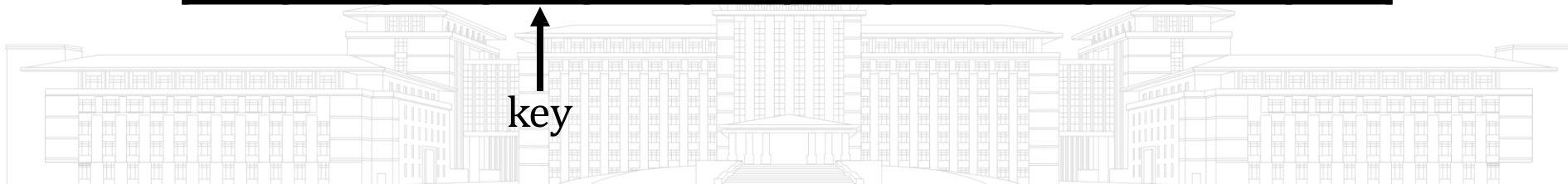
- 选择排序
- 从待排序元素中迭代选出最小值并排序
- 比较次数：66次

116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

- 插入排序
- 依次将每个元素插入到已排序序列之中

125	130	132	123	127	129	117	121	126	116	120	122
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

↑
key



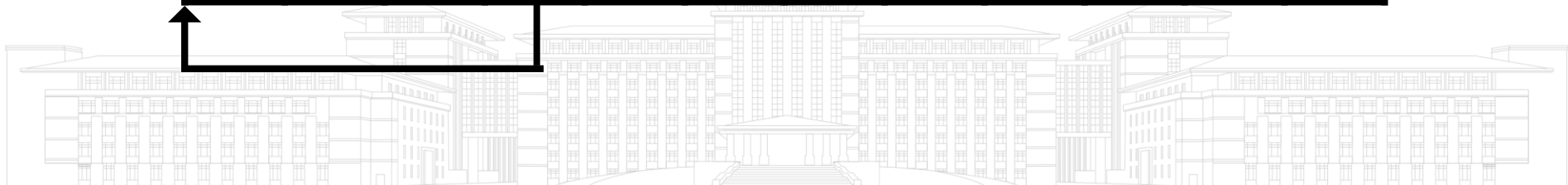
杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序
- 比较次数：66次

116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

- 插入排序
- 依次将每个元素插入到已排序序列之中

125	130	132	123	127	129	117	121	126	116	120	122
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----



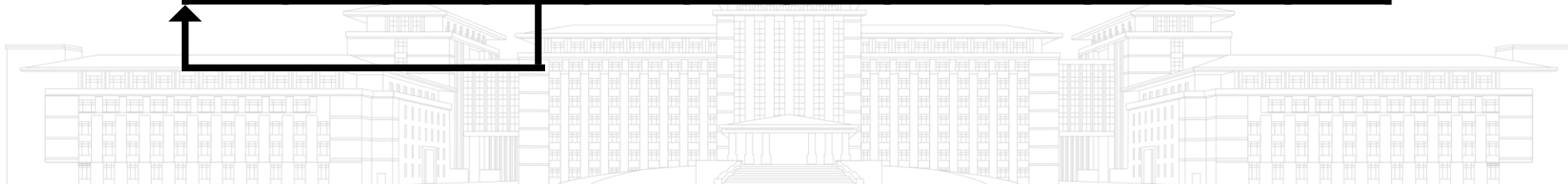
杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序
- 比较次数：66次

116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

- 插入排序
- 依次将每个元素插入到已排序序列之中

123	125	130	132	127	129	117	121	126	116	120	122
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----



杠铃增重问题

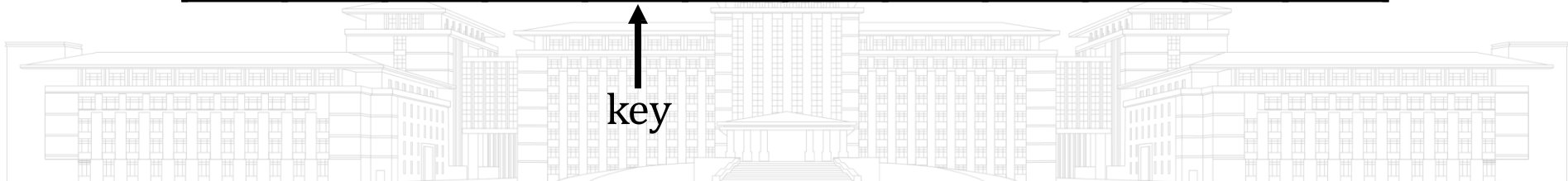
- 选择排序
- 从待排序元素中迭代选出最小值并排序
- 比较次数：66次

116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

- 插入排序
- 依次将每个元素插入到已排序序列之中

123	125	130	132	127	129	117	121	126	116	120	122
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

↑
key



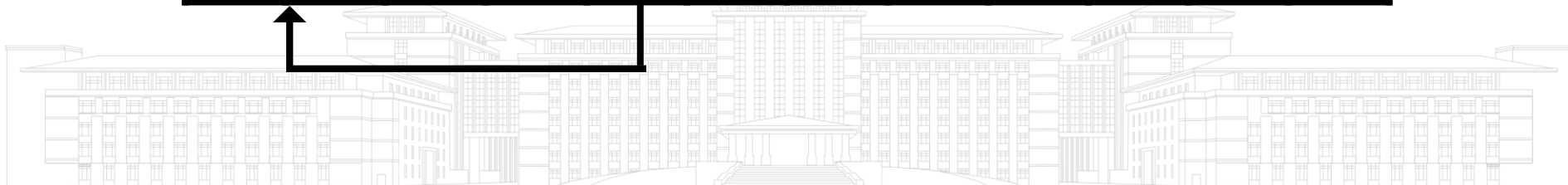
杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序
- 比较次数：66次

116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

- 插入排序
- 依次将每个元素插入到已排序序列之中

123	125	130	132	127	129	117	121	126	116	120	122
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----



杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序
- 比较次数：66次

116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

- 插入排序
- 依次将每个元素插入到已排序序列之中

123	127	125	130	132	129	117	121	126	116	120	122
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----



杠铃增重问题

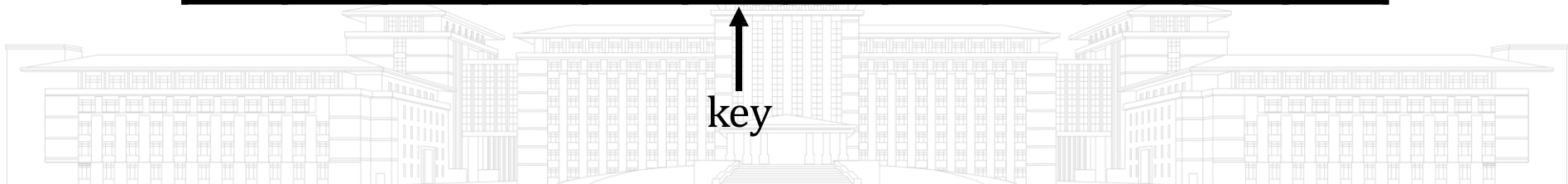
- 选择排序
- 从待排序元素中迭代选出最小值并排序
- 比较次数：66次

116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

- 插入排序
- 依次将每个元素插入到已排序序列之中

123	127	125	130	132	129	117	121	126	116	120	122
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

↑
key



杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序
- 比较次数：66次

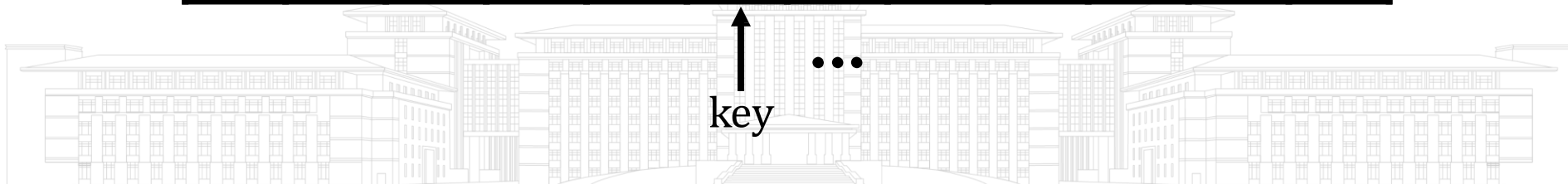
116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

- 插入排序
- 依次将每个元素插入到已排序序列之中

123	127	125	130	132	129	117	121	126	116	120	122
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

↑
key

...



杠铃增重问题

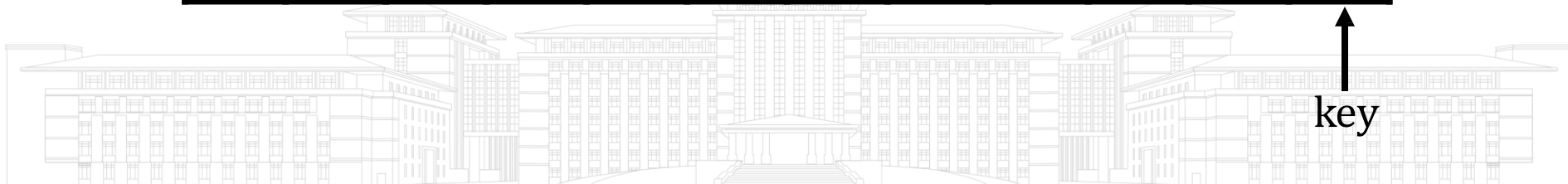
- 选择排序
- 从待排序元素中迭代选出最小值并排序
- 比较次数：66次

116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

- 插入排序
- 依次将每个元素插入到已排序序列之中

116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

↑
key



杠铃增重问题

- 选择排序
- 从待排序元素中迭代选出最小值并排序
- 比较次数：66次

116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

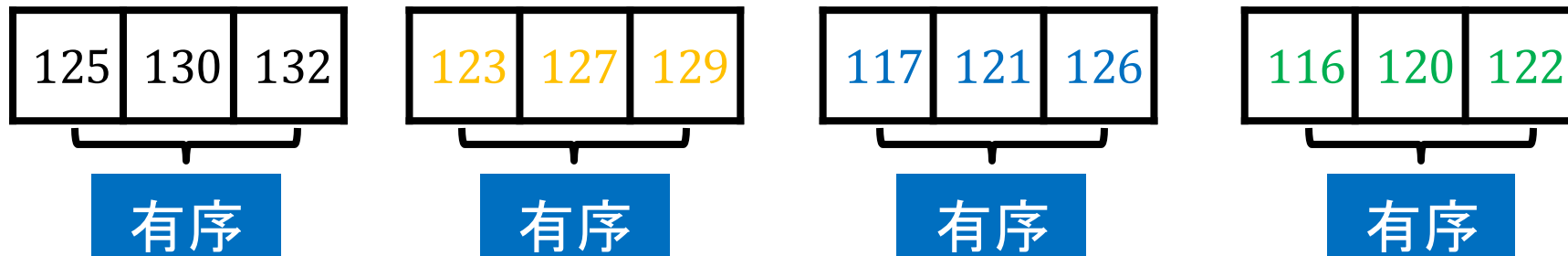
- 插入排序
- 依次将每个元素插入到已排序序列之中
- 比较次数：55次

116	117	120	121	122	123	125	126	127	129	130	132
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

思考：是否还有更高效的算法？

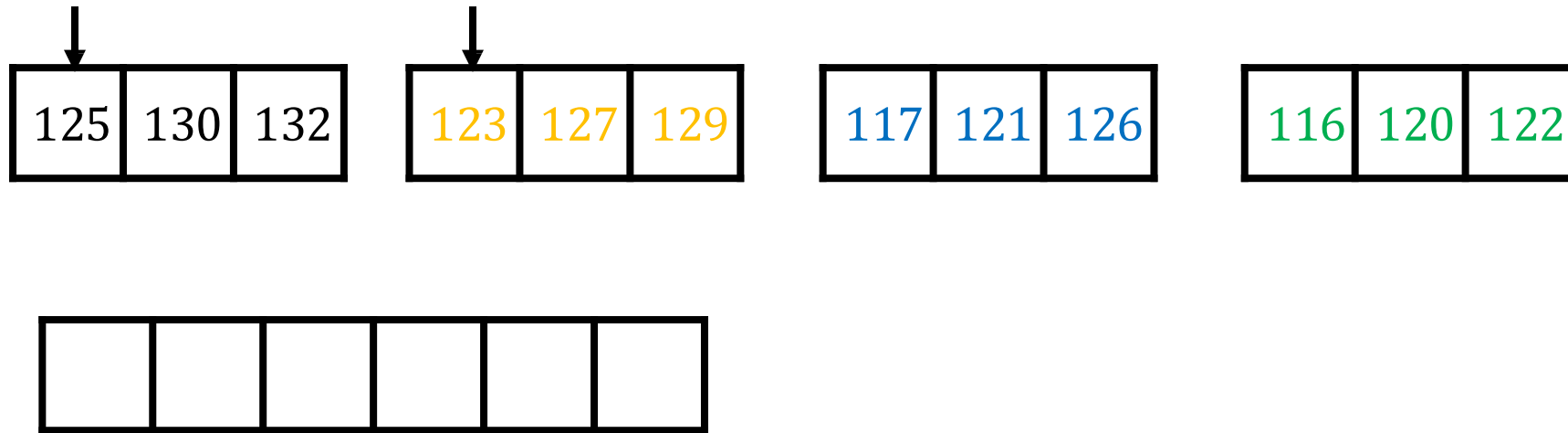
杠铃增重问题

- 问题特点：局部有序



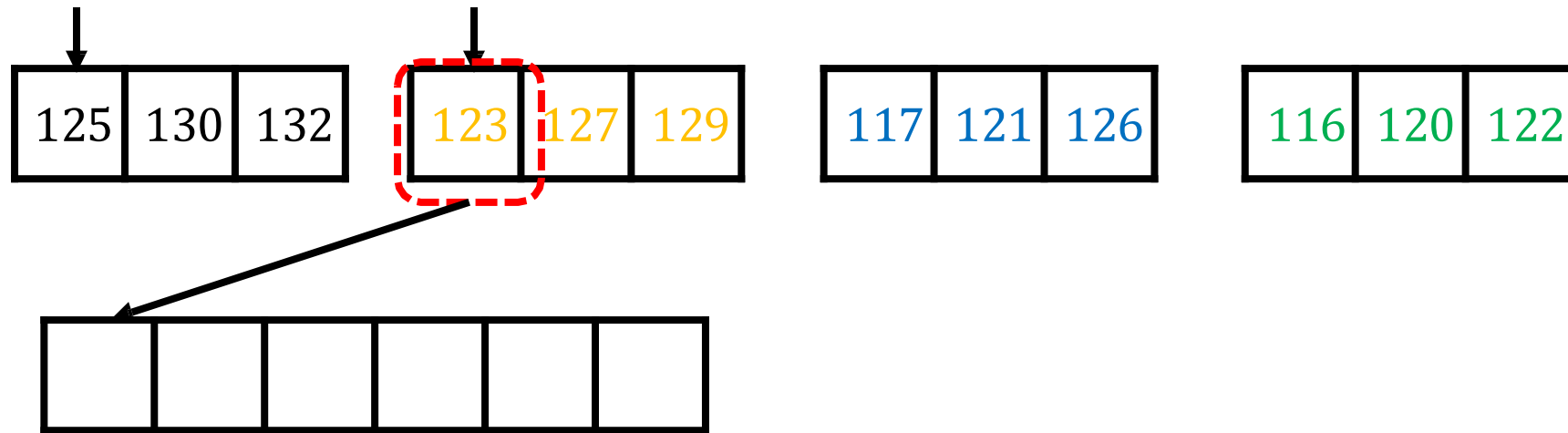
杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组



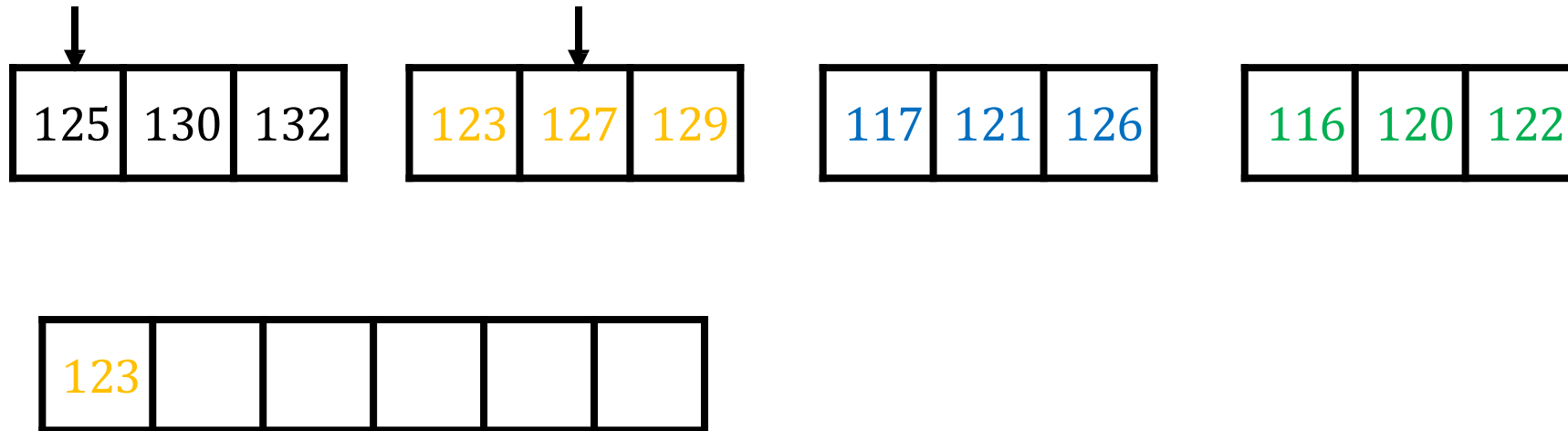
杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组



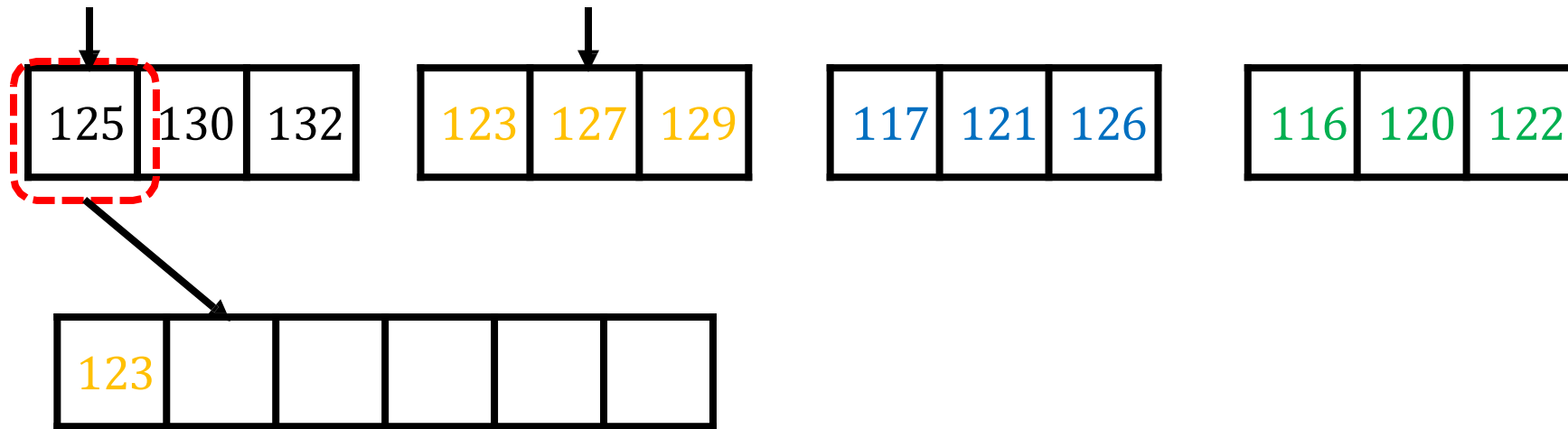
杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组



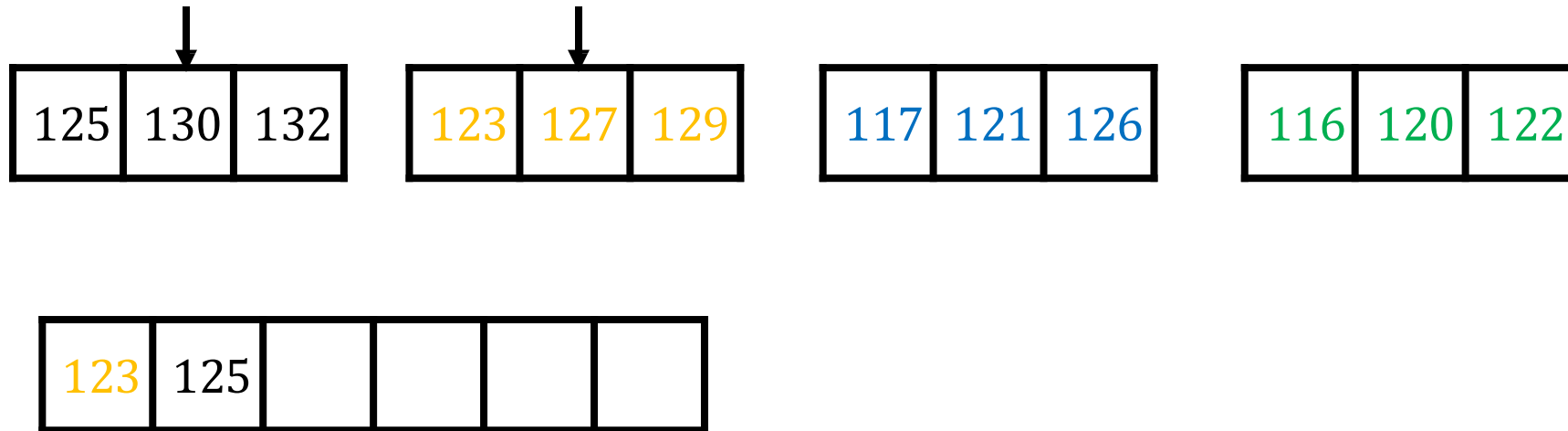
杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组



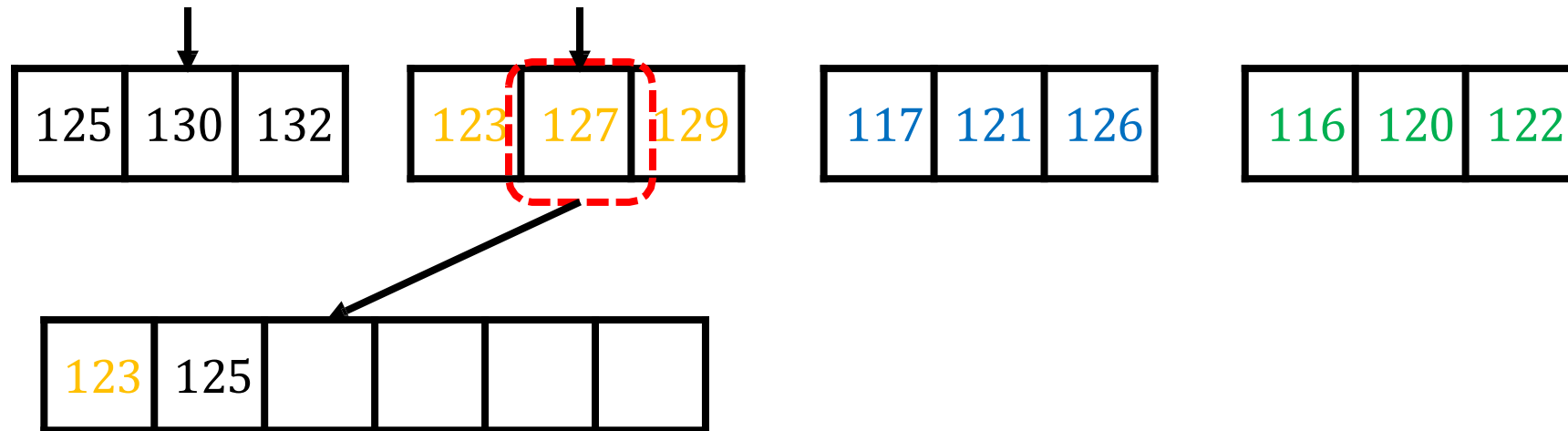
杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组



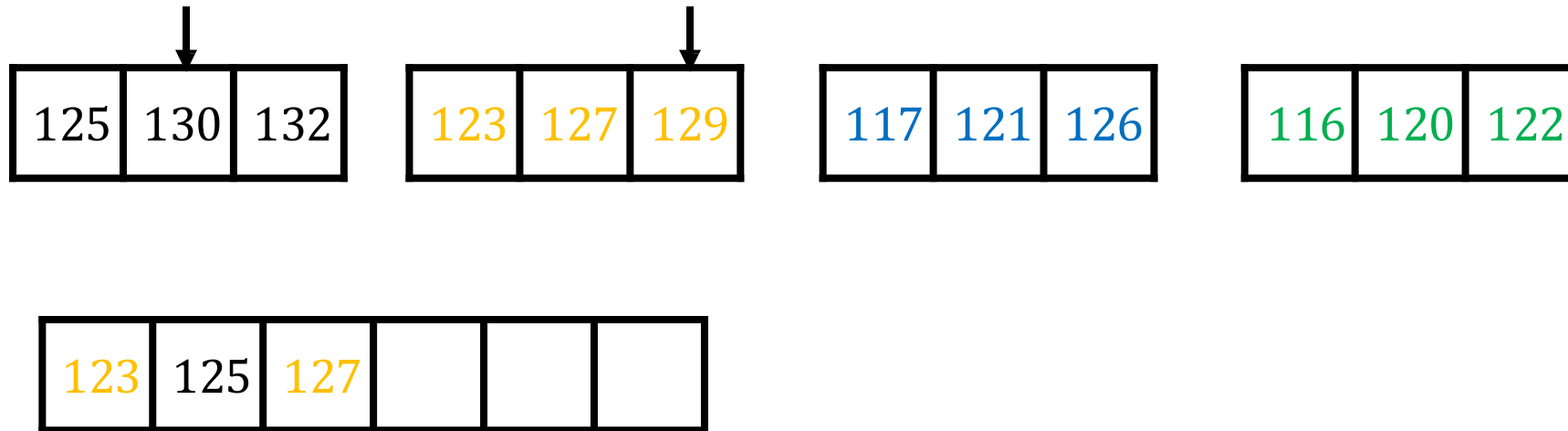
杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组



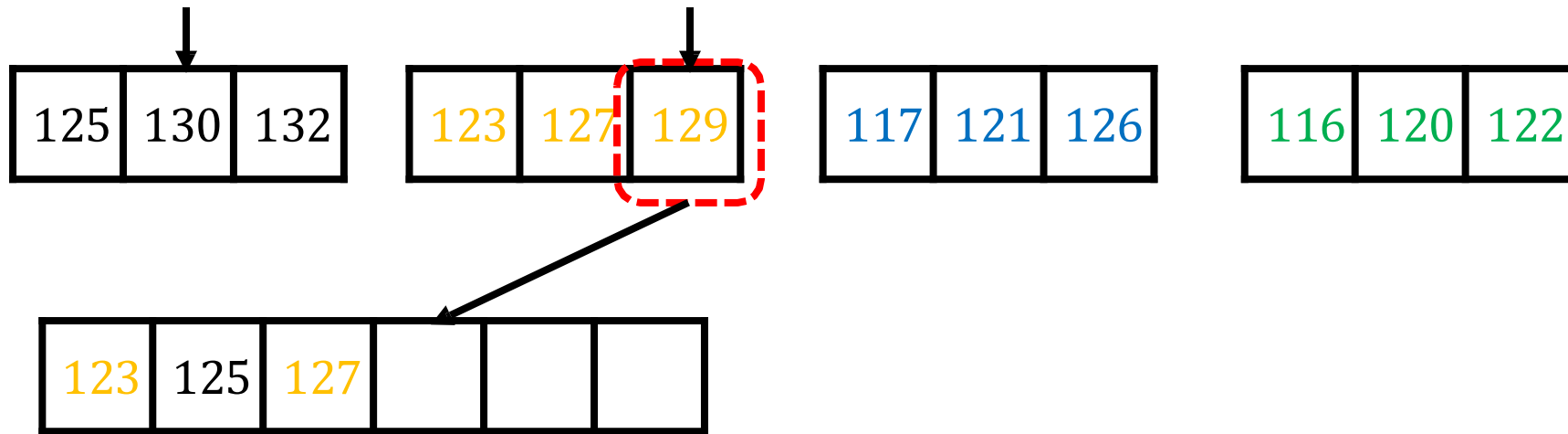
杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组



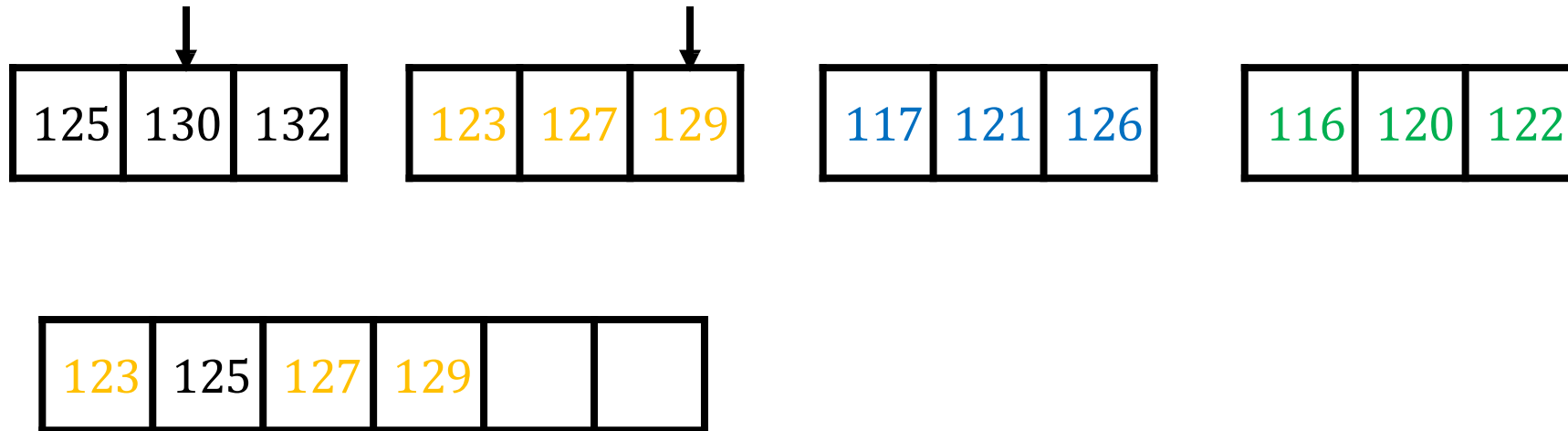
杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组



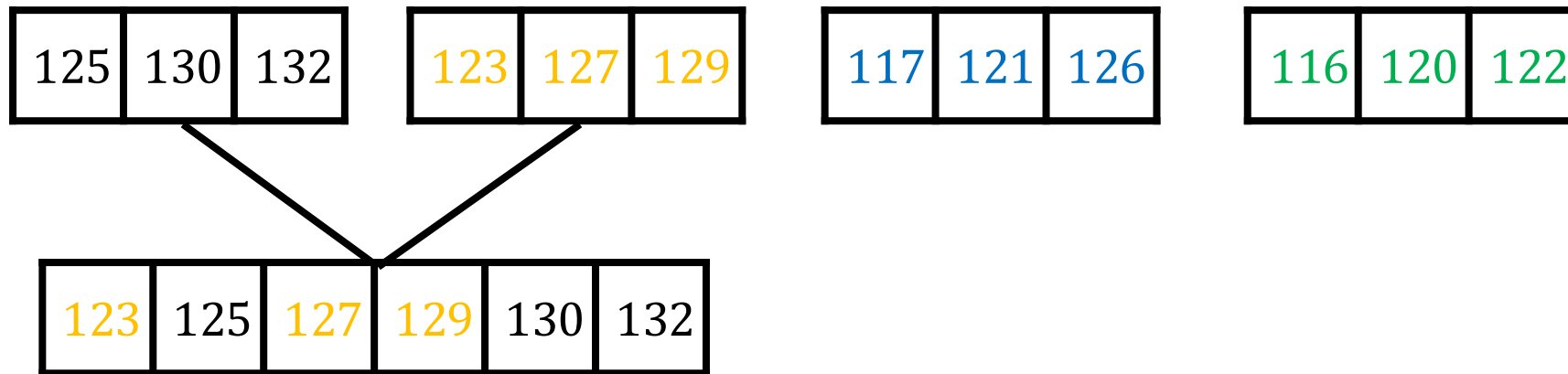
杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组



杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组





杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组
- 后续策略：逐一合并



杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组
- 后续策略：逐一合并

125	130	132
-----	-----	-----

123	127	129
-----	-----	-----

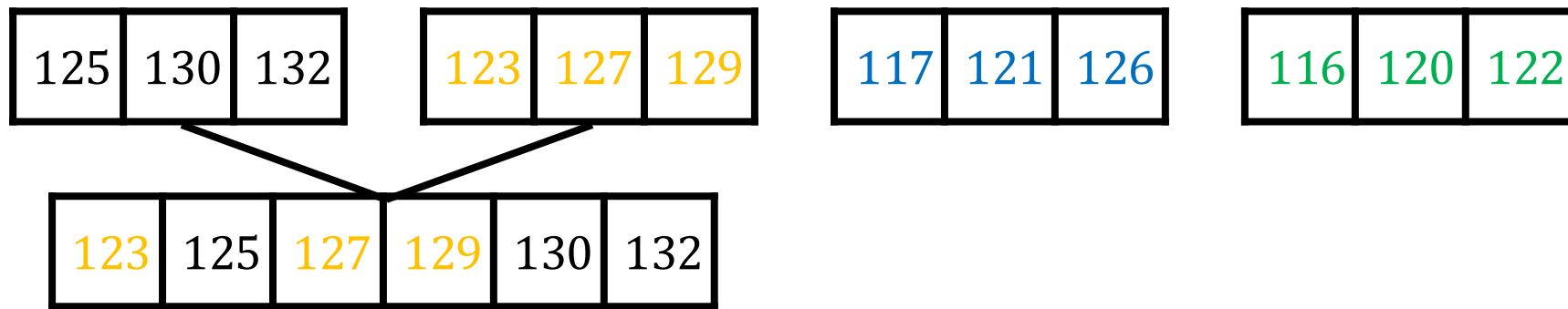
117	121	126
-----	-----	-----

116	120	122
-----	-----	-----



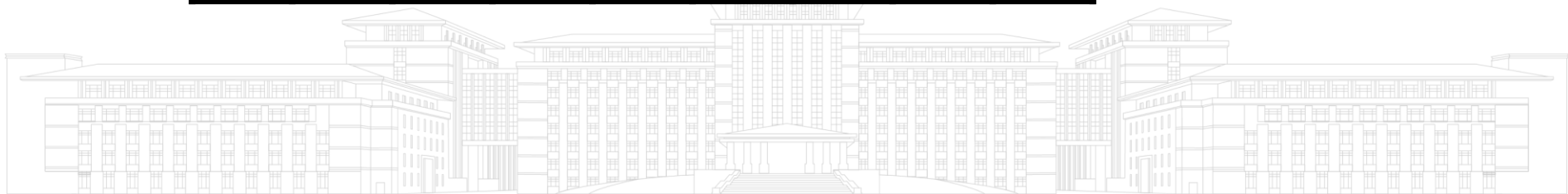
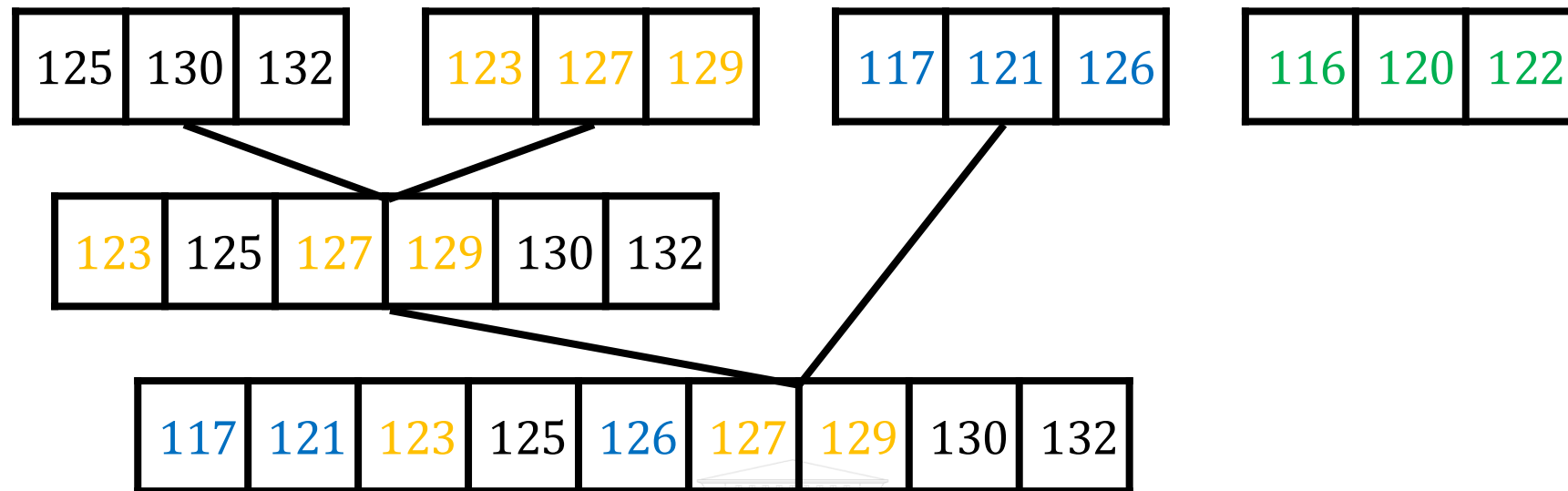
杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组
- 后续策略：逐一合并



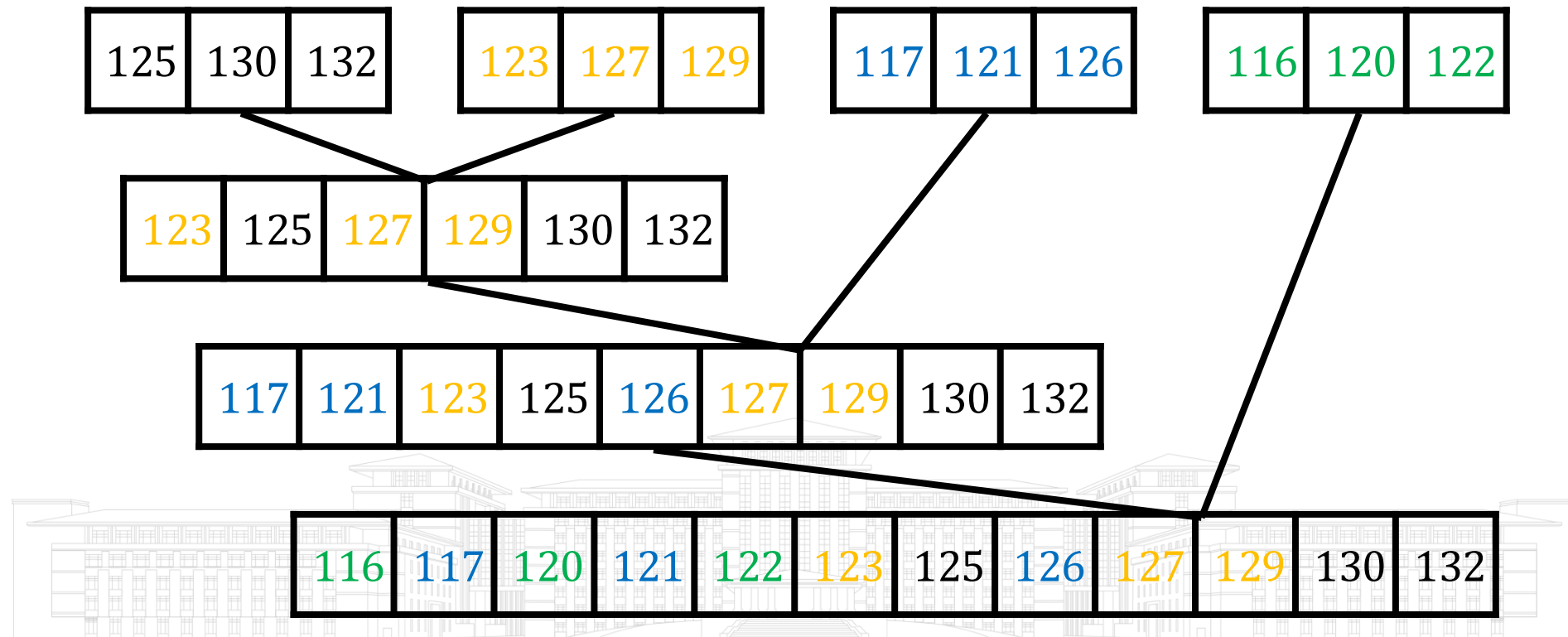
杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组
- 后续策略：逐一合并



杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组
- 后续策略：逐一合并



杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组
- 后续策略：
 - 逐一合并，比较27次

125	130	132
-----	-----	-----

123	127	129
-----	-----	-----

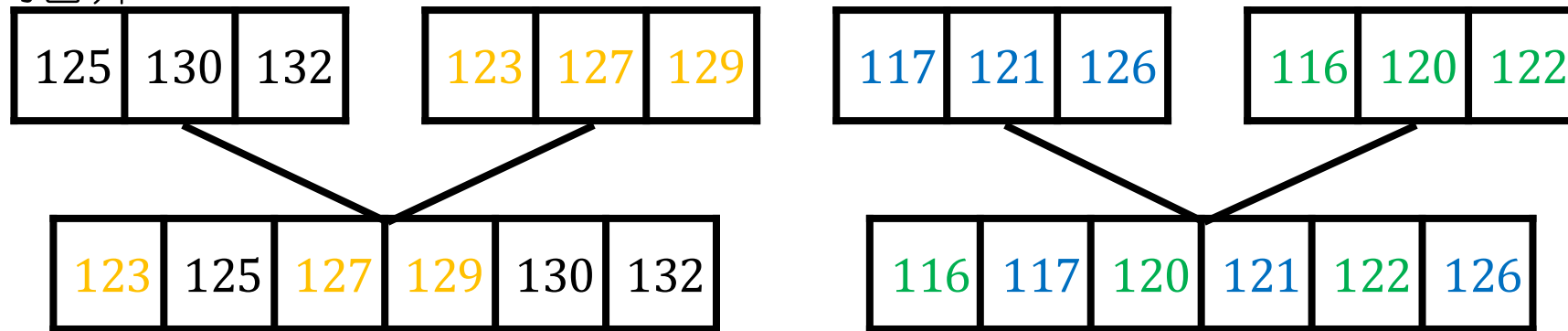
117	121	126
-----	-----	-----

116	120	122
-----	-----	-----



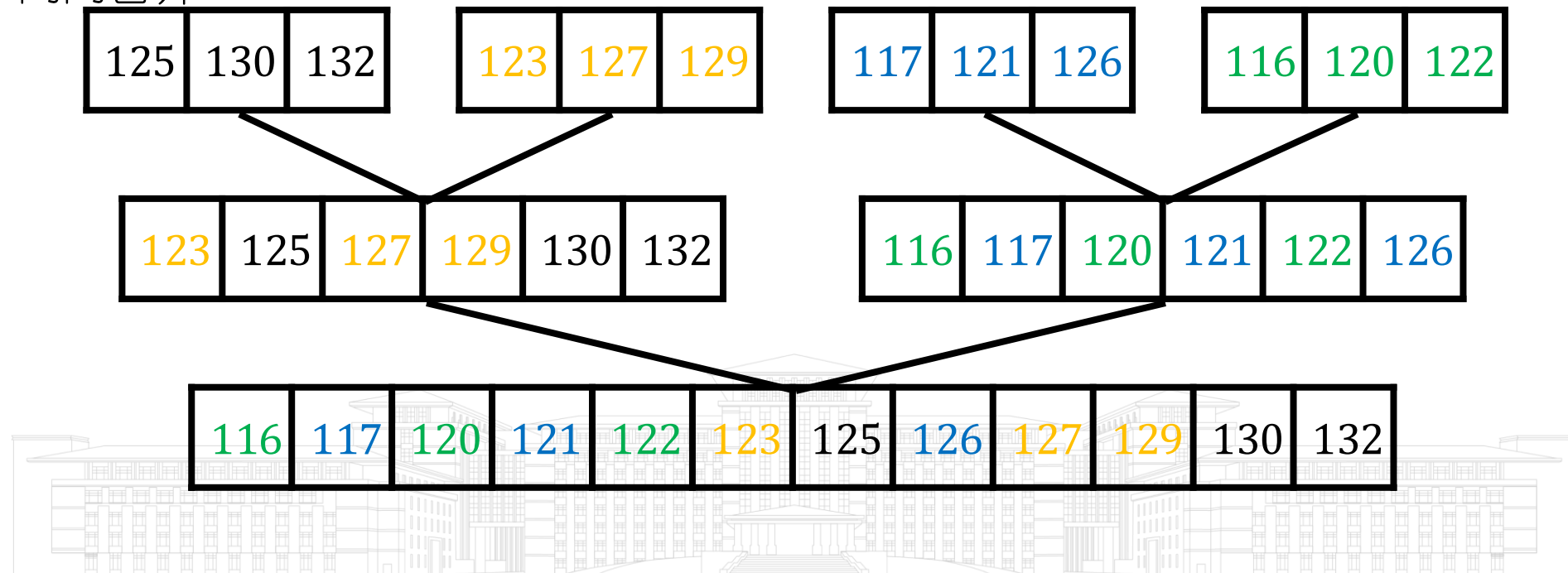
杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组
- 后续策略：
 - 逐一合并，比较27次
 - 两两合并



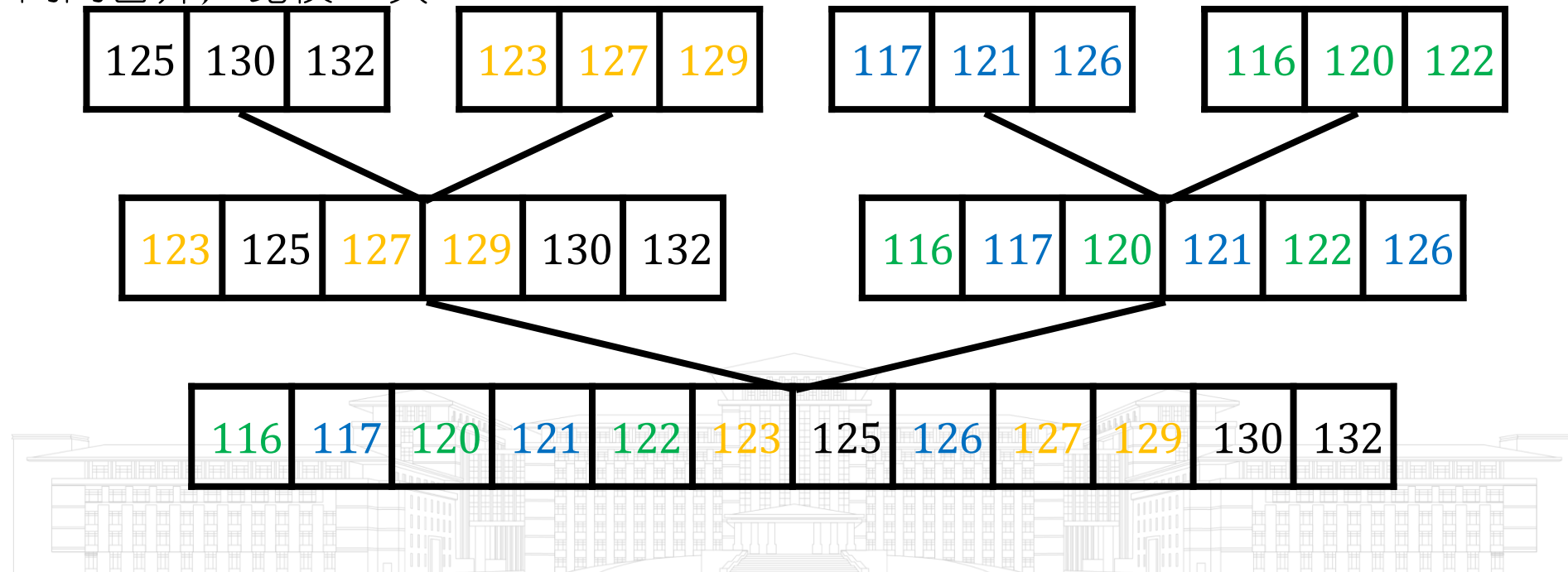
杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组
- 后续策略：
 - 逐一合并，比较27次
 - 两两合并



杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组
- 后续策略：
 - 逐一合并，比较27次
 - 两两合并，比较24次



杠铃增重问题

- 问题特点：局部有序
- 快速合并：比较两有序数组当前最小元素，将较小者逐一合入新数组
- 后续策略：
 - 逐一合并，比较27次
 - 两两合并，比较24次

策略名称	4位选手	8位选手	16位选手
逐一合并	27次	105次	405次
两两合并	24次	72次	192次

求解杠铃增重问题的两两合并策略对排序问题有何启发？

从杠铃增重问题到排序问题

- 输入：包含 n 个数字的数组 $[a_1, a_2, \dots, a_n]$
- 输出： $[a_1', a_2', \dots, a_n']$ ，满足 $a_1' \leq a_2' \leq \dots \leq a_n'$

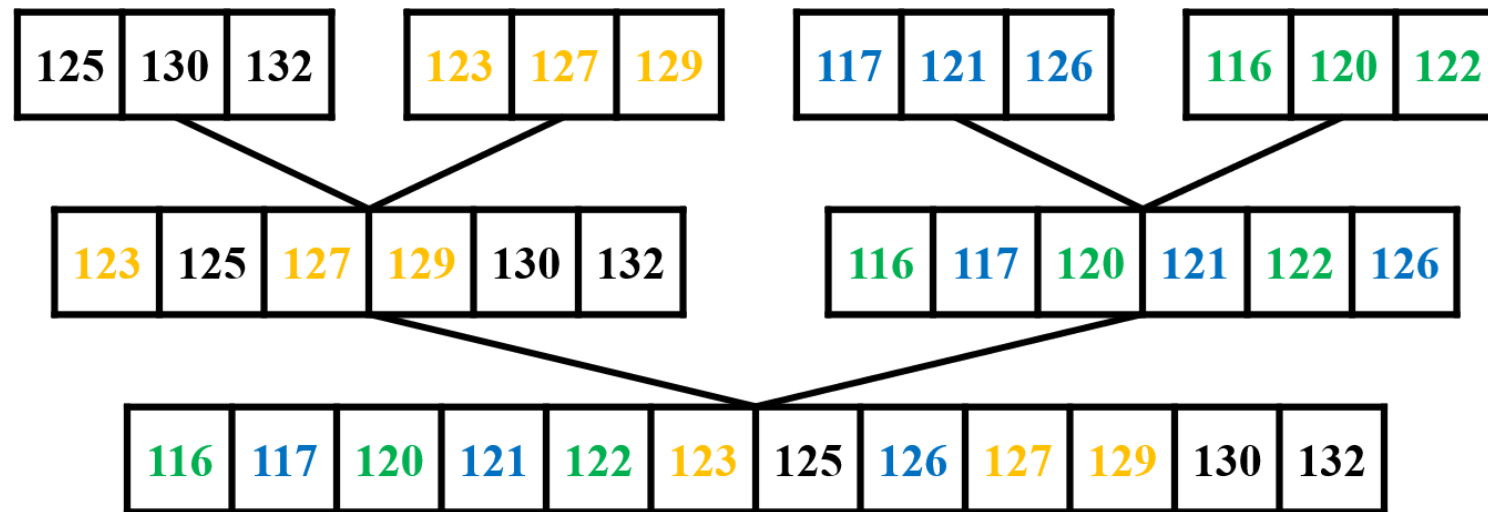
示例

24	17	40	28	13	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

从杠铃增重问题到排序问题

- 问题输入变化

杠铃增重问题



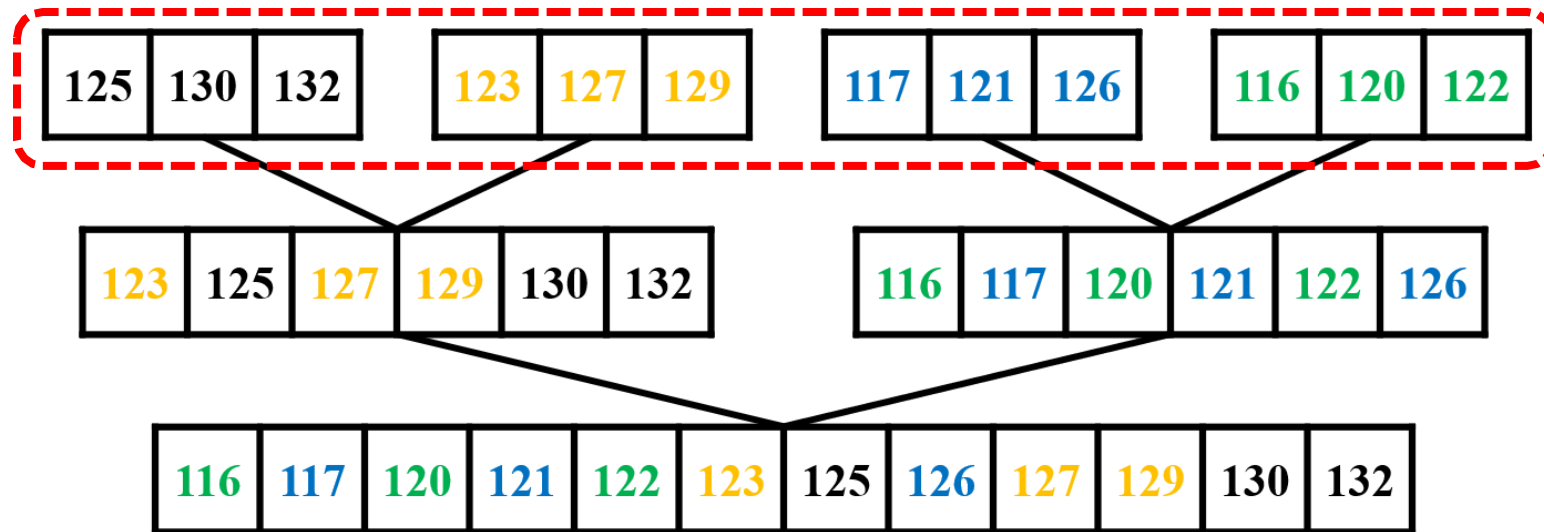
排序问题

24	17	40	28	13	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

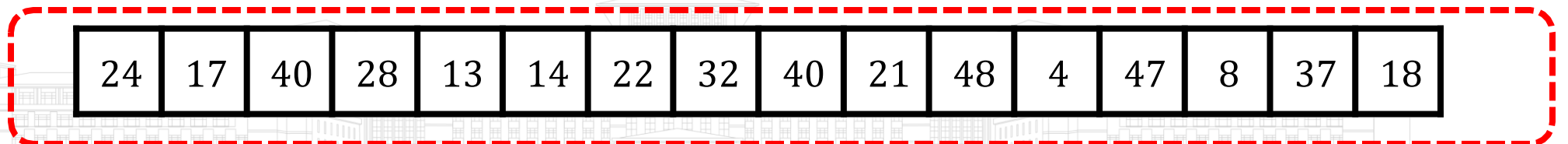
从杠铃增重问题到排序问题

- 问题输入变化
 - 完整数组输入

杠铃增重问题



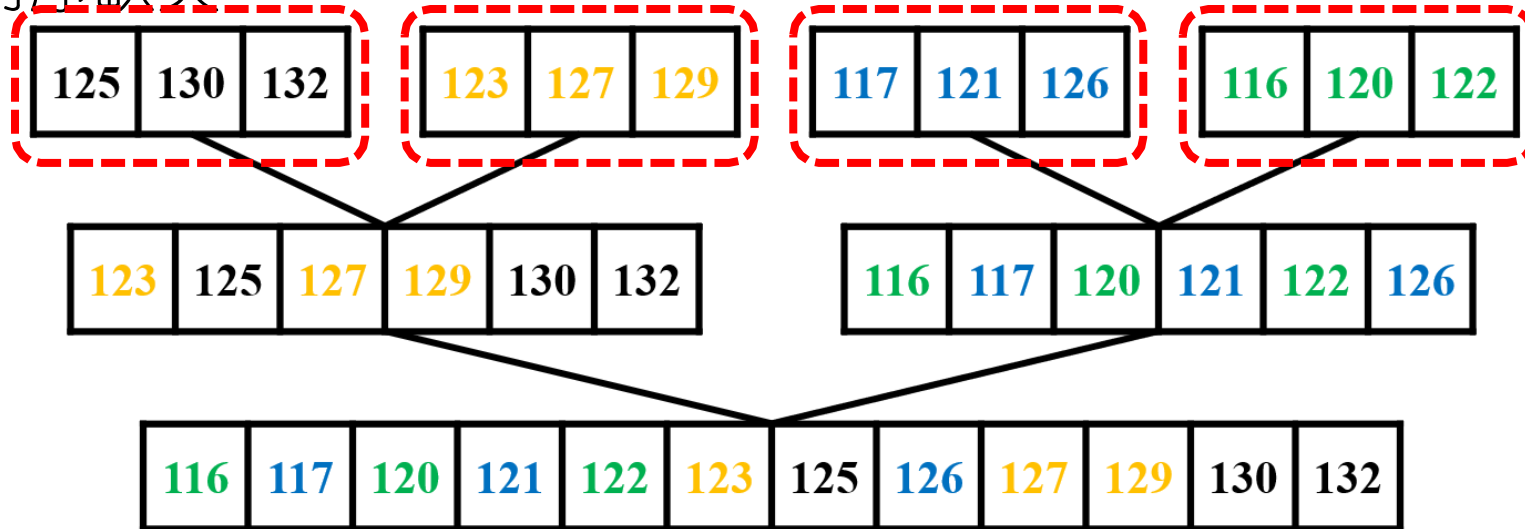
排序问题



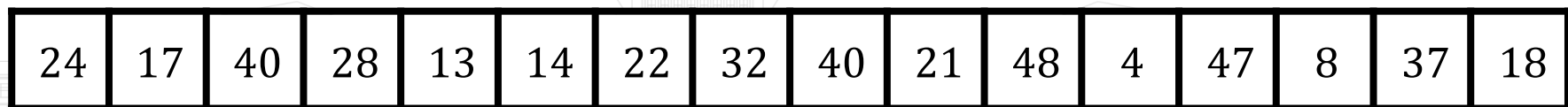
从杠铃增重问题到排序问题

- 问题输入变化
 - 完整数组输入
 - 局部有序缺失

杠铃增重问题



排序问题



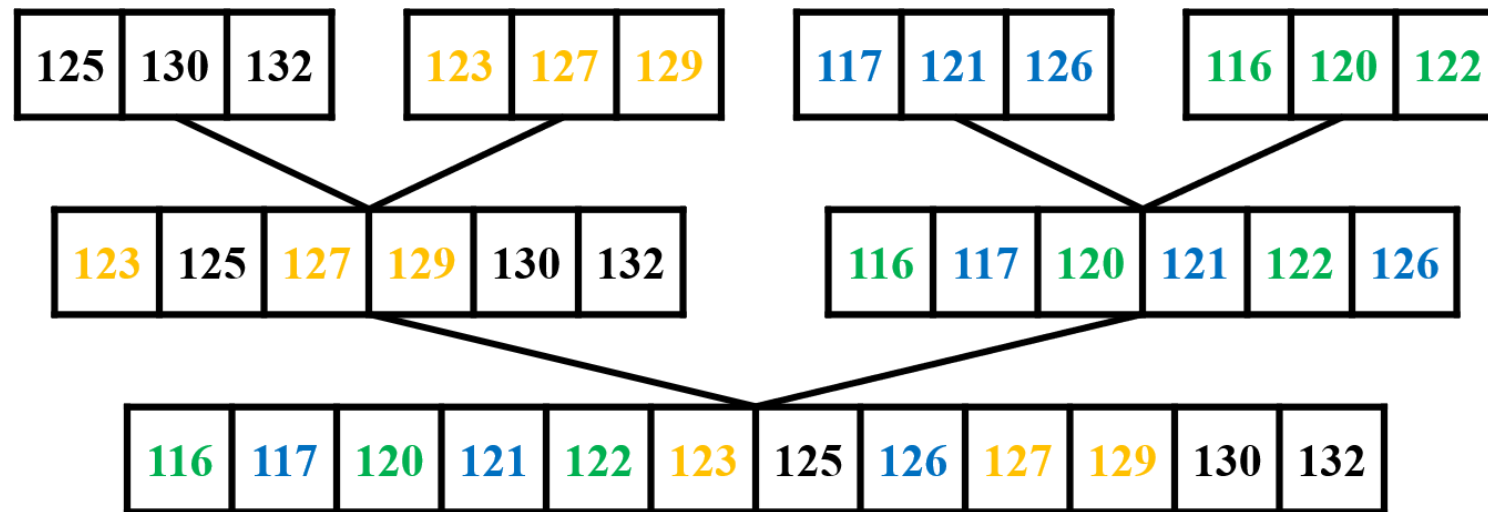
从杠铃增重问题到排序问题

- 问题输入变化

- 完整数组输入
- 局部有序缺失

两两合并策略如何适应问题输入的变化？

杠铃增重问题



排序问题

24	17	40	28	13	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

分而治之

- 思想：将整个问题分成若干个小问题后分而治之。
 - 分(Divide)：子问题与原问题具有相同的特征，但规模更小。
 - 治(Conquer)：反复使用分治策略，直到可以直接求解子问题为止。
 - 合(Combine)：将子问题的解组合成原问题的解。



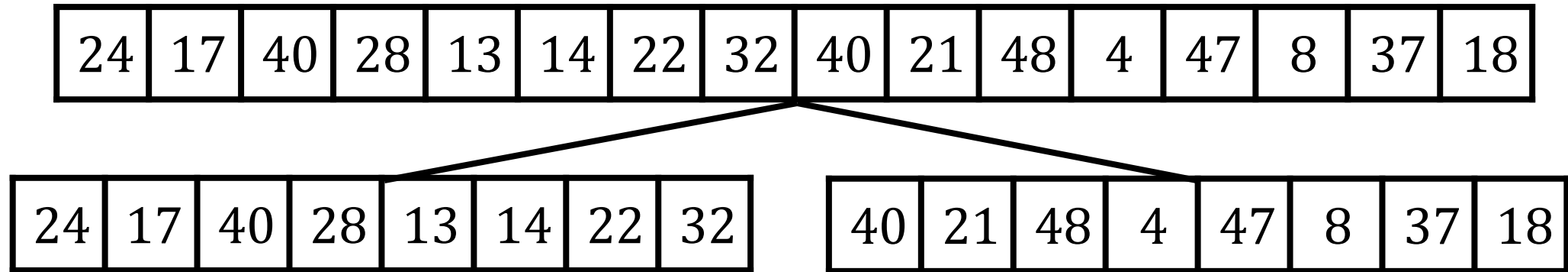
分解子问题



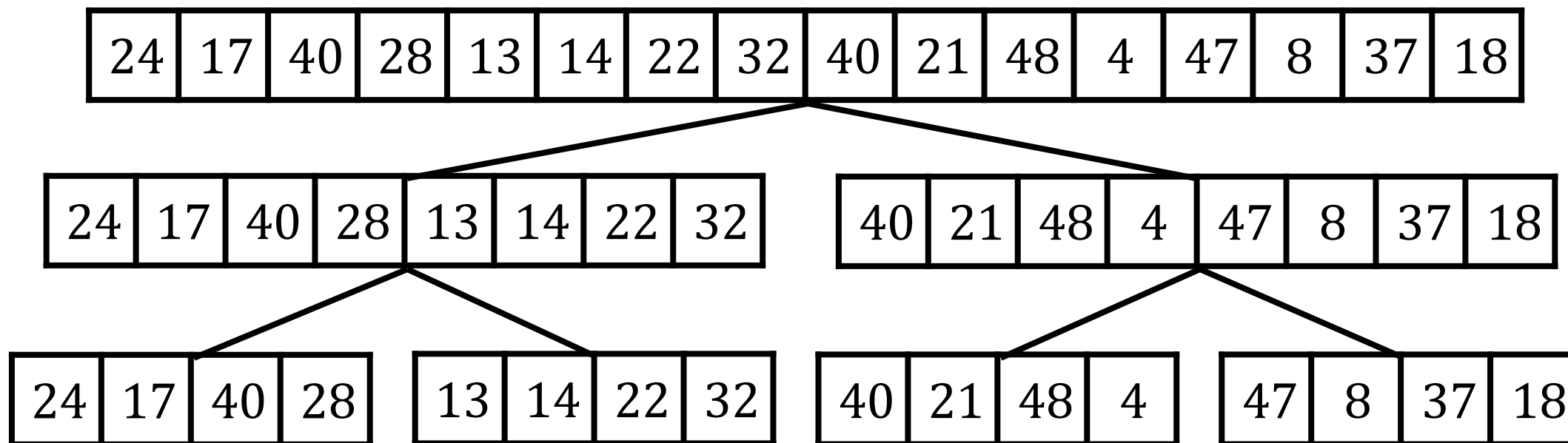
24	17	40	28	13	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----



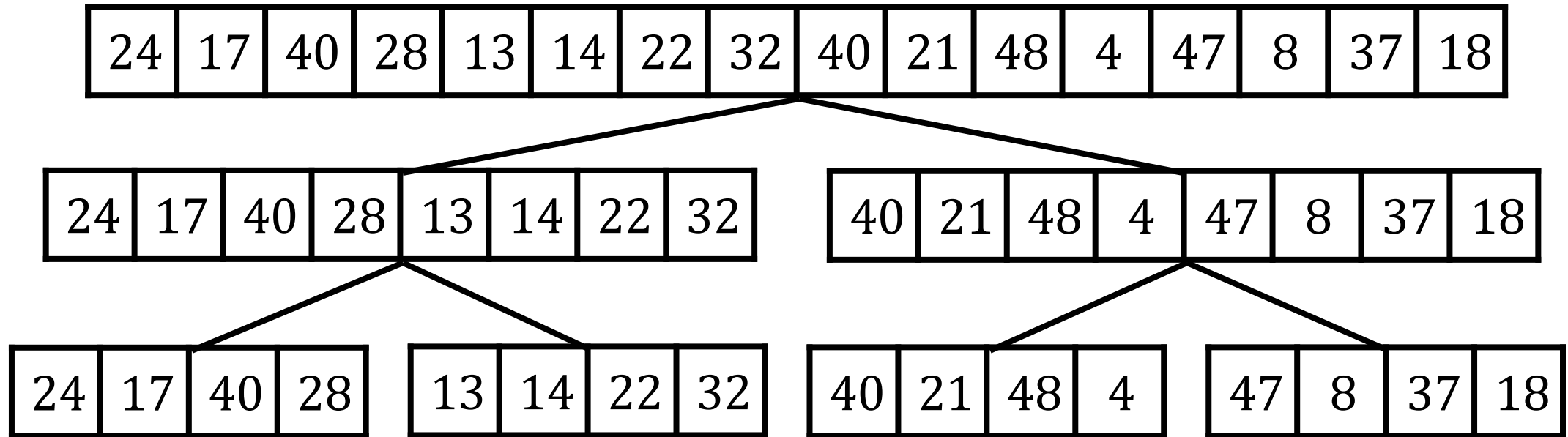
分解子问题



分解子问题



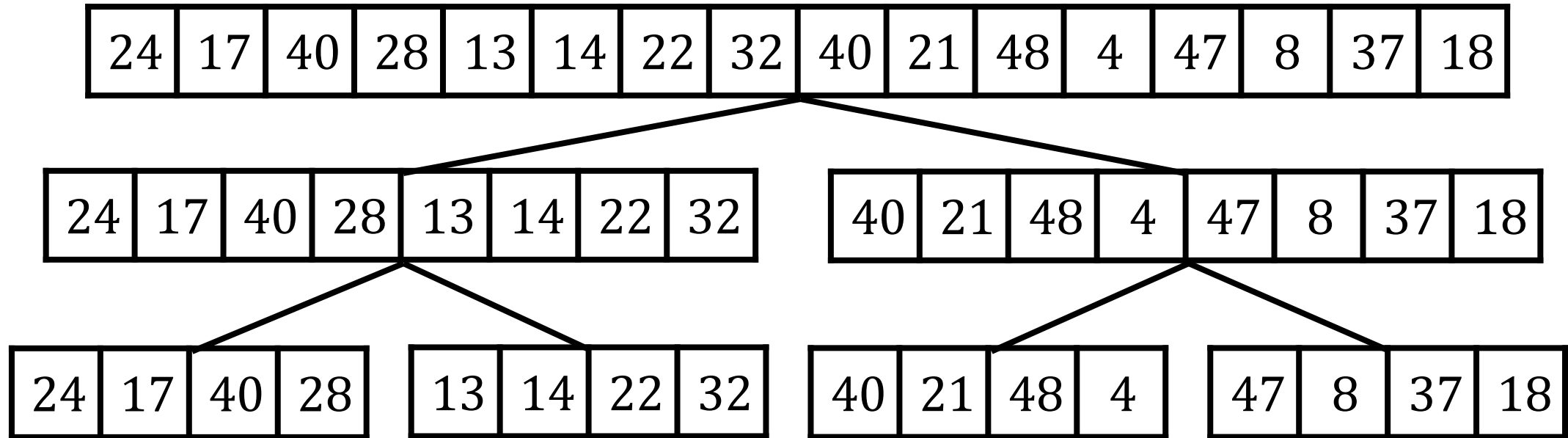
分解子问题



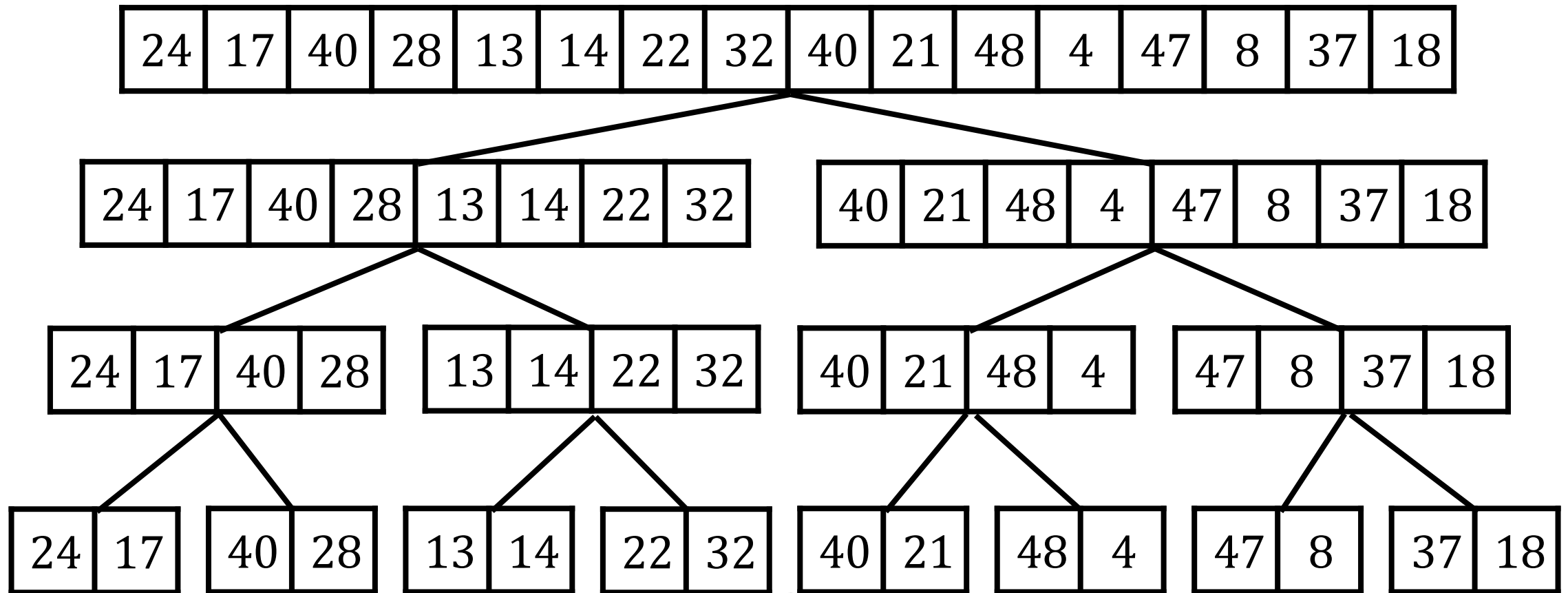
通过分解处理完整数组输入的变化



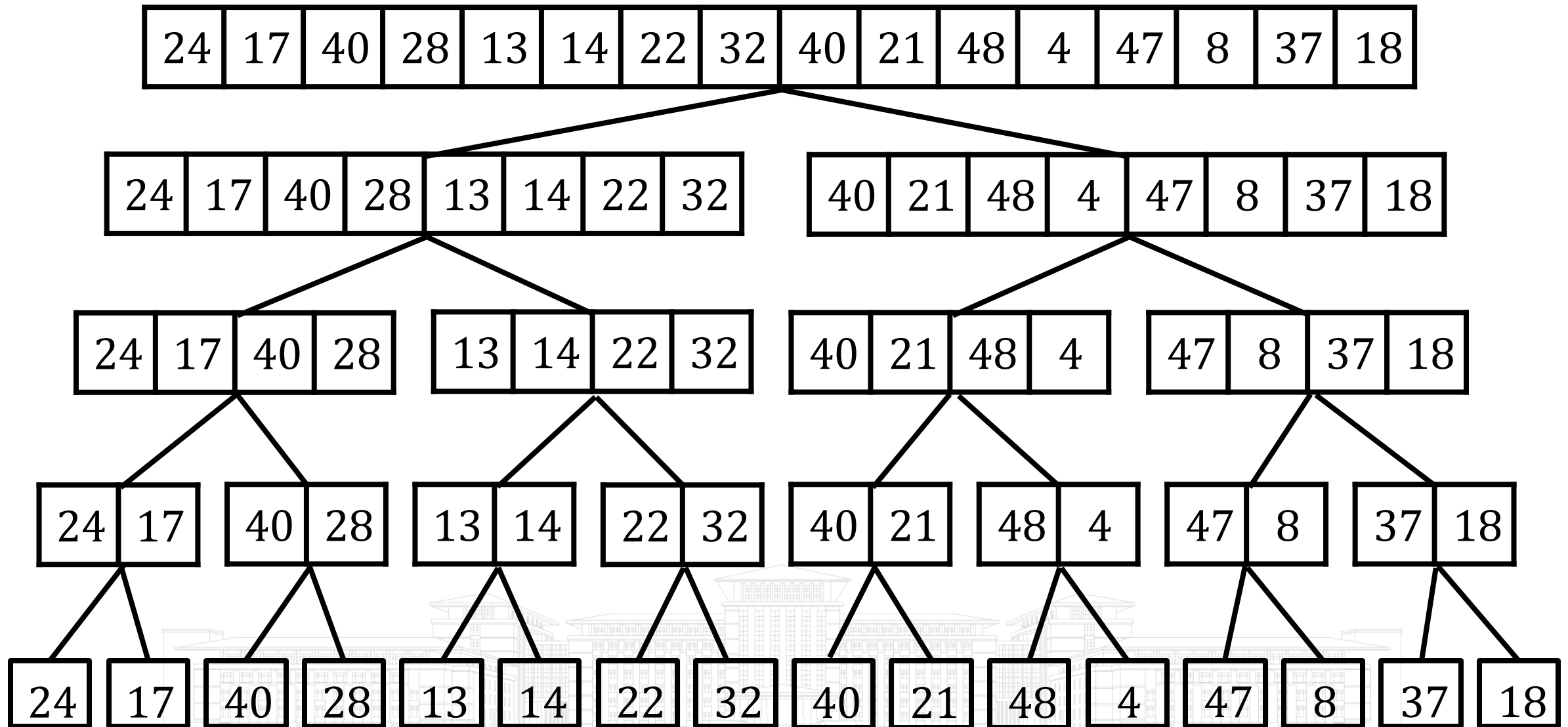
分解子问题



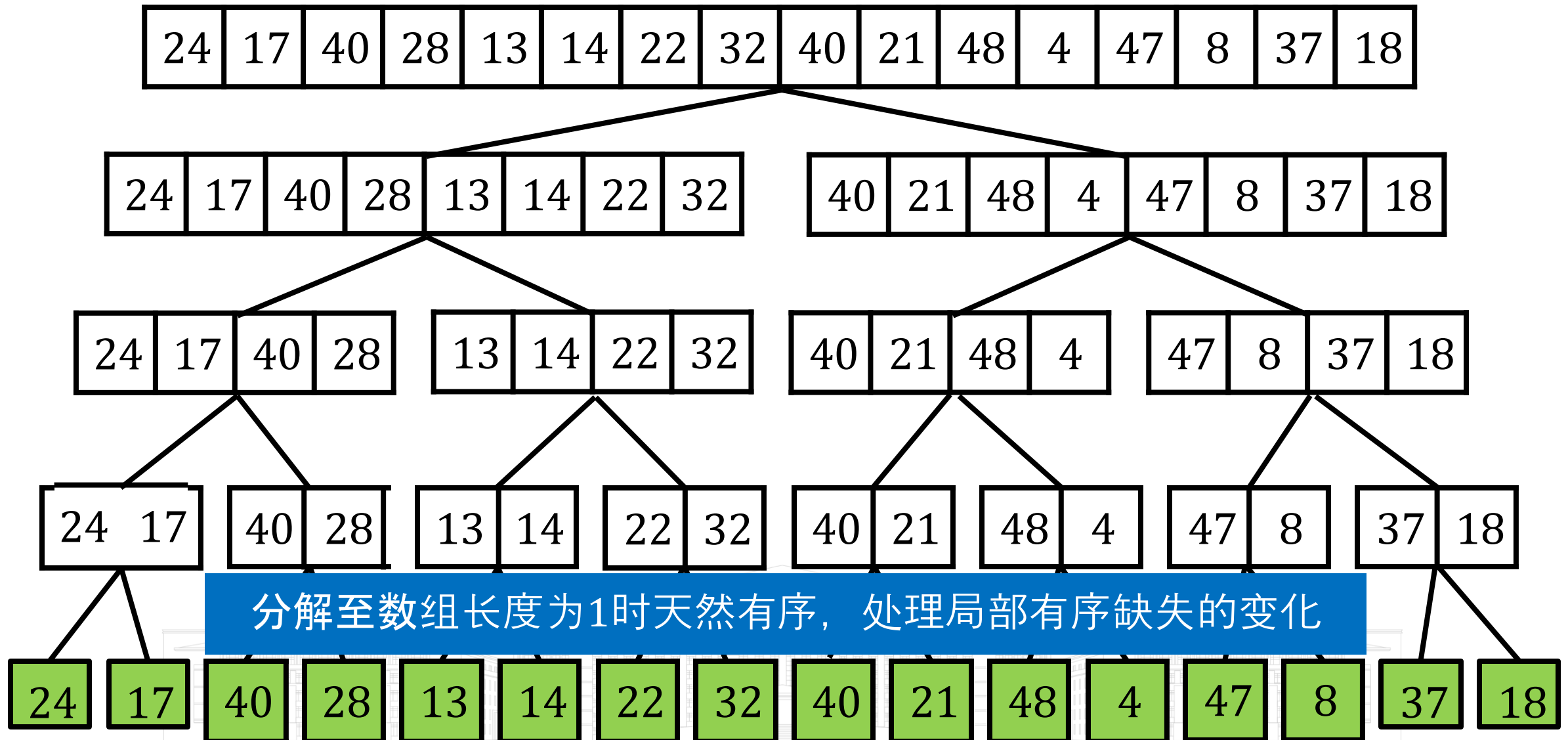
分解子问题



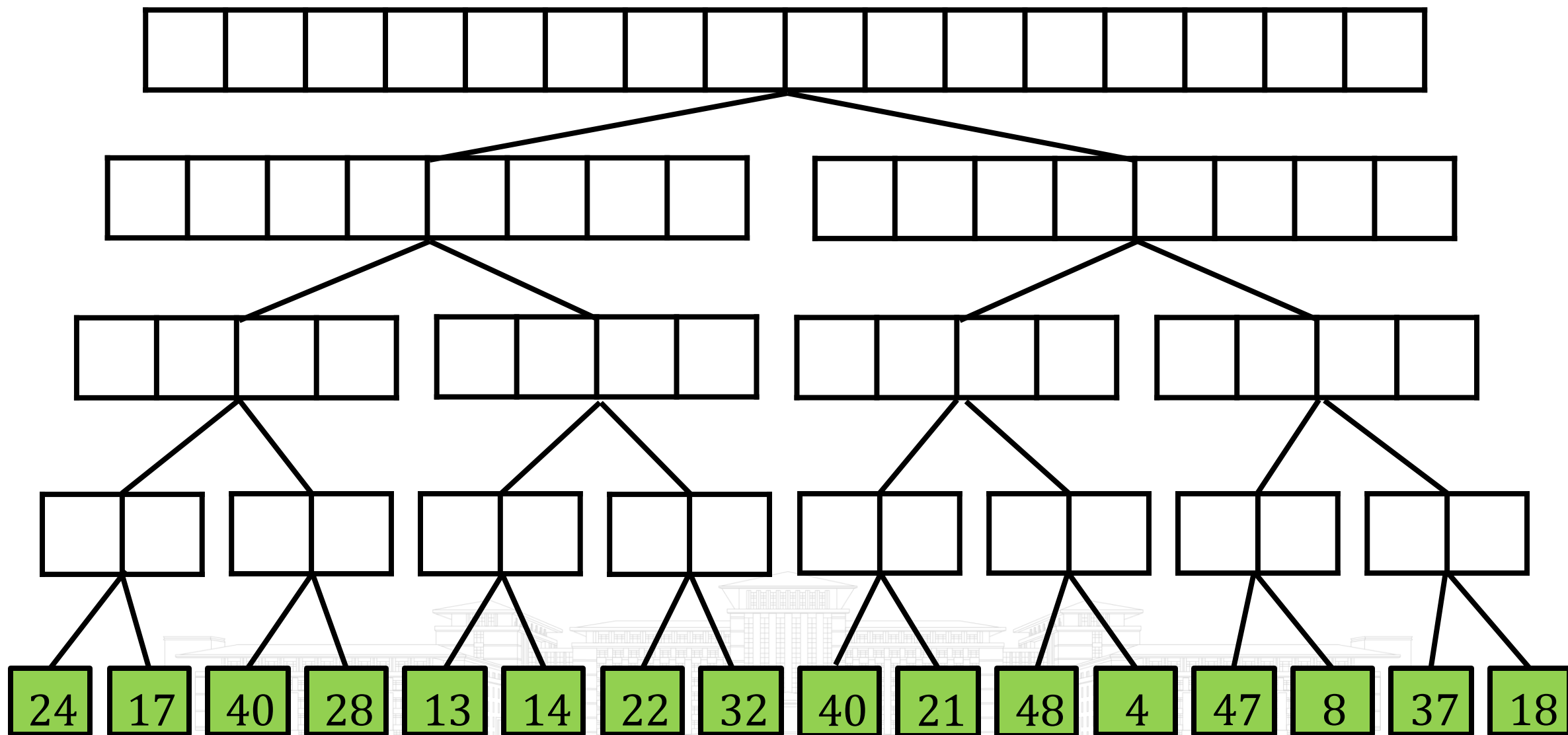
分解子问题



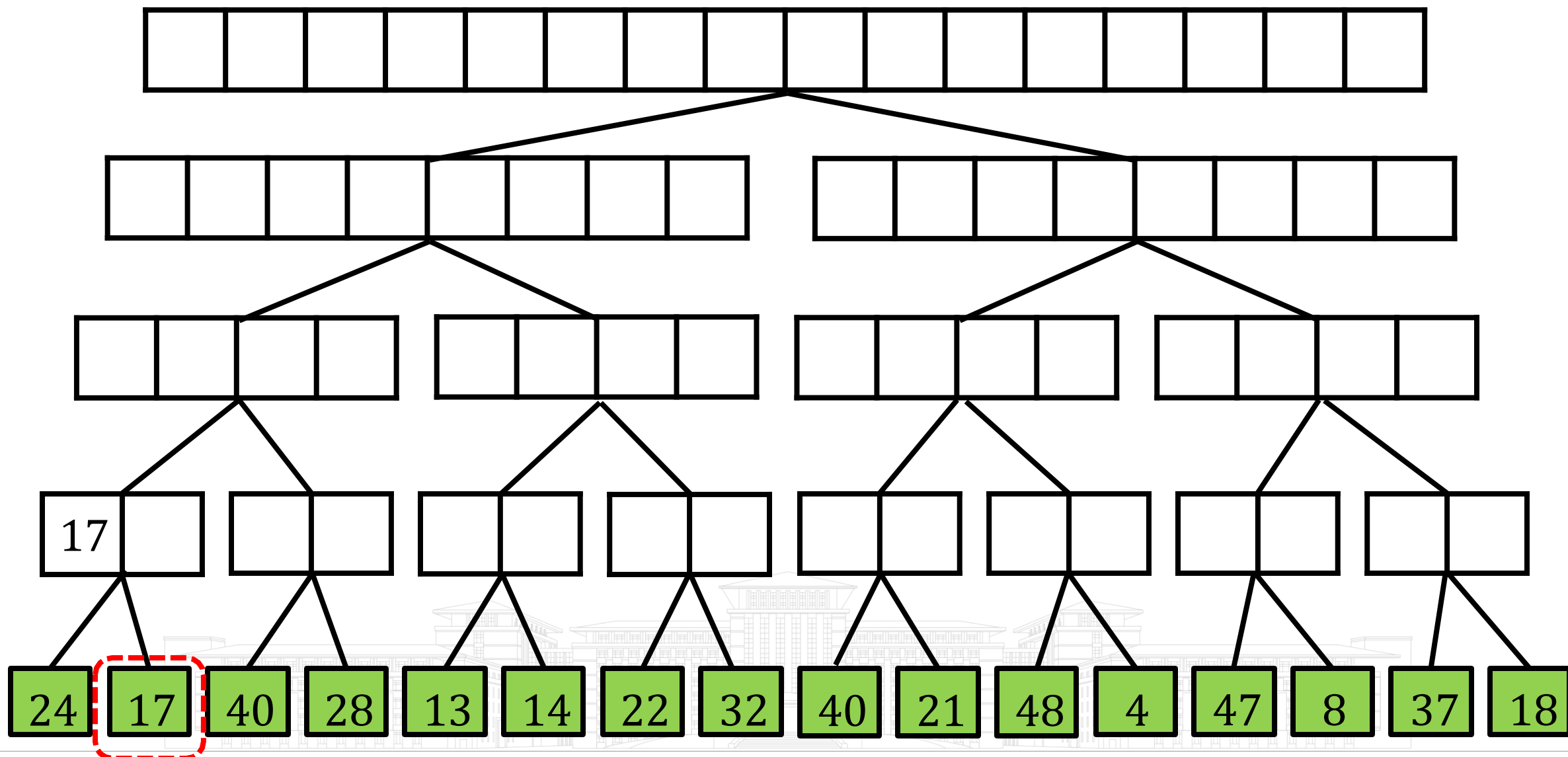
分解子问题



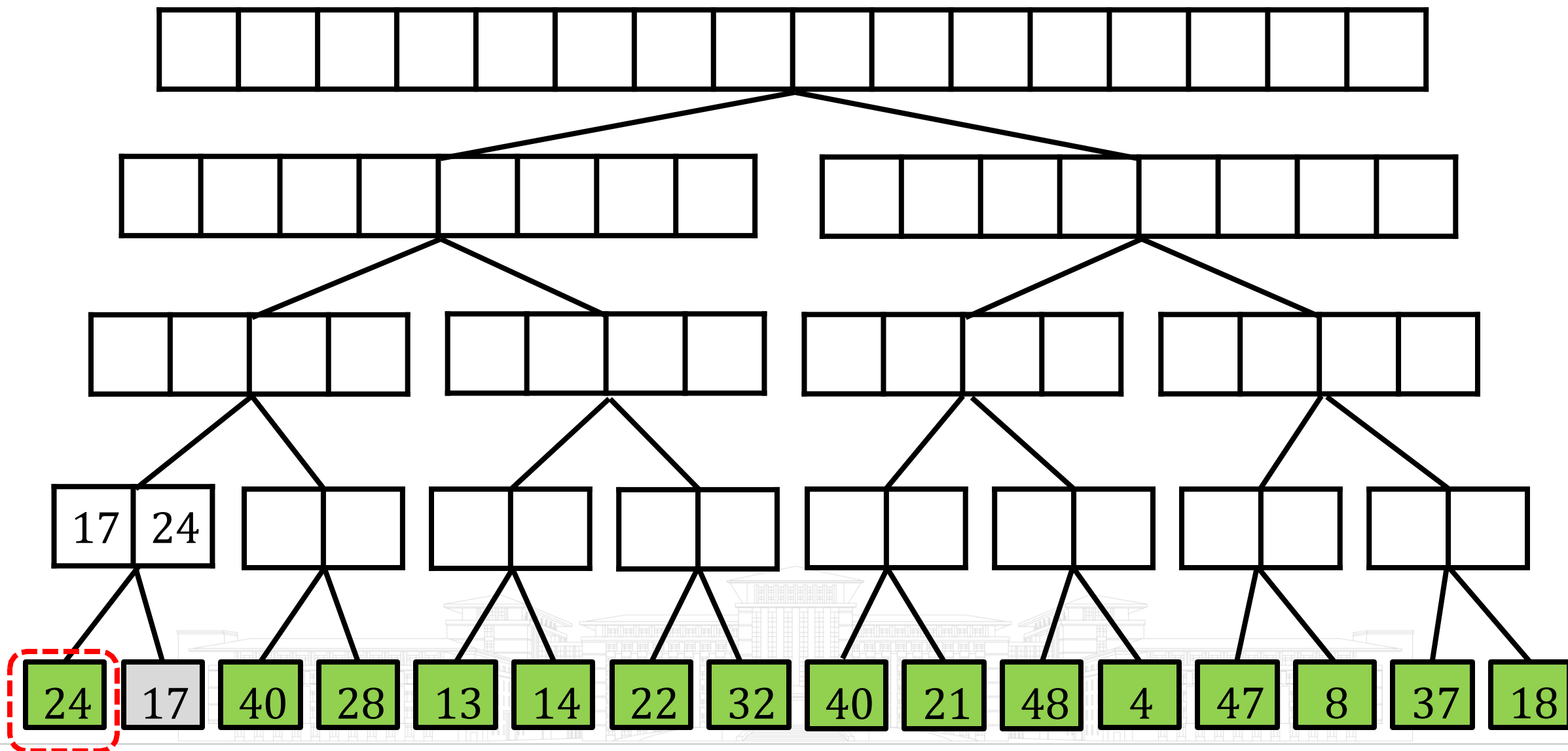
两两合并



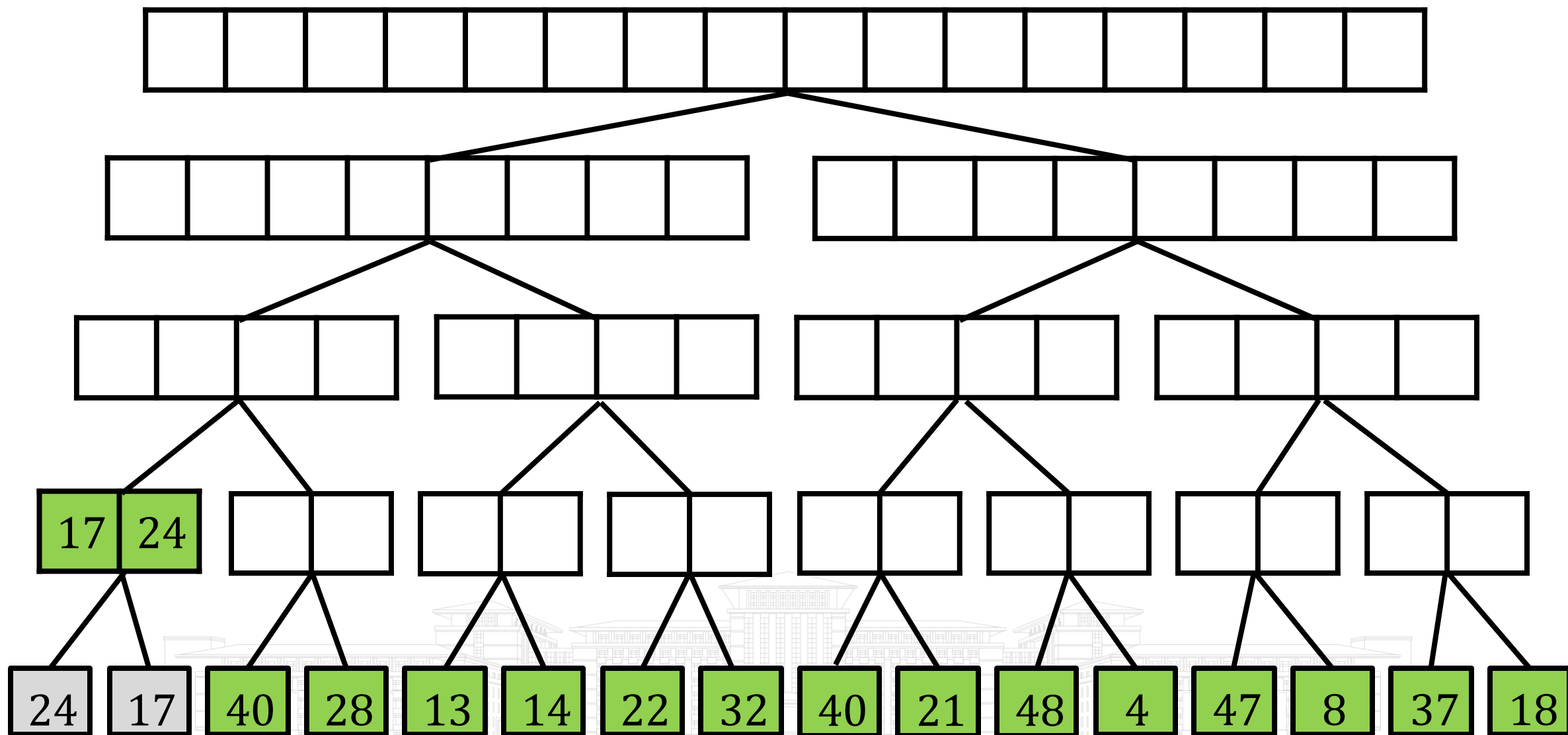
两两合并



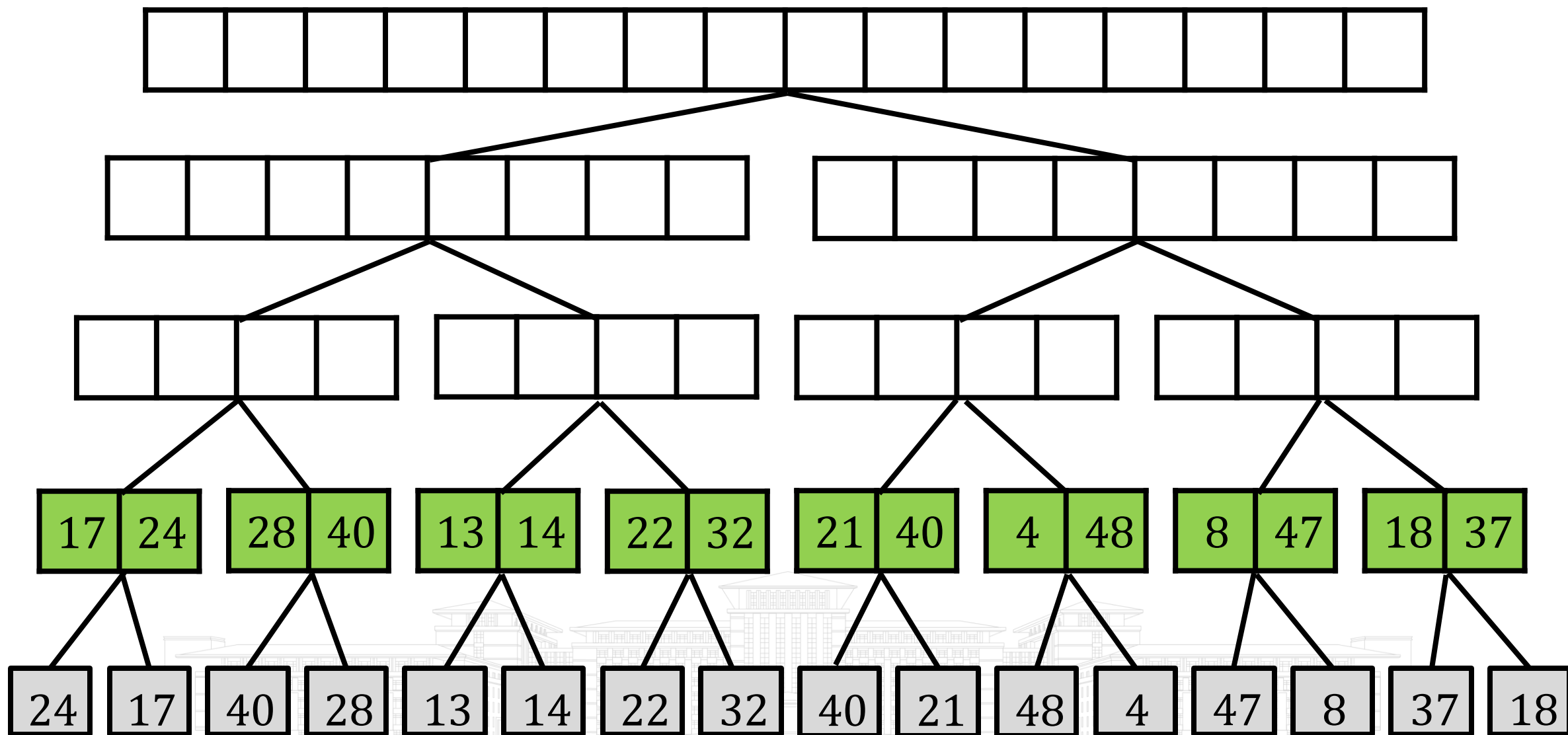
两两合并



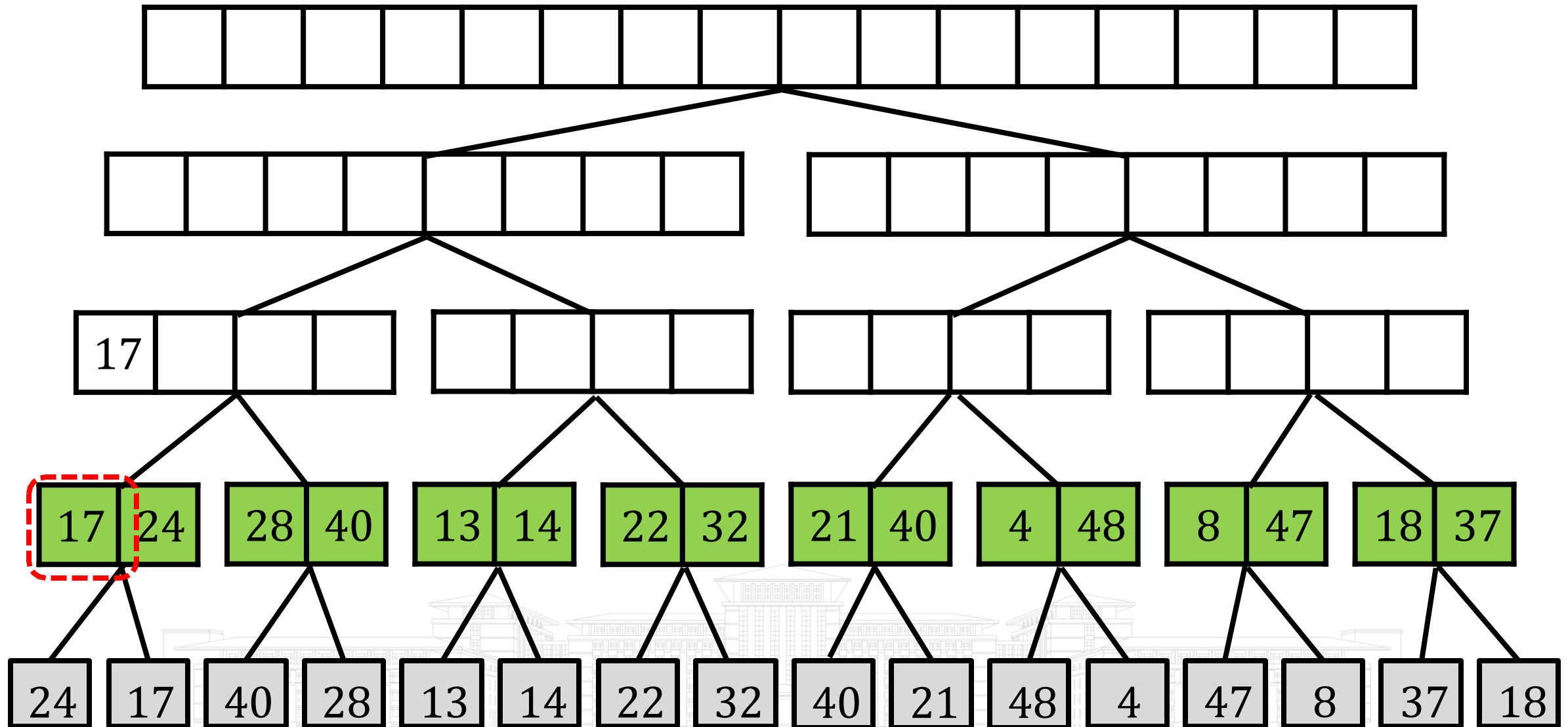
两两合并



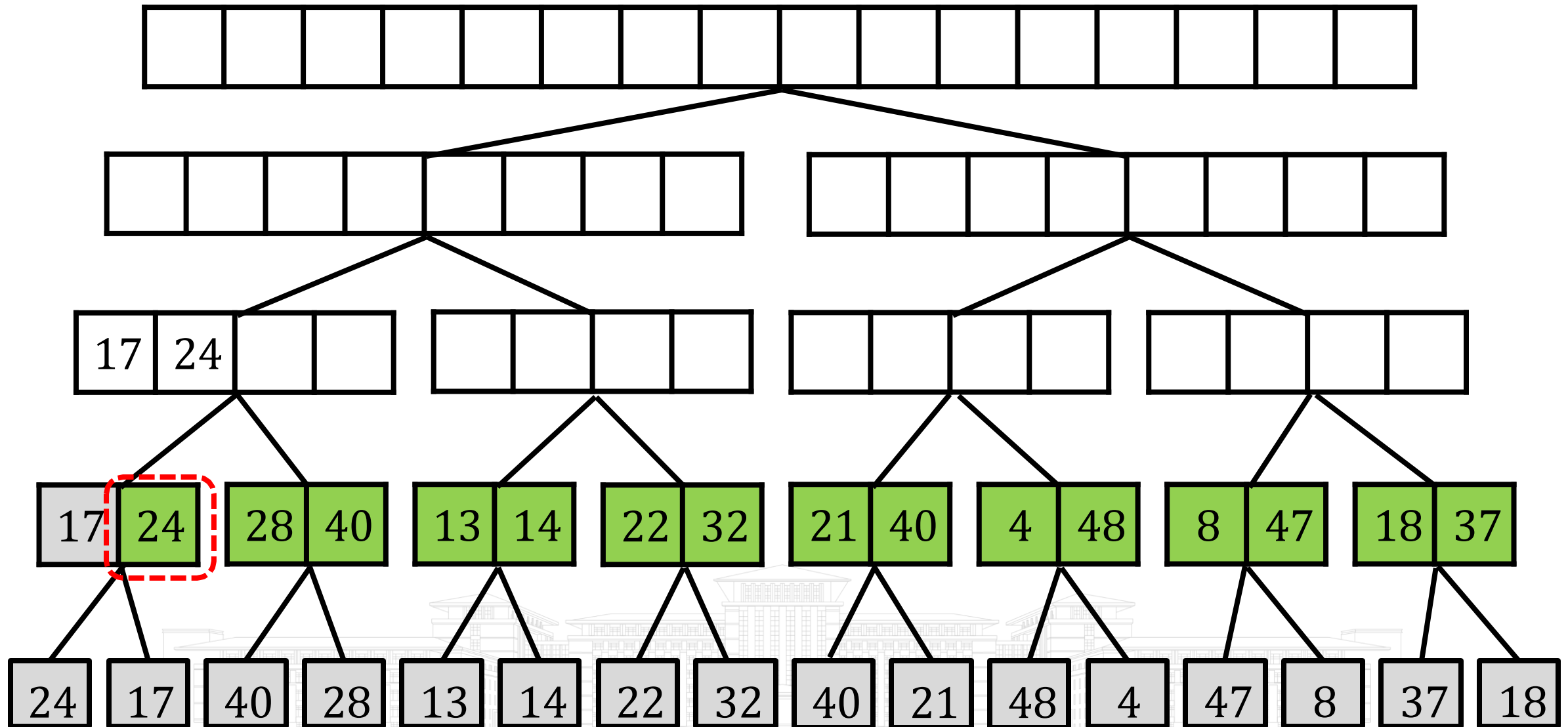
两两合并



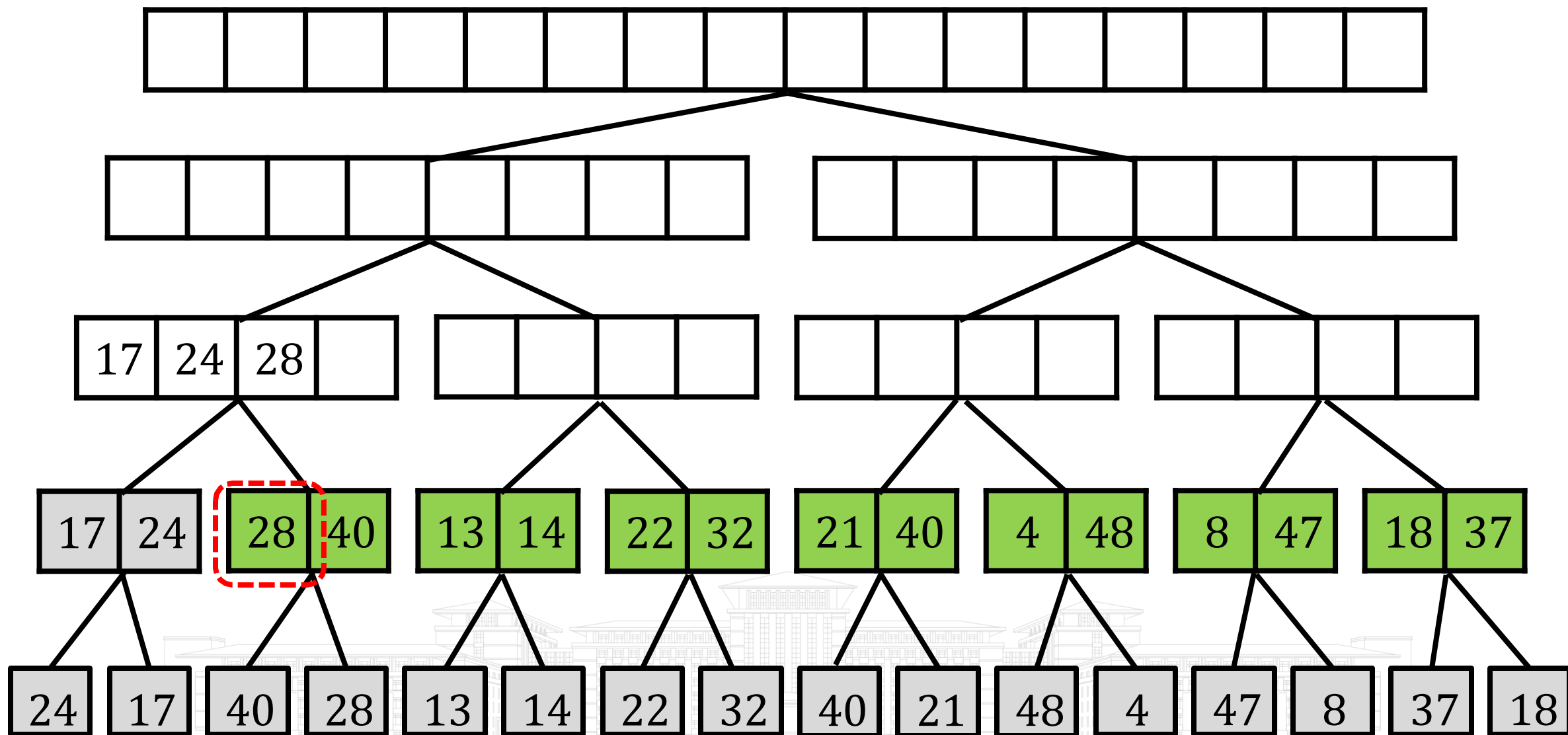
两两合并



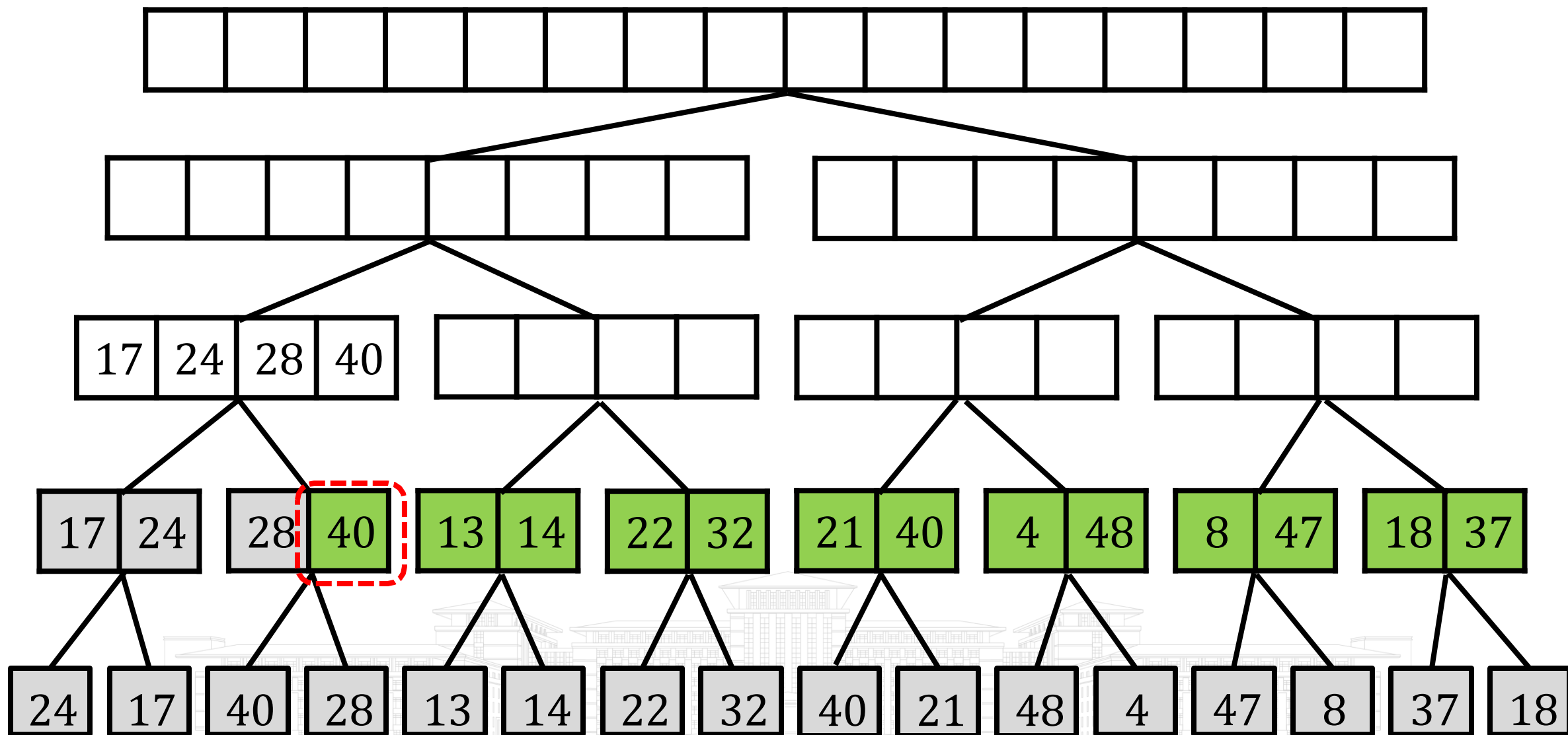
两两合并



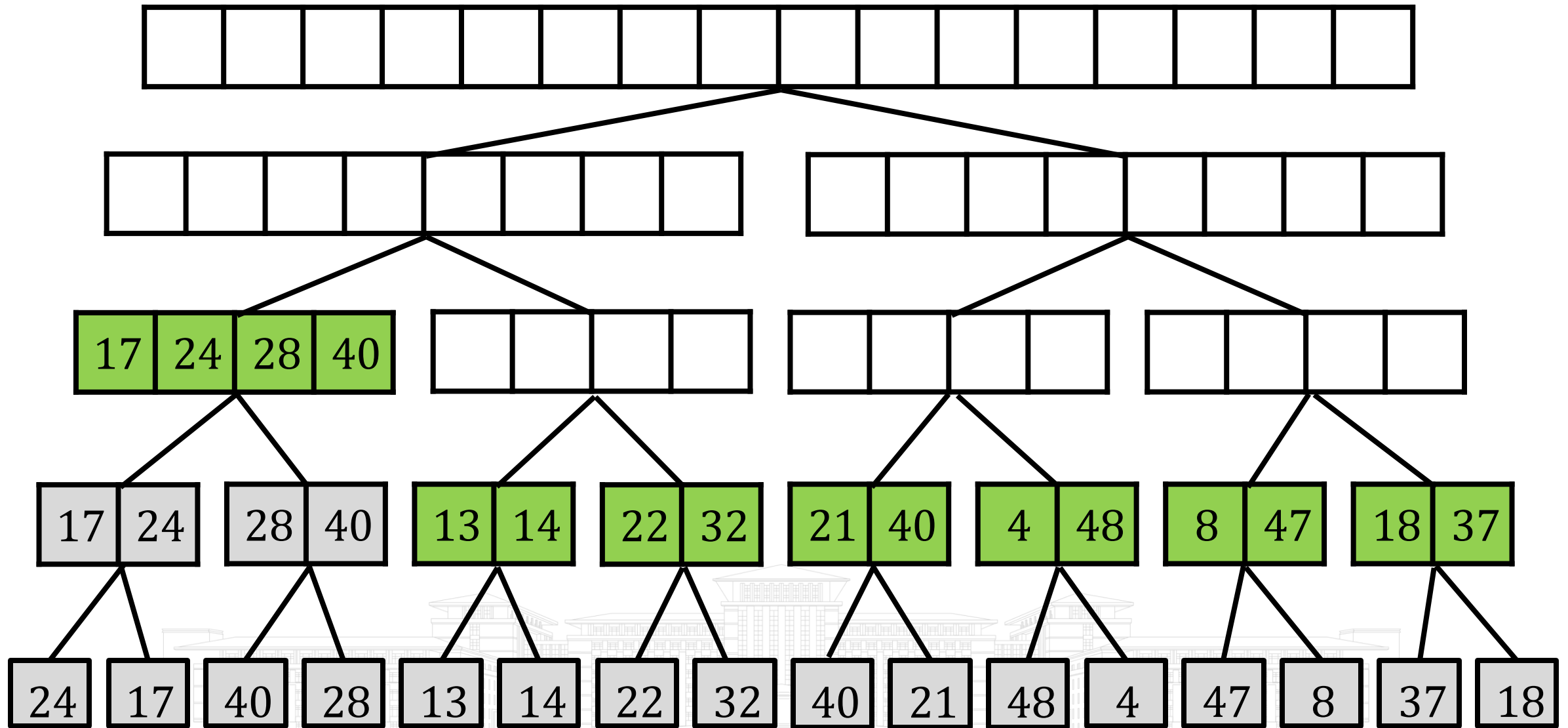
两两合并



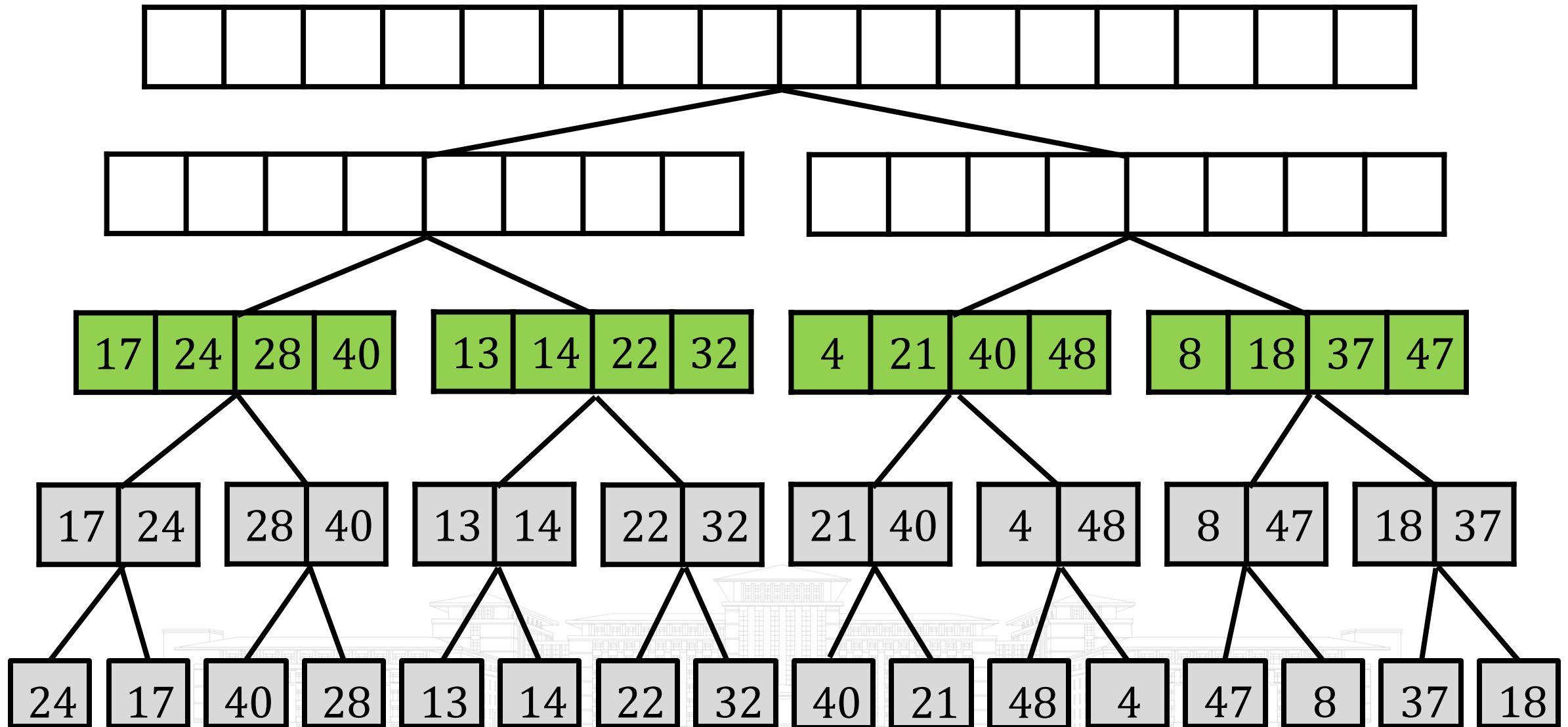
两两合并



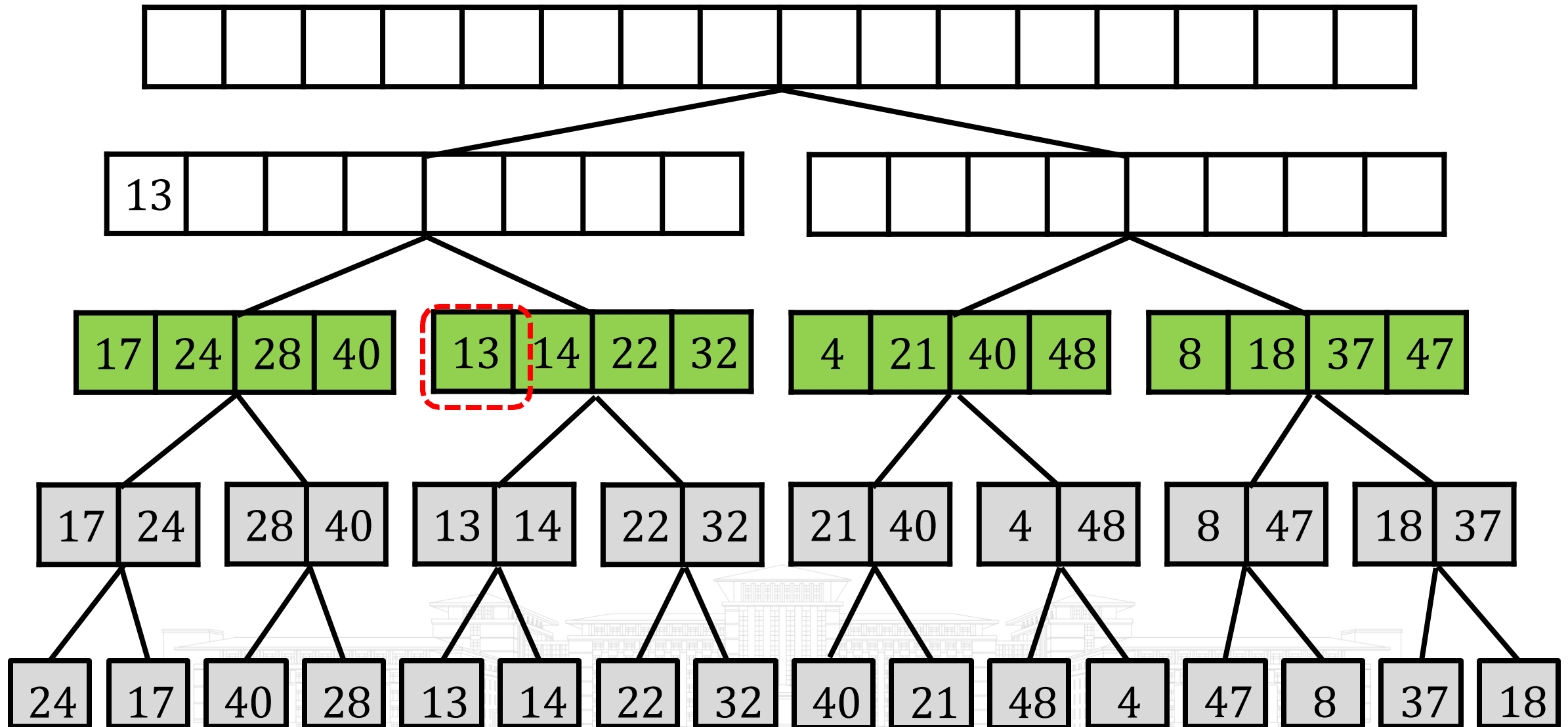
两两合并



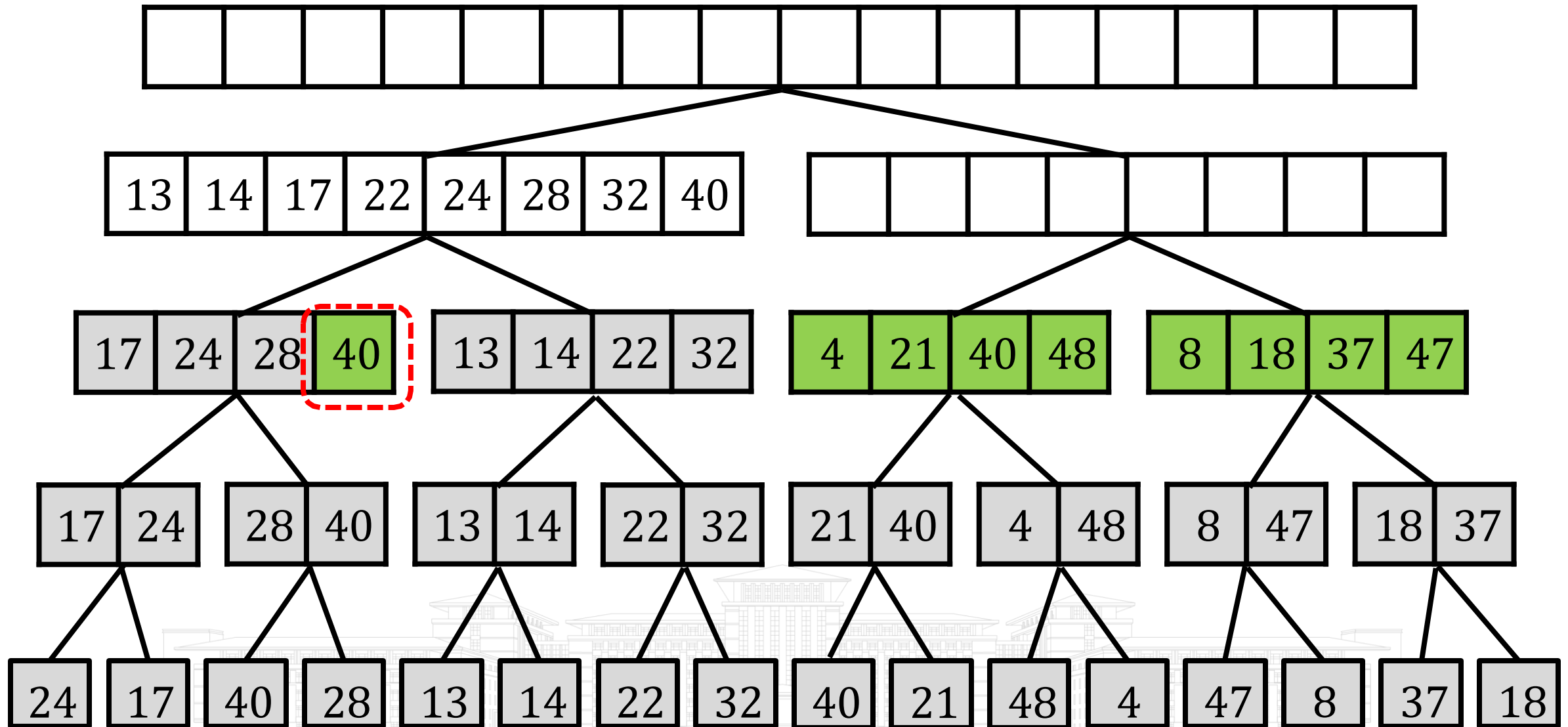
两两合并



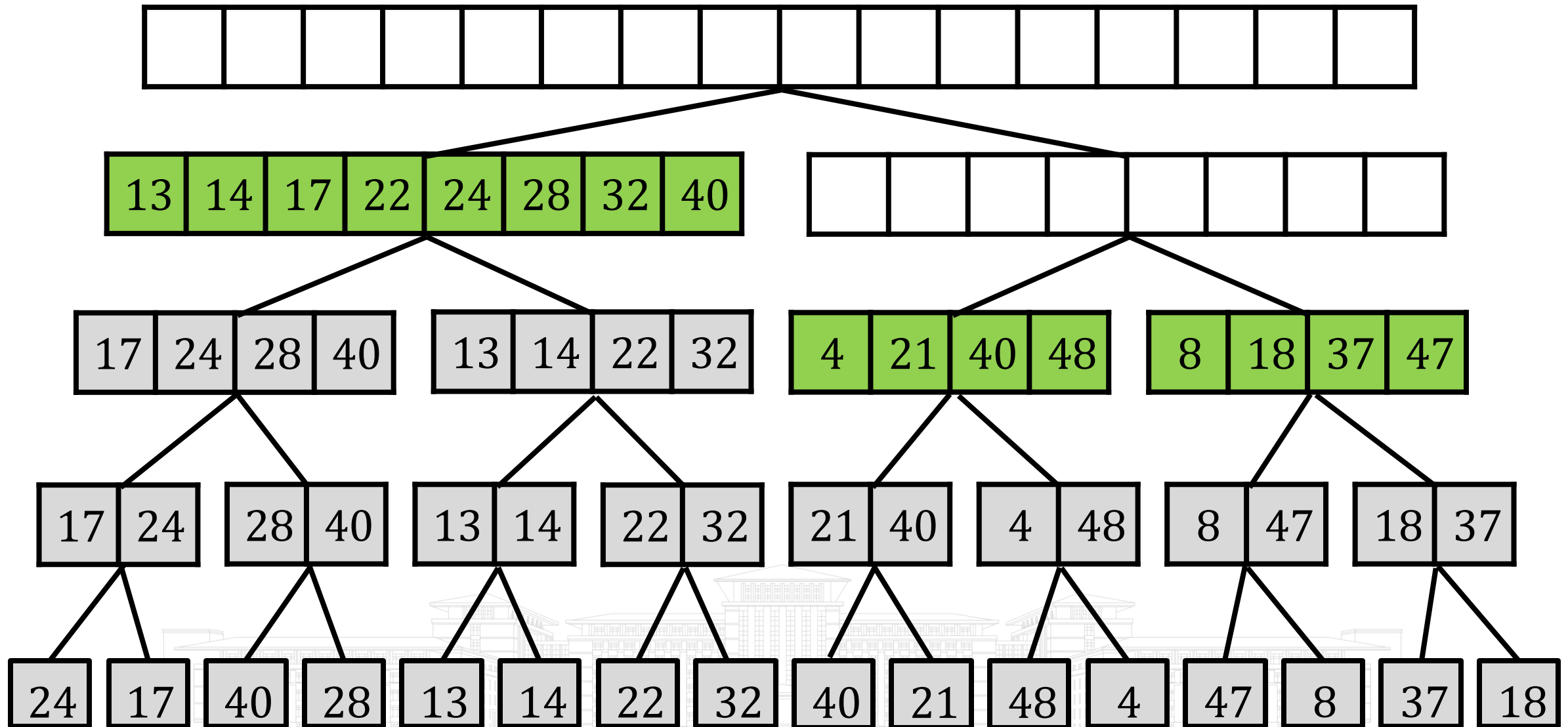
两两合并



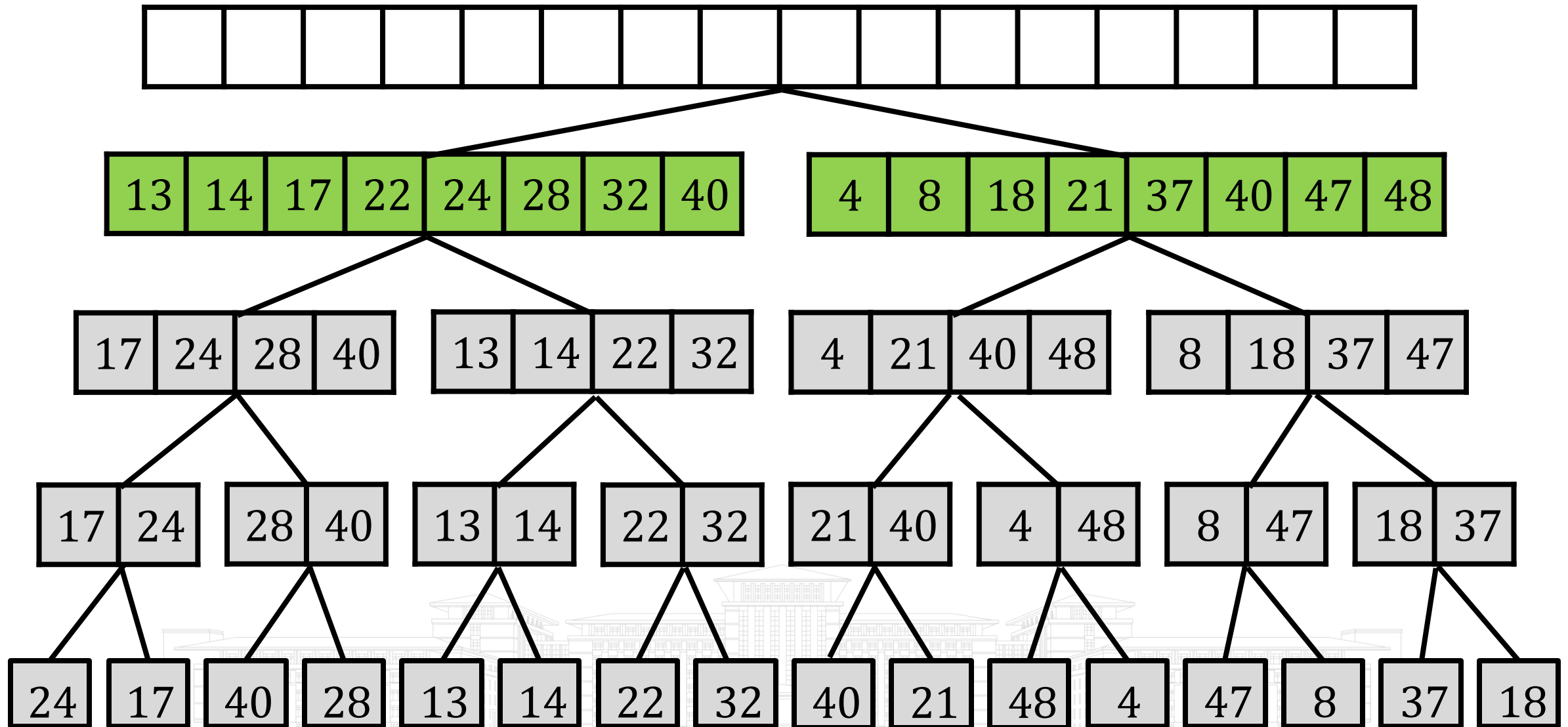
两两合并



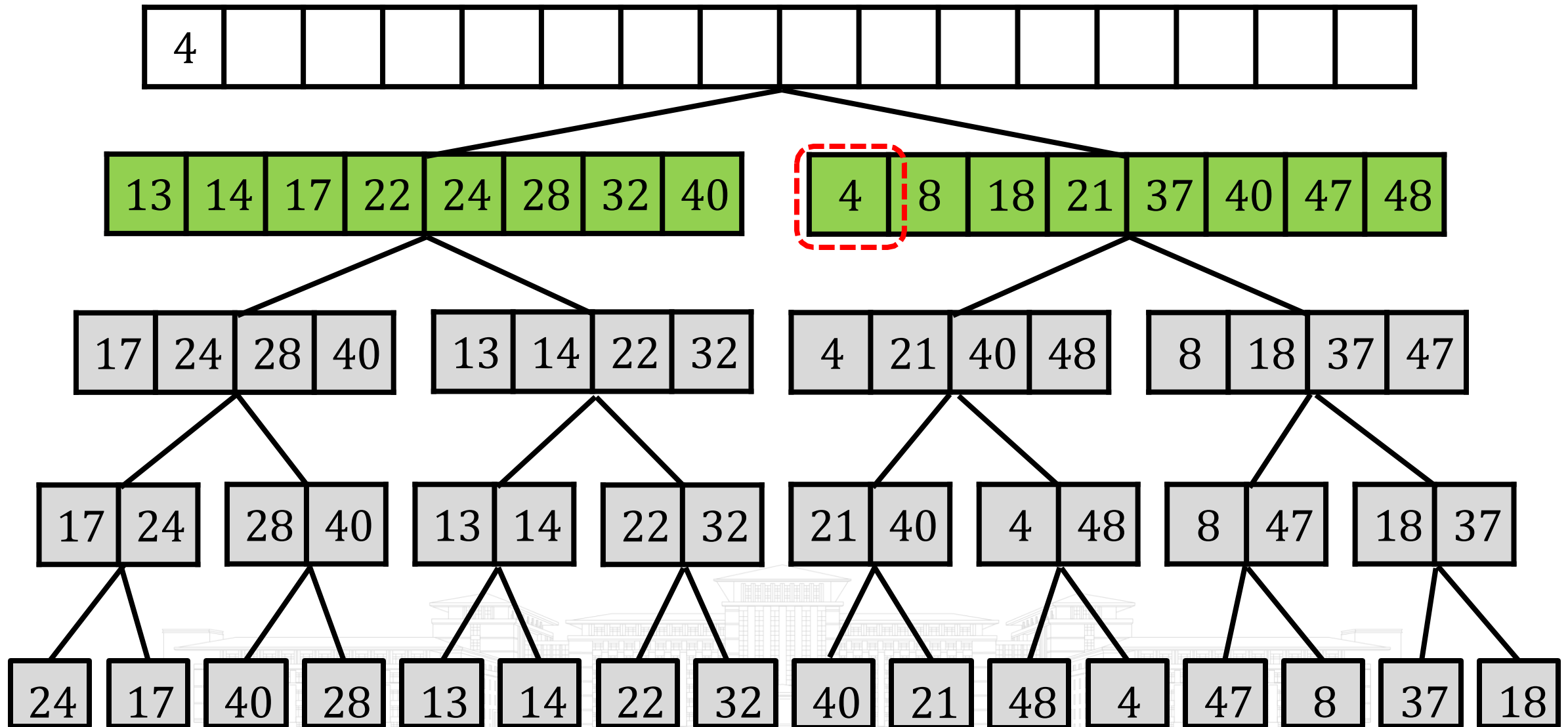
两两合并



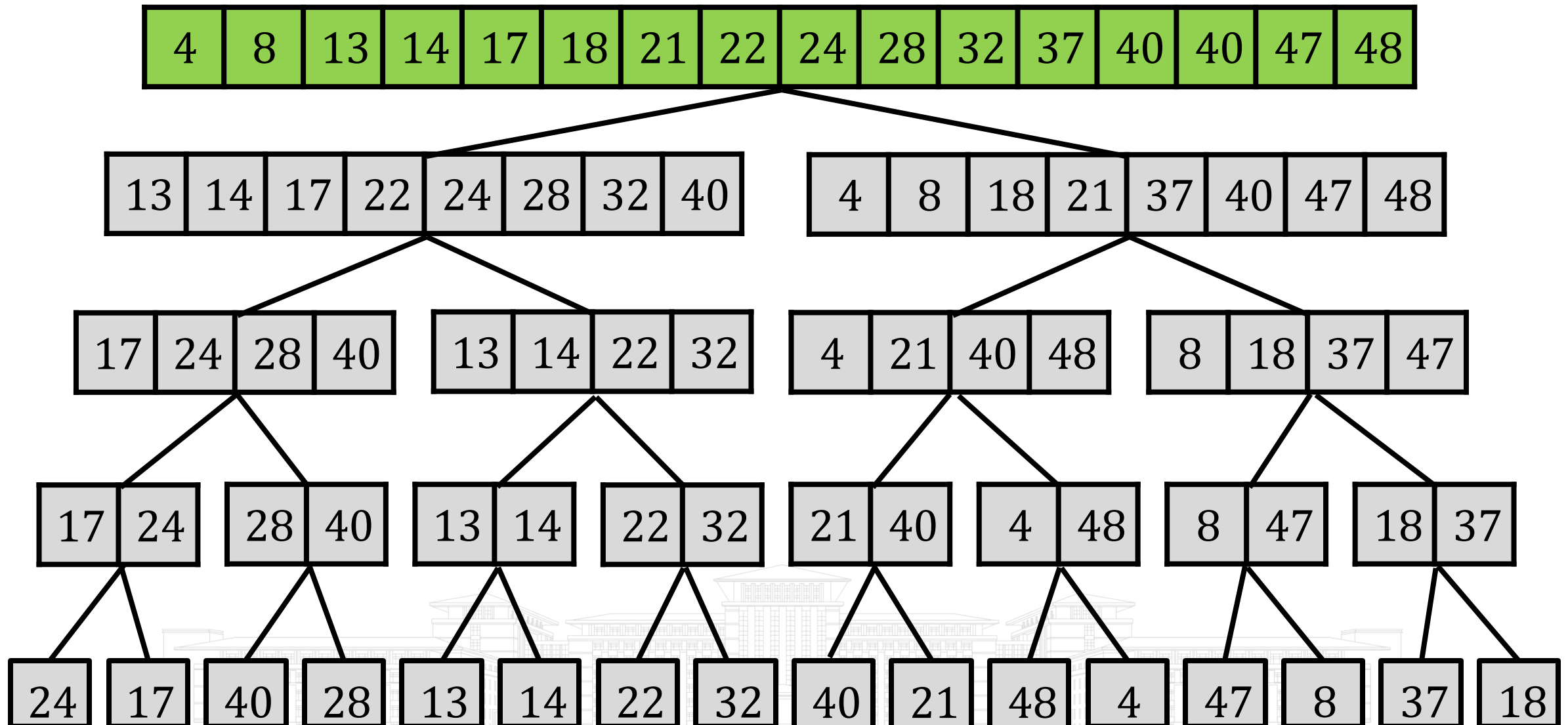
两两合并



两两合并



两两合并



归并排序

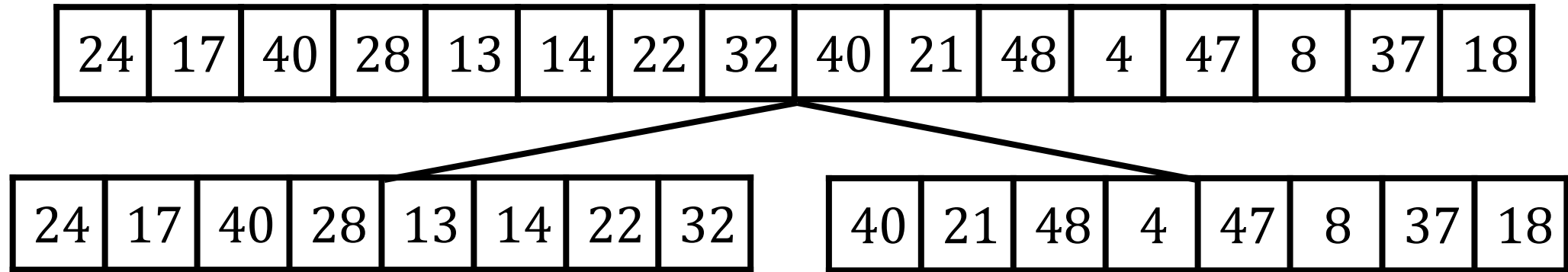
● 1945年，冯 • 诺伊曼提出归并排序



现代计算机之父
约翰 • 冯 • 诺伊曼
John von Neumann



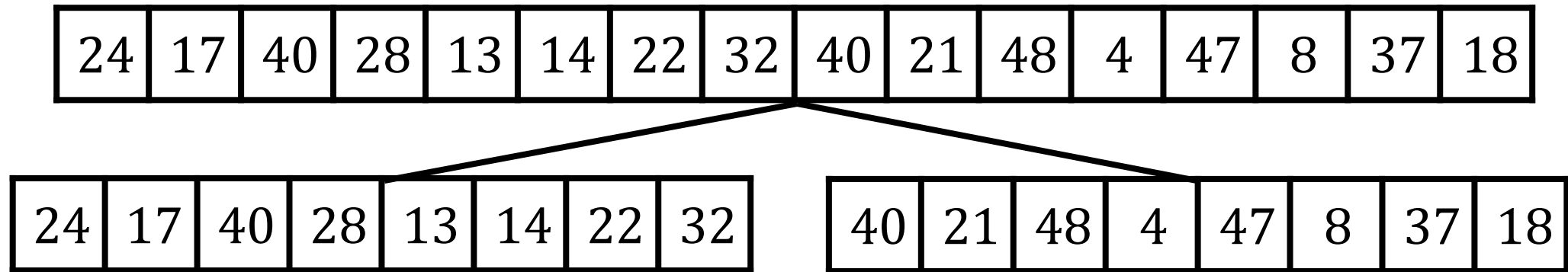
归并排序



● 算法流程



归并排序



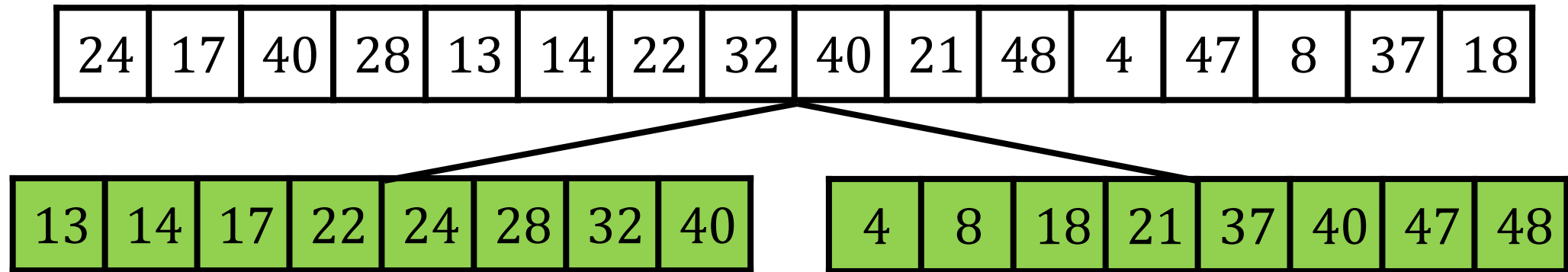
● 算法流程

- 将数组 $A[1..n]$ 排序问题 **分解** 为 $A[1..\lfloor \frac{n}{2} \rfloor]$ 和 $A[\lfloor \frac{n}{2} \rfloor + 1..n]$

分解原问题



归并排序



● 算法流程

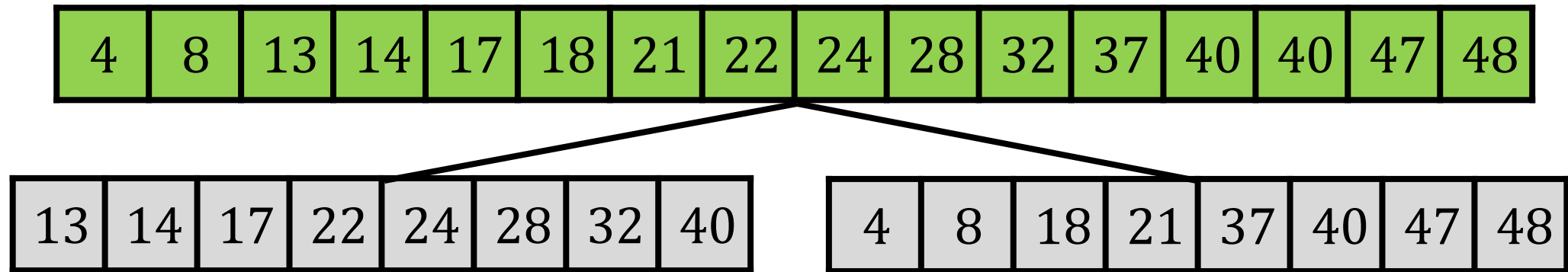
- 将数组 $A[1..n]$ 排序问题**分解**为 $A[1..\lfloor \frac{n}{2} \rfloor]$ 和 $A[\lfloor \frac{n}{2} \rfloor + 1..n]$
- **递归解决**子问题得到两个有序的子数组

分解原问题

解决子问题



归并排序



● 算法流程

- 将数组 $A[1..n]$ 排序问题**分解**为 $A[1..\lfloor \frac{n}{2} \rfloor]$ 和 $A[\lfloor \frac{n}{2} \rfloor + 1..n]$
- **递归解决**子问题得到两个有序的子数组
- 将两个有序子数组**合并**为一个有序数组

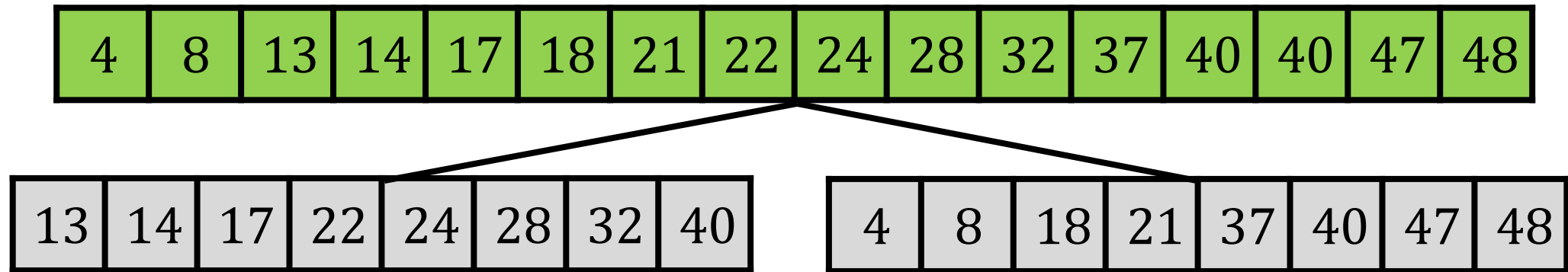
分解原问题

解决子问题

合并问题解

归并排序：分解数组，递归求解，合并排序

归并排序



● 算法流程

- 将数组 $A[1..n]$ 排序问题**分解**为 $A[1..\lfloor \frac{n}{2} \rfloor]$ 和 $A[\lfloor \frac{n}{2} \rfloor + 1..n]$
- **递归解决**子问题得到两个有序的子数组
- 将两个有序子数组**合并**为一个有序数组

分解原问题

解决子问题

合并问题解

归并排序：分解数组，递归求解，合并排序

分而治之

归并排序：伪代码

初始调用：MergeSort(A,1,n)

输入：数组 $A[1..n]$, 数组下标 $left, right$

输出：递增数组 $A[left..right]$

if $left \geq right$ **then**

 | **return** $A[left..right]$

end

$mid \leftarrow \lfloor \frac{left+right}{2} \rfloor$

MergeSort($A, left, mid$)

MergeSort($A, mid + 1, right$)

Merge($A, left, mid, right$)

return $A[left..right]$



归并排序：伪代码

初始调用：MergeSort(A,1,n)

输入：数组 $A[1..n]$, 数组下标 $left, right$

输出：递增数组 $A[left..right]$

```
if  $left \geq right$  then  
| return  $A[left..right]$   
end
```

递归终止：仅有一个元素

$mid \leftarrow \lfloor \frac{left+right}{2} \rfloor$

MergeSort($A, left, mid$)

MergeSort($A, mid + 1, right$)

Merge($A, left, mid, right$)

return $A[left..right]$



归并排序：伪代码

初始调用：MergeSort(A,1,n)

输入：数组 $A[1..n]$, 数组下标 $left, right$

输出：递增数组 $A[left..right]$

if $left \geq right$ then

 | return $A[left..right]$

end

$mid \leftarrow \lfloor \frac{left+right}{2} \rfloor$

MergeSort($A, left, mid$)

MergeSort($A, mid + 1, right$)

Merge($A, left, mid, right$)

return $A[left..right]$

计算子问题规模



归并排序：伪代码

初始调用：MergeSort(A,1,n)

输入：数组 $A[1..n]$, 数组下标 $left, right$

输出：递增数组 $A[left..right]$

if $left \geq right$ then

 | return $A[left..right]$

end

$mid \leftarrow \lfloor \frac{left+right}{2} \rfloor$

MergeSort($A, left, mid$)

MergeSort($A, mid + 1, right$)

Merge($A, left, mid, right$)

return $A[left..right]$

递归求解子问题



归并排序：伪代码

初始调用：MergeSort(A,1,n)

输入：数组 $A[1..n]$, 数组下标 $left, right$

输出：递增数组 $A[left..right]$

if $left \geq right$ then

 | return $A[left..right]$

end

$mid \leftarrow \lfloor \frac{left+right}{2} \rfloor$

MergeSort($A, left, mid$)

MergeSort($A, mid + 1, right$)

Merge($A, left, mid, right$)

return $A[left..right]$

合并子问题解



归并排序：伪代码

● Merge($A, left, mid, right$)

输入: 数组 $A[1..n]$, 数组下标 $left, mid, right$

输出: 递增数组 $A[left..right]$

$A'[left..right] \leftarrow A[left..right]$

$i \leftarrow left, j \leftarrow mid + 1, k \leftarrow 0$

while $i \leq mid$ **and** $j \leq right$ **do**

if $A'[i] \leq A'[j]$ **then**

$A[left + k] \leftarrow A'[i]$

$k \leftarrow k + 1, i \leftarrow i + 1$

end

else

$A[left + k] \leftarrow A'[j]$

$k \leftarrow k + 1, j \leftarrow j + 1$

end

end

if $i \leq mid$ **then**

$A[left + k..right] \leftarrow A'[i..mid]$

end

else

$A[left + k..right] \leftarrow A'[j..right]$

end

return $A[left..right]$

初始化

归并排序：伪代码

● Merge($A, left, mid, right$)

输入: 数组 $A[1..n]$, 数组下标 $left, mid, right$

输出: 递增数组 $A[left..right]$

$A'[left..right] \leftarrow A[left..right]$

$i \leftarrow left, j \leftarrow mid + 1, k \leftarrow 0$

while $i \leq mid$ **and** $j \leq right$ **do**

if $A'[i] \leq A'[j]$ **then**

$A[left + k] \leftarrow A'[i]$

$k \leftarrow k + 1, i \leftarrow i + 1$

end

else

$A[left + k] \leftarrow A'[j]$

$k \leftarrow k + 1, j \leftarrow j + 1$

end

end

if $i \leq mid$ **then**

$A[left + k..right] \leftarrow A'[i..mid]$

end

else

$A[left + k..right] \leftarrow A'[j..right]$

end

return $A[left..right]$

遍历子数组，进行合并

归并排序：伪代码

● Merge($A, left, mid, right$)

输入: 数组 $A[1..n]$, 数组下标 $left, mid, right$

输出: 递增数组 $A[left..right]$

$A'[left..right] \leftarrow A[left..right]$

$i \leftarrow left, j \leftarrow mid + 1, k \leftarrow 0$

while $i \leq mid$ and $j \leq right$ **do**

if $A'[i] \leq A'[j]$ **then**

$A[left + k] \leftarrow A'[i]$

$k \leftarrow k + 1, i \leftarrow i + 1$

end

else

$A[left + k] \leftarrow A'[j]$

$k \leftarrow k + 1, j \leftarrow j + 1$

end

end

if $i \leq mid$ **then**

$A[left + k..right] \leftarrow A'[i..mid]$

end

else

$A[left + k..right] \leftarrow A'[j..right]$

end

return $A[left..right]$

添加剩余元素保证有序

归并排序：伪代码

● Merge($A, left, mid, right$)

输入: 数组 $A[1..n]$, 数组下标 $left, mid, right$

输出: 递增数组 $A[left..right]$

$A'[left..right] \leftarrow A[left..right]$

$i \leftarrow left, j \leftarrow mid + 1, k \leftarrow 0$

while $i \leq mid$ and $j \leq right$ **do**

if $A'[i] \leq A'[j]$ **then**

$A[left + k] \leftarrow A'[i]$

$k \leftarrow k + 1, i \leftarrow i + 1$

end

else

$A[left + k] \leftarrow A'[j]$

$k \leftarrow k + 1, j \leftarrow j + 1$

end

end

if $i \leq mid$ **then**

$A[left + k..right] \leftarrow A'[i..mid]$

end

else

$A[left + k..right] \leftarrow A'[j..right]$

end

return $A[left..right]$

时间复杂度： $O(n)$

归并排序：复杂度分析

- $T(n)$: 完成MergeSort(A,1,n) 的运行时间
 - 为便于分析, 假设 n 是2的幂



归并排序：复杂度分析

- $T(n)$: 完成MergeSort($A, 1, n$) 的运行时间
 - 为便于分析, 假设 n 是2的幂

输入: 数组 $A[1..n]$, 数组下标 $left, right$

输出: 递增数组 $A[left..right]$

if $left \geq right$ then

 | return $A[left..right]$

end

$mid \leftarrow \lfloor \frac{left+right}{2} \rfloor$

MergeSort($A, left, mid$)

MergeSort($A, mid + 1, right$)

Merge($A, left, mid, right$)

return $A[left..right]$

} $O(1)$

→ $O(T(n)/2)$

→ $O(T(n)/2)$

→ $O(n)$



归并排序：复杂度分析

- $T(n)$: 完成MergeSort(A,1,n) 的运行时间
 - 为便于分析, 假设n是2的幂

输入: 数组 $A[1..n]$, 数组下标 $left, right$

输出: 递增数组 $A[left..right]$

if $left \geq right$ then

 | return $A[left..right]$

end

$mid \leftarrow \lfloor \frac{left+right}{2} \rfloor$

MergeSort($A, left, mid$)

MergeSort($A, mid + 1, right$)

Merge($A, left, mid, right$)

return $A[left..right]$

} $O(1)$

→ $O(T(n)/2)$

→ $O(T(n)/2)$

→ $O(n)$

$$T(n) = \begin{cases} 2T(n/2) + O(n) & \text{if } n > 1 \\ O(1) & \text{if } n = 1 \end{cases}$$

归并排序：复杂度分析

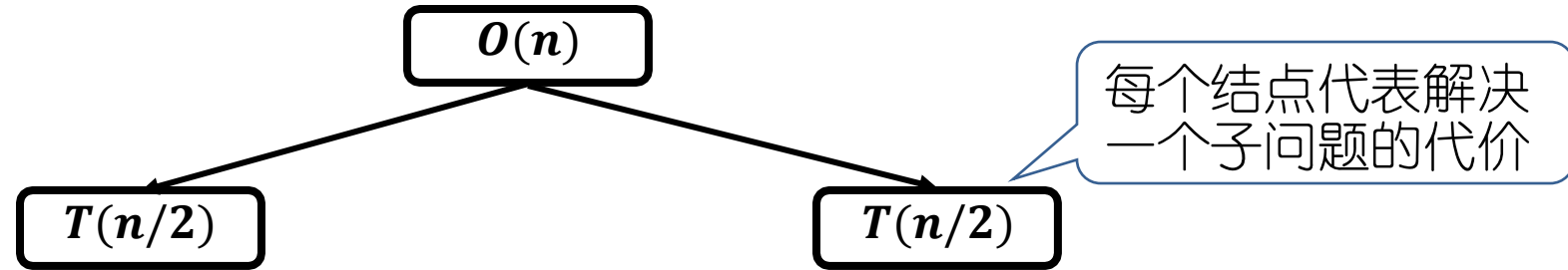
- 递归树法：用树表示递归
- $$T(n) = \begin{cases} 2T(n/2) + O(n) & \text{if } n > 1 \\ O(1) & \text{if } n = 1 \end{cases}$$

$T(n)$



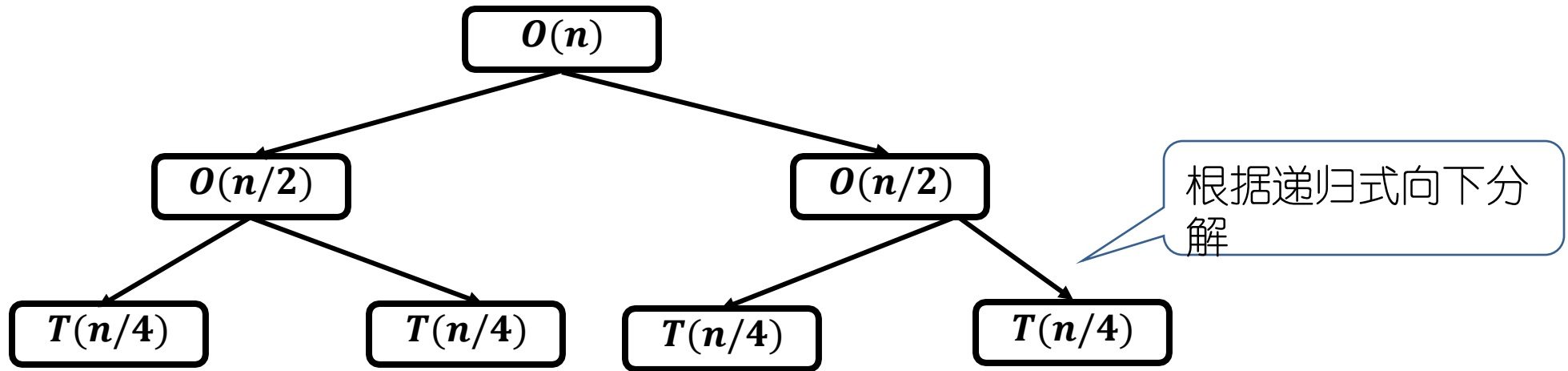
归并排序：复杂度分析

- 递归树法：用树表示递归
- $$T(n) = \begin{cases} 2T(n/2) + O(n) & \text{if } n > 1 \\ O(1) & \text{if } n = 1 \end{cases}$$



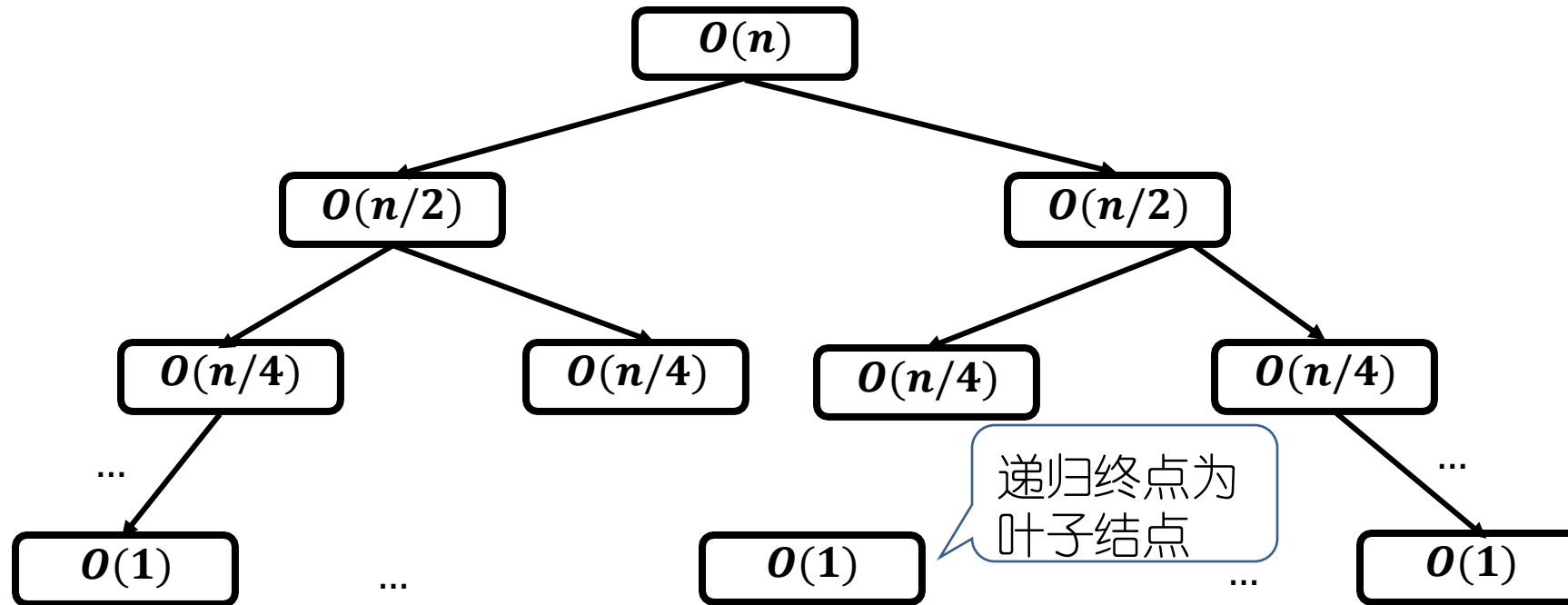
归并排序：复杂度分析

- 递归树法：用树表示递归
- $$T(n) = \begin{cases} 2T(n/2) + O(n) & \text{if } n > 1 \\ O(1) & \text{if } n = 1 \end{cases}$$



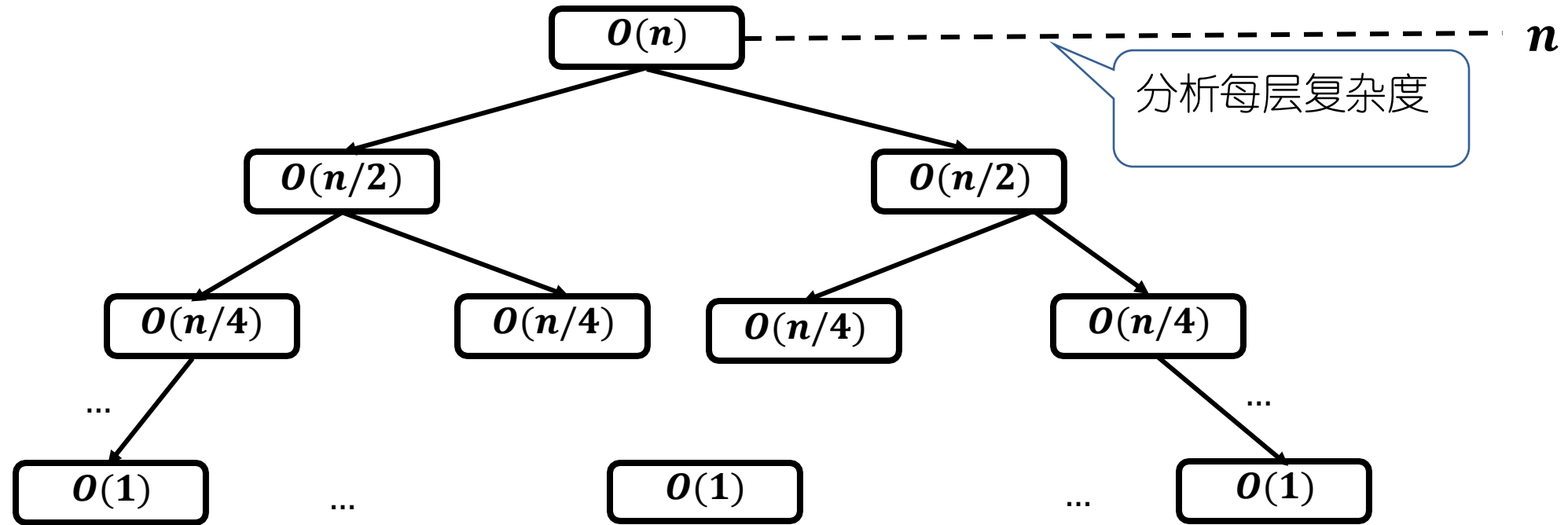
归并排序：复杂度分析

- 递归树法：用树表示递归
- $$T(n) = \begin{cases} 2T(n/2) + O(n) & \text{if } n > 1 \\ O(1) & \text{if } n = 1 \end{cases}$$



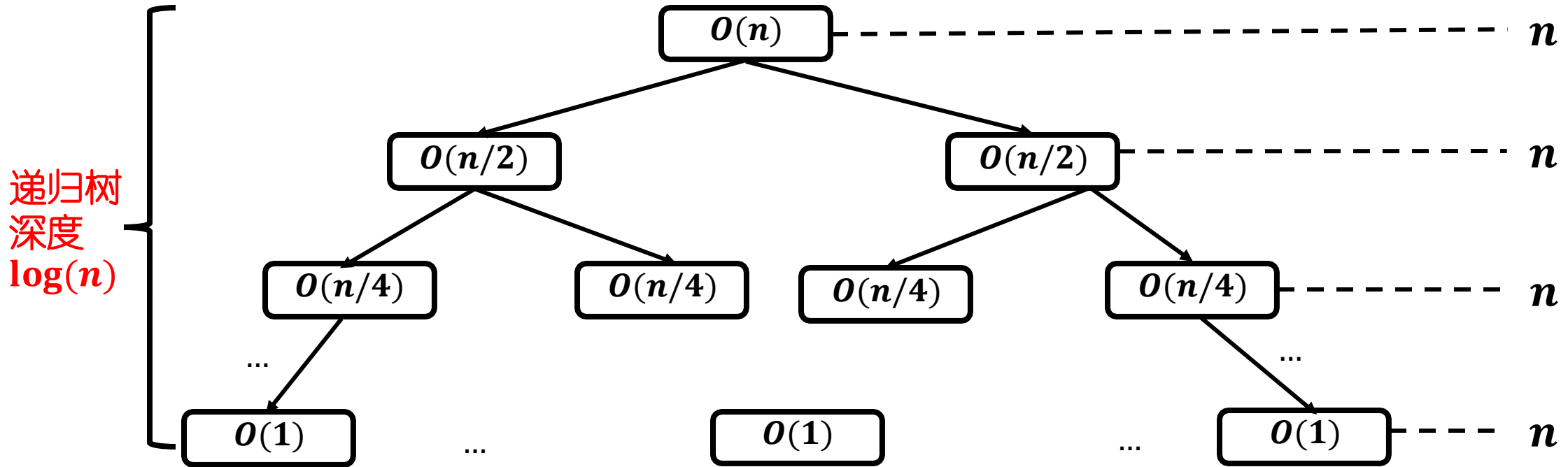
归并排序：复杂度分析

- 递归树法：用树表示递归
- $$T(n) = \begin{cases} 2T(n/2) + O(n) & \text{if } n > 1 \\ O(1) & \text{if } n = 1 \end{cases}$$



归并排序：复杂度分析

- 递归树法：用树表示递归
- $$T(n) = \begin{cases} 2T(n/2) + O(n) & \text{if } n > 1 \\ O(1) & \text{if } n = 1 \end{cases}$$



$$T(n) = O(n \log n)$$



归并排序算法的优化

- 时间：递归处理消耗了很多时间。
 - 当子集合的元素个数很少时，采用直接插入算法减少时间消耗。
- 空间：辅助数组A增加了算法空间，每次调用Merge时，A中结果复制到A'中，消耗了一部分时间。
 - 用一个链表LINK(1:n)代替A，LINK中元素为A'中元素的指针，它指向下一个元素所在的下标位置。

思考：能否采用自底向上的设计方式来取消对系统栈空间的需要？



归并排序算法的优化

